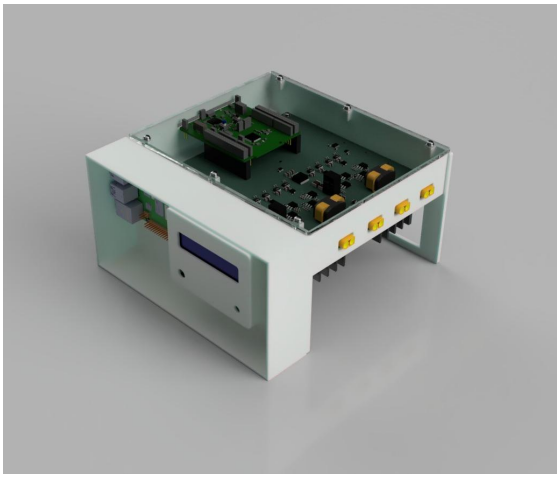
PROJECT DELIVERY REPORT

Controlling Differential Power Processors in Solar Powered Applications

THE DESIGN AND IMPLEMENTATION OF MODERN STATE-SPACE METHODS TO SERIAL PARTIAL POWER CONVERTERS



STUDENT ENGINEER
Jared Saunders

n11034459

SUPERVISOR
Assoc Prof Geoff Walker

School of Electrical Engineering & Robotics

Version	Date	Author	Changes / Comments
1.0	17/04/26	Jared Saunders	Project Proposal
2.0	29/05/26	Jared Saunders	Semester 1 Progress Report
3.0	23/10/26	Jared Saunders	Project Delivery Report

Executive Summary

This project designed, built and field-tested a controller for the 'Power Shuffler', and showed on a real solar string that it recovers power a shaded string would otherwise lose. Shading one panel in a series string limits the current of the whole string, and the bypass diodes used to protect it simply discard the shaded panel's power. The shuffler is a serial partial power converter (S-PPC), a type of differential power processor, that instead moves the difference in power between series-connected modules to balance their voltages. A sister report covers the shuffler hardware; this report covers its control system, from the analytical model through to closed-loop validation on the bench and in the field.

Each shuffler is an interleaved buck-boost converter built from decoupling capacitors, a coupled inductor and two half-bridges. Each shuffler shares PV modules with its neighbours, so their dynamics are coupled and cannot be described by a set of single-input single-output transfer functions. The controller is therefore built on a multiple-input multiple-output state-space model with three states per shuffler (module-voltage difference, shuffled current and circulating current) and two inputs: common-mode duty (set by the two half-bridge duties together, which shuffles power between modules) and differential duty (their difference, which controls the circulating current). The development process followed a unique spin on a systems engineering approach. The analytical model set the controller architecture. A Simulink model of a four-module string validated these control loops, and its requirements drove the choice of sensors and microcontroller. The same C control core was then implemented inside a hardware-accurate QSPICE model, adding sensor noise and sampling delay, before going into the firmware unchanged. The final product is a single board with two shufflers, Hall-effect current sensors, power-monitor voltage sensors and an STM32G474RE microcontroller, housed in a 3D-printed enclosure with a Raspberry Pi 4B that streams telemetry to a timeseries database.

On the bench (NGE103 supply, DL3021 electronic load) the closed loop held the module-voltage differences within $\pm 0.010\,\mathrm{V}$ and the circulating current within $\pm 0.07\,\mathrm{A}$ across 22 settled test segments at 200–400 kHz. The 44 Hz telemetry is too slow to resolve the step response on hardware, so it is taken from the simulation models. For a 1 V step at 30 % gain and 200 kHz, QSPICE gives a rise time of 6.5 ms and a 2 % settling time of 11.9 ms with no overshoot, corroborated by the analytical model (7.6 ms and 13.6 ms). Of the switching frequencies tested, 300 kHz was the best compromise; 400 kHz gave marginal gains at higher losses with almost no change in response from gain increases.

The most significant result is from the solar test: three 80 W panels without bypass diodes in series into a 61 V constant-voltage load. With the top module shaded to almost zero current (a shirt over about a third of the panel), a plain series string would have delivered no power at all. With the shufflers running, the string delivered up to 99 W during the shading event and 10.6 Wh over 36 minutes of testing, nearly all of it energy that would otherwise have been lost.

The main limitation is scope: one board with two shufflers that are hard-wired together, so they cannot be driven independently and boards cannot be stacked, which leaves scalability untested. The top-level INA239 (U10) does not work, so V_{M1} was inferred from the electronic load; the gains have to be re-derived in simulation whenever the switching frequency changes; and poor weather on the test day meant the closed loop was only partly exercised against real panel dynamics. The recommended next steps are a stackable board with independently driven shufflers, to test strings of more than three modules, and a study of other applications for S-PPCs that may offer more impact to industry than solar.

Abstract

Solar is one of the fastest-growing sources of electricity, yet a solar string is only as strong as its weakest panel. When a panel in a series string is shaded it limits the current of the whole string, and the bypass diodes used by industry to protect it simply skip the shaded cells and leave their energy on the table. To extract this energy a device coined the 'Power Shuffler' has been built: a bidirectional interleaved buck-boost converter that equalises the module voltages by moving current up and down the string. The complexity of modeling the dynamics of the shuffler stems from how physical states are coupled, not just by the coupled inductor itself but also through connection to neighbouring shufflers, so the controller is a multiple-input multiple-output state-space design, developed through a systems-engineering V-model: an analytical model, a high-level Simulink controller architecture, controller component selection, a combined controller-and-hardware simulation in QSPICE, and finally firmware tested on the bench and in the sun. Even in open loop at a fixed 50 % duty the shuffler returned power from a partially shaded string. In closed loop, the decoupling PI gain matrix held the module-voltage differences within $\pm 0.010\,\mathrm{V}$ in every settled bench run, below the sensing noise, and simulation shows no overshoot at the deployed gains with 2 % settling in 31–41 ms at 10 % of the simulated gains (18–21 ms at 20 %). In the solar test, with one of three 80 W modules shaded to almost zero current, a plain series string would have delivered nothing; the shufflers delivered up to 99 W and 10.6 Wh over 36 minutes. The controller is stable and extracts power during partial shading events. The next step is a stackable board to test strings of more than three modules.

Contents

Executive Summary i

Abstract ii

1 Introduction 1

2 Literature Review 2
2.1 Partial shading and MPPT . . . 2
2.2 Existing shuffler concepts: DPP and S-PPC . . . 3
2.3 Controlling the interleaved buck-boost . . . 3
2.4 Modelling, simulation and test methods . . . 4
2.5 The gap . . . 4

3 Methodology 4
3.1 Controller Design Pipeline . . . 4
3.2 Background . . . 5
3.3 Analytical Solution . . . 6
3.4 Model-in-the-Loop (MIL) . . . 9
3.5 Component Selection . . . 9
3.6 Gain Refinement Pipeline . . . 11
3.7 Firmware / Software . . . 12
3.8 Risks . . . 14
3.9 Ethics . . . 14
3.10 Sustainability . . . 14

4 Results, Analysis and Discussion 14
4.1 Proposed Solution . . . 14
4.2 Analytical . . . 15
4.3 Simulink . . . 16
4.4 QSPICE . . . 16
4.5 Bench Testing . . . 17
4.6 Solar Test . . . 19
4.7 Synthesising Results . . . 20

5 Conclusion 21

A Research Project Timeline 26

B Risk Register 26

C Analytical Solution 31
C.1 Single shuffler averaged model (lead-up, Semester 1) . . . 31
C.2 N-module string model (deployed) . . . 32
C.3 Switching resistance R_{sw} . . . 34
C.4 Open items . . . 35

D Controller, Supervisor and Firmware Pseudocode 35

E Firmware, Peripherals and Telemetry Contract **38**
E.1 Scheduling . . . 38
E.2 Board routing and current sampling . . . 38
E.3 Peripherals . . . 39
E.4 Fault bits . . . 39
E.5 Telemetry and command contract . . . 39
E.6 Firmware images . . . 40
E.7 Raspberry Pi stack . . . 40

F Gain Sets **41**

G Simulink Model and Validation Suite **41**

H Simulation Model Sheets **44**

I Bench Test Records (16 Oct 2026) **45**

J Solar Test Records (20 Oct 2026) **47**

K Synthesis: Inventory and Metrics **48**
K.1 Metric definitions . . . 48
K.2 Test-case inventory and data rules . . . 48
K.3 Hardware closed-loop steady state . . . 49
K.4 Cross-phase step metrics . . . 50

L Thermal Data and Oscilloscope Captures **50**

M The Journey **52**

List of Figures

1 P–V curves of a four-module string (AIKO Neostar A555, single-diode model with bypass diodes) under five irradiance patterns. . . . 2
2 Controller design pipeline mapped onto the systems-engineering V-model . . . 5
3 One interleaved buck-boost power shuffler between modules j and $j+1$ . . . 6
4 Origin of the winding equations . . . 7
5 Small-signal nodal model of a three-module string with two shufflers . . . 8
6 Simulink MIL model, top level . . . 9
7 Controller data flow . . . 10
8 Controller components . . . 10
9 Gain refinement pipeline . . . 11
10 Deployed control architecture . . . 12
11 Supervisory state machine of the firmware (`6_RTOS/App/app.c`, 100 Hz task). . . . 13
12 The built power shuffler on 20 Oct 2026 . . . 15
13 Simulink carpark twin (averaged switches, sensing model, solar plant) . . . 16
14 QSPICE switching model with the firmware control DLL, solar plant, $k = 0.3$ . . . 17
15 Open-loop duty steps on the bench (16 Oct, 200 kHz) . . . 18
16 Bench, 300 kHz closed loop, $k = 0.3$ (B06) . . . 19
17 Carpark string at peak sun, 200 kHz closed loop, $k = 0.1$ (S02, 20 Oct 2026) . . . 20
18 $+1$ V reference step on $v_{d,2}$ (200 kHz gain set) against gain scale . . . 21

19 Project timeline across the two semesters (teaching weeks) with the milestone week of each phase. . . . 26
20 Single-shuffler model elements. . . . 31
21 Single-shuffler stability pictures (Semester 1 and break). . . . 32
22 Derivation drawing of the $N = 3$ string 33
23 Open-loop phase portraits of the voltage mode in the $(v_d, Z_0 i_m)$ plane 34
24 Closed-loop poles of the analytical $N = 3$ model as the gain scale k is swept 34
25 Controller routing on the Version 0 PCB. . . . 38
26 Hall-current sampling scheme 39
27 Two of the MIL validation tests 42
28 Inside the MIL plant: four AIKO-class modules and three averaged-switch shufflers. 42
29 Controller, supervisor and MPPT blocks of the Simulink MIL model 43
30 Simulink carpark twin 44
31 QSPICE integrated model, top sheet 44
32 Simulink carpark twin: power-source and load variants 45
33 QSPICE integrated model, controller sheet 45
34 Hottest INA239 die temperature in every test window 51
35 Grafana capture of the 100 kHz open-loop exploration on 13–14 Oct, which ran but behaved erratically. . . . 51
36 Oscilloscope captures of the switch-node waveforms at 200, 300 and 400 kHz (dead time and 180° interleave check). . . . 51
37 Photographs of the two set-ups 52
38 Build photographs (1 of 2). . . . 53
39 Build and test photographs (2 of 2) 54

List of Tables

1 Mismatch loss of the shaded strings 3
2 Controller components and the datasheet figures behind their selection. . . . 11
3 Bench and solar experimental set-ups. . . . 13
4 DC gain matrix $G(N)$ read as "what 1 % of duty does" 15
5 Peak-to-peak switching ripple: analytical against QSPICE 18
6 Development phases and their V-model counterpart. . . . 26
7 Milestones and progress achieved (proposal → progress report → delivery). . . . 27
8 Project risk register with outcome. . . . 28
9 Design time constants vs gain scale and the closed-loop pole frequencies at $k = 1$. . . 33
10 Execution contexts of the firmware (bare-metal superloop plus interrupts; no RTOS). 38
11 Peripheral configuration (`shuffler_hw.c/h`, `sensors.c`, `ina2x9.c`, `comms.c`). . . 39
12 Firmware fault bitmask 40
13 Firmware variants used on 16 and 20 Oct (`7_Pi/firmware/VERSION.txt`). . . . 40
14 Base gain matrices M of the deployed and candidate gain sets. . . . 41
15 Validate test matrix. . . . 41
16 Open-loop duty steps on the bench (B01s, 16 Oct, 200 kHz) 45
17 Open-loop steady state on hardware (duties 0.5) 46
18 Gain changes applied while the loop was running (`C,APPLY` during RUN_CL) and the 300 kHz stability limit (from T1/T3 and the pod command log). . . . 46
19 Loop closures on hardware (IDLE → SOFTSTART → RUN_CL) 46
20 Power and energy in the solar test windows (20 Oct 2026) 47
21 Detected shade / irradiance events in the 20 Oct solar windows (current-mismatch and string-power level shifts; irradiance was not logged). . . . 47

22 Test-case inventory 48
23 Closed-loop steady state on hardware (settled RUN_CL segments ≥ 5 s) 49
24 $+1$ V reference step on v_{d2} (S4), 200 kHz gain set 50
25 Circulating-current channel i_{c2} during the $+1$ V v_{d2} reference step (S4, 200 kHz) . . 50
26 Shade disturbance (S3: module 3 loses 1/3 of its current, PV80 plant) 50

1 Introduction

Photovoltaic (PV) solar is the fastest-growing energy source globally [1] and Australia's leading renewable resource. Rooftop and large-scale solar together supplied 19.6 % of Australia's electricity in 2024, more than wind (13.4 %) or hydro (5.5 %) [2]. Australia added 6.8 GW of renewables in 2025, and renewables now supply 43.7 % of generation in the National Electricity Market (NEM) [3]. To collect as much energy as possible, PV arrays are paired with maximum power point trackers (MPPTs), which adjust the power converter so that the array operates at its maximum power point (MPP). When a series string of PV modules is partially shaded, the bypass diodes of the shaded modules conduct, the MPP drops and the power–voltage curve develops multiple local maxima, turning MPPT into a global optimisation problem. The power shuffler addresses this by creating new paths for current to move up and down the string, equalising the module voltages so that the whole string is pinned to a single MPP. Because shufflers are partial power processors, they only need to be rated for the mismatch power rather than the full string power [4, 5]. This project revives QUT's earlier work on the shuffler [6].

The aim of this project is to design and implement a controller for the power shuffler. The research question that guided the work is: how can a controller capture the coupled dynamics of multiple interleaved buck-boost converters and command their currents so that the module voltages are balanced and maximum power is extracted despite shading?

The project has two parts. A sister report covers the electrical design of the shuffler, including the converter topology, QSPICE hardware simulations, circuit board design, manufacturing and hardware validation. In contrast, this report covers all aspects of the control system architecture: analytical solutions, Simulink models, controller component selection, gain refinement, firmware deployment and finally validation of the closed loop on the bench and out in the field. The final product is a single board carrying two shufflers, tested with a three-module string.

The report begins with a literature review of partial shading, shuffler concepts and their control (section 2), which identifies the gap this project addresses. The methodology (section 3) follows the design pipeline from power-system modelling, including a dynamics model that applies to any string of N PV modules, to a fully integrated system working in the field. The raw results from real-time telemetry and simulation are then analysed and synthesised (section 4) to quantify the performance of the controller and the energy it returns.

Objectives.

- Derive an analytical model of the interleaved buck-boost converter dynamics (section 3.3 and appendix C).
- Design a control system architecture (sections 3.4 and 3.6).
- Select the controller's sensors and microcontroller (section 3.5).
- Validate the controller experimentally on a real PV system (sections 4.5 and 4.6).
- Create a self-contained product that showcases the controller's autonomy (section 3.7).

2 Literature Review

This review seeks to articulate the gap in the literature for partial shading that the aims and objectives of the project seek to fill. Analysing how it defeats a conventional maximum power point tracker (section 2.1). Exploring the differential power processing converters that are proposed to recover the lost energy (section 2.2). Discovers how converters of this kind have been controlled (section 2.3). the modelling, simulation and test methods used to develop them (section 2.4). Finally, identifying the gap this project fills (section 2.5).

2.1 Partial shading and MPPT

At its core a solar controller is a maximum power point tracker (MPPT): a buck or boost stage whose duty is adjusted so that the array operates at its point of highest power. The MPPT literature is mature. Esram and Chapman catalogue 19 tracking methods [7], and Sarvi and Azadian update the field into classical, bio-inspired and machine-learning classes [8]. Most of these methods assume that the power–voltage (P–V) curve of the array has a single peak.

Partial shading breaks that assumption [9, 10, 11]. Every module in a series string carries the same current, so a shaded module either limits the string current to what it can produce or, once its bypass diode conducts, drops out of the string altogether. Each combination of conducting and bypassed modules gives the P–V curve its own peak. Figure 1 shows this for the four-module string model built in Semester 1. For a scenario of three modules at 1000 W/m^2 and one module at 500 W/m^2 the string offers 1656 W with that module bypassed or about 1240 W with all four conducting. For a graduated shading scenario four separate peaks appear, with the global maximum of 914 W at only 78 V. A conventional perturb-and-observe (P&O) tracker climbs whichever peak it starts nearest and is trapped there [12]. Even a tracker that finds the global maximum leaves energy behind: table 1 compares each string's global maximum with the sum of its modules' individual maximum power points, and the mismatch loss reaches 34 % under graduated shading.

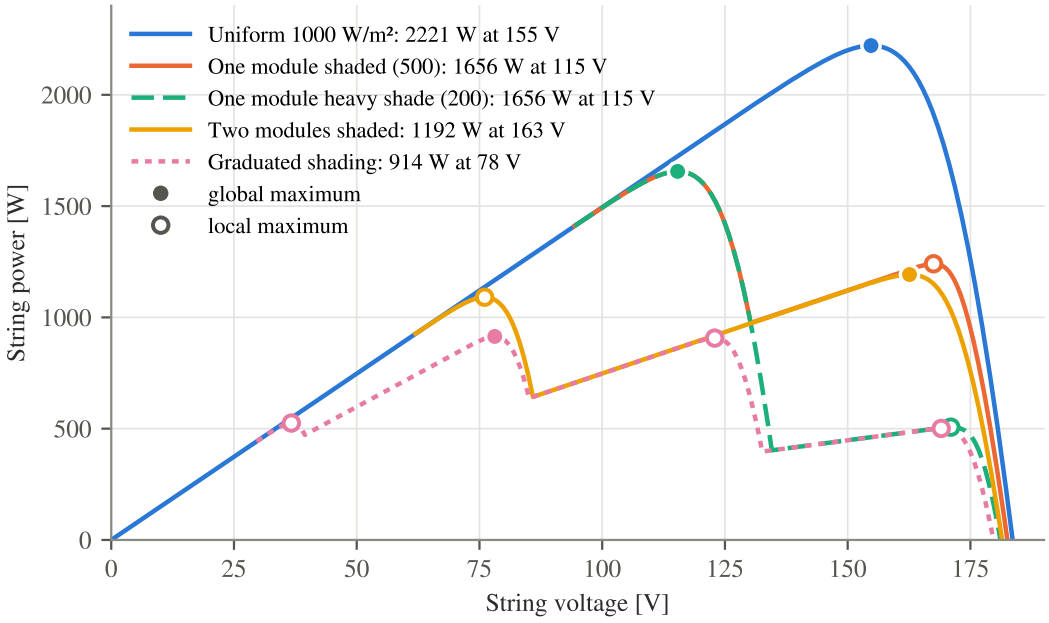


Figure 1: P–V curves of a four-module string (AIKO Neostar A555, single-diode model with bypass diodes) under five irradiance patterns.

Array reconfiguration (series-parallel, bridge-linked and total-cross-tied) reduces this mismatch loss but cannot remove it [13]. Bypass diodes protect the shaded cells, but at the cost of a multi-peaked P–V curve and the energy of every bypassed module. Recovering that energy requires moving power

Table 1: Mismatch loss of the shaded strings in fig. 1: string global maximum against the sum of the modules' individual maxima (the power a lossless per-module tracker could extract).

Scenario	Irradiance [W/m^2]	Module MPPs [W]	String MPP [W]	Loss
Uniform	1000, 1000, 1000, 1000	2221	2221	0.0 %
One module shaded	1000, 1000, 1000, 500	1944	1656	14.8 %
One module heavy shade	1000, 1000, 1000, 200	1775	1656	6.7 %
Two modules shaded	1000, 1000, 500, 500	1667	1193	28.5 %
Graduated shading	1000, 800, 500, 200	1388	914	34.1 %

between the modules themselves.

2.2 Existing shuffler concepts: DPP and S-PPC

The power shuffler is part of the family of differential power processors (DPP) [4], whose reason for being is to move differences in power around a system, effectively balancing out either voltage or current. The serial partial power converter (S-PPC) specialises in moving the power differentials of modules that are connected in series [14].

The idea has a long history at QUT: Walker, Xue and Sernia first proposed per-module "shuffling" converters that let each module in a string operate at its own maximum power point [6]. Hui's later QUT work documents why this matters outside the lab [15]. Shufflers of a very different design were fitted to ATN's solar car *Priscilla* for the 2019 Bridgestone World Solar Challenge. IN a prestigious challenge such as this every watt matters. The curvature of the 1.25 kW array and obstacles passing overhead shade it unevenly while driving. Drops in performance of these solar panels translate to reduced speed and cruise time in this endurance race. The design was rushed into the car under tight timelines with limited testing, the devices did not perform reliably and ultimately failed during the challenge, and the follow-on work was redirected to making them operate safely and reliably. It is a clear reminder that protection and safety features have to be designed into a shuffler from the start (section 3.8). Kasper et al. classify panel-integrated DC–DC concepts and identify the buck-boost as the most promising [4, 16]. Olalla et al. formalise the modelling of DPP architectures and show near-ideal energy extraction with converter ratings that scale with the mismatch rather than the string power [17, 5]. Jeong et al. review DPP architectures [18] and demonstrate a four-unit segmented DPP experimentally [14]; Li and Cobos give a framework for comparing the VA ratings of partial power converters [19]; and dos Santos et al. present a practical two-switch S-PPC [20]. A common thread runs through these works: their focus is the power stage and the architecture, and the control is either not discussed or kept simple and classical, with a PI loop on a single unit.

The S-PPC used in this project is the interleaved buck-boost converter: two half-bridge legs joined through a coupled inductor to the midpoint between two series modules (fig. 3). At a duty of $D = 0.5$ the midpoint sits at half the pair voltage; moving D away from 0.5 moves current from one module to the other, and in steady state the current circulating between the two phases should be zero.

2.3 Controlling the interleaved buck-boost

A single buck-boost converter is described by one second-order ordinary differential equation (ODE), and classical frequency-domain design works well for it. Interleaving changes this the presence of a second phase and the coupled inductance couple the states (ie. PV module voltages and the currents through each inductor winding), so they cannot be controlled independently [21]. Kabalci and Boyar add an extended Kalman filter to an interleaved boost converter to handle these coupled states [22], and the behaviour of coupled inductors in interleaved converters is treated in [23, 24].

State-space methods handle this coupling naturally. Batzelis et al. give a six-state representation of irradiance-driven PV dynamics that includes the boost converter, the PI integrator and the MPPT

reference [25]; this is the modelling style adopted here, extended with the shuffler's winding dynamics. Other state-space MPPT work reports that LQR and LQR-PI controllers outperform P&O [26, 27, 28].

2.4 Modelling, simulation and test methods

Switching converters are normally designed for control with averaged models. State-space averaging replaces the switched circuit with a continuous-time model whose inputs are the duty cycles [29], and Kassakian et al. develop averaged and linearised models of the buck-boost family as the basis for its control design [30]. Averaging discards the switching ripple, so the model is valid only well below the switching frequency, and it says nothing about dead time, sampling delay or sensor noise.

The simulation tools follow from those two limits. Simulink is the standard environment for model-based design: the controller and an averaged plant run together in one model, which makes it the natural tool for model-in-the-loop (MIL) verification on the left of the V-model [31, 32]. QSPICE is a switching-level SPICE simulator with built-in C++ compilation, so compiled C code can run as a block inside the circuit [33]. That allows the firmware's own control core to be tested against real switches, drivers, dead time and sensing models before it reaches hardware, closing the gap the averaged model leaves.

For experimental testing, MPPT performance is standardised: EN 50530 defines dynamic test sequences of irradiance ramps for measuring tracking efficiency, and Li et al. use them to compare MPPT algorithms experimentally [34]. Riquelme-Domínguez and Martínez evaluate MPPT algorithms on state-space models under several such dynamic test procedures [12]. These procedures need a programmable PV simulator. This project draws on the same approach, repeatable steps on the bench with programmable supplies and an electronic load followed by validation on a real string outdoors, but a full EN 50530 campaign is outside its scope.

2.5 The gap

Past controllers have been able to rely on classical methods such as PI because each controls a single converter. Multiple-input multiple-output (MIMO) control of a string of coupled shufflers has had very limited exploration, and control of the interleaved buck-boost converter is itself a niche topic. To the author's knowledge, no published work models an N-module string of interleaved buck-boost DPP converters as one MIMO plant, identifies its DC gain matrix and closes a decoupling integral controller on real hardware, with the module-voltage differences and circulating currents as outputs. This project does that, carrying the design through analytical, Simulink, QSPICE, firmware, bench and solar phases, and reports the same metrics for every phase on one set of test cases.

3 Methodology

3.1 Controller Design Pipeline

The methods applied to the project correspond to the stages of the V-model in fig. 2. It is a way for systems engineers to map out development across a product's life cycle [32]. The first step in the design process is to determine the target use case and application of the product through background research, refining project objectives. To determine the system requirements for a controller the plant is modelled. The controller algorithms are then adpated to Simulink where a high-level structure forms. Based on this abstract model, components are selected for the controller PCB based on the requirements of the controller architecture. Selecting marketplace controllers and sensors, opens the door for firmware development because now there is a defined interface. The firmware's control code is then combined with the plant models in software-in-the-loop and QSPICE simulations, which give a theoretical approximation of the experiments. The system is tested in the electronics lab, and finally outdoors with a string of three PV modules, which is the core objective and ultimate validation of the system. Table 6 (appendix A) links each phase to its V-model counterpart and its primary artefact.

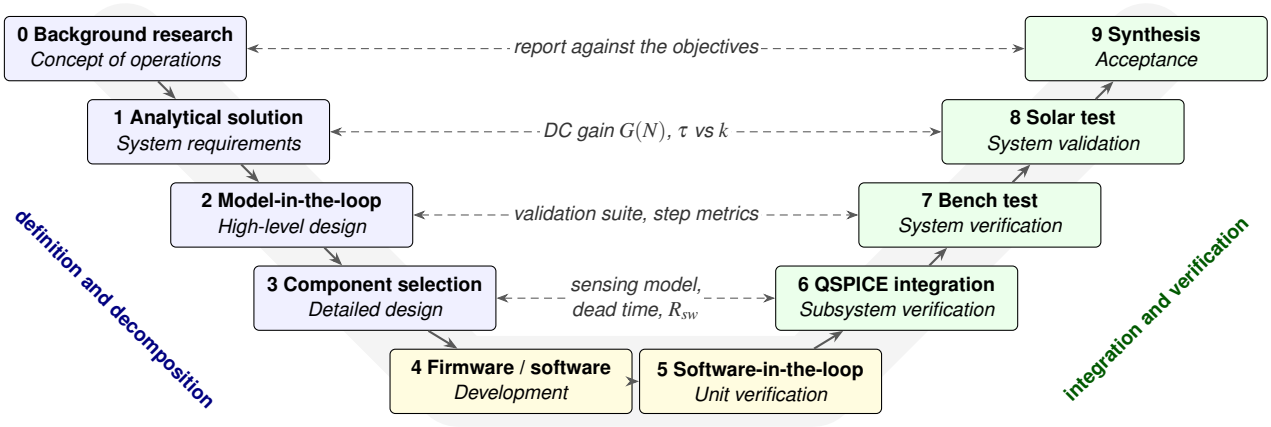


Figure 2: Controller design pipeline mapped onto the systems-engineering V-model. Dashed links are the verification pairs used in section 4.7.

This method suits the problem because every phase produces a pass/fail artefact that the next phase consumes (model, then gains, then `params.h`, then the bench), so a fault is isolated at the level where it was introduced. The C control core is the same source file in software-in-the-loop, in QSPICE and in the firmware, so what is simulated is what is deployed.

The work was divided between control and power electronics. The hardware teammate, Son Nguyen, owned the power stage, the QSPICE schematic and the PCB layout. This project owned the control. The hand-over interface between the two was the QSPICE schematic and the sensor set. The team was proactive with regular progress updates occurring often more than twice a week. The supervisor, Assoc. Prof. Walker was updated eveyr fortnight either via email or in-person meetings.

Every bench and solar run was logged as telemetry at about 44 Hz into InfluxDB, alongside the electronic load's own log at 5 Hz. Grafana screenshots defined the test windows, and the analysis pipeline re-exports those windows and computes every metric from the same CSV files (section 4.7).

3.2 Background

Figure 3 shows one shuffler drawn in the variables of the N-module model. Its two half-bridge legs, phase A and phase B, switch between the top rail ϕ_j and the bottom rail ϕ_{j+2} at duties d_A and d_B, and each switch node drives one winding of a coupled inductor whose common end is tied to the midpoint ϕ_{j+1} between the two modules. The capacitors C_j and C_{j+1} decouple the modules from the switching ripple. The sum of the winding currents, the shuffled current $i_m = i_A + i_B$, moves charge between the two modules. Their difference, the circulating current $i_c = i_A - i_B$, flows only around the two legs: it moves no power between the modules and is pure loss, so it is regulated to zero. In steady state the two legs switch 180° apart at the same duty, and the duty sets the ratio of the two module voltages.

The target system is two shufflers, one controller, a telemetry system and a user interface on a string of three PV modules. Three is the minimum needed to show shufflers interacting in a string, and the foundation for scaling. The early designs were rated to the highest-rated commercial panel available, the AIKO Neostar 555 W (V_{mp} 38.7 V, I_{mp} 14.35 A), so that the controller would never be the limiting element. The outdoor experiment used 80 W KickAss KAUSSP80 panels [35] for safety and availability, and the models carry both parameter sets.

Each shuffler has two control inputs, the common-mode duty $u_c = d_A + d_B - 1$ and the differential-mode duty $u_d = d_A - d_B$, and two controlled outputs, the voltage difference $v_{d,j} = v_j - v_{j+1}$ and the circulating current $i_{c,j}$. With $N-1$ shufflers on N modules, every module except the two at the ends is shared by two shufflers, so each u_c moves two module voltages and the plant is a coupled ladder and thus the control problem is multiple-input multiple-output.

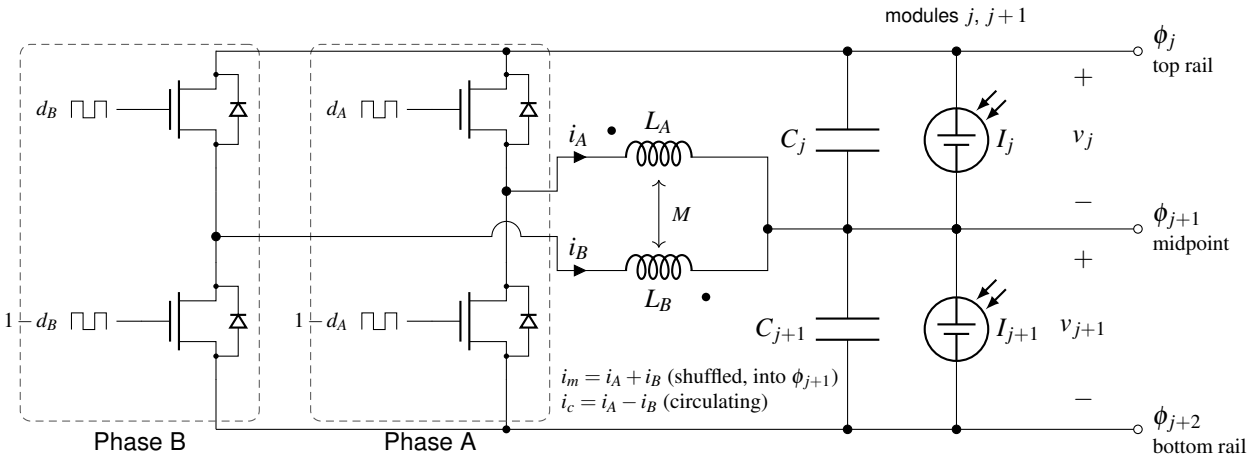


Figure 3: One interleaved buck-boost power shuffler j between modules j and $j+1$ (layout after the progress report). The two legs drive anti-series coupled windings L_A, L_B into the midpoint ϕ_{j+1}; C_j, C_{j+1} decouple the modules from the switching ripple.

3.3 Analytical Solution

Single shuffler (Semester 1 model, appendix C.1)

The Semester 1 model describes one shuffler with the state $x = [I_{L1}, I_{L2}, V_{C1}, V_{C2}]^T$, the duties $[d_A, d_B]^T$ as inputs and the module and load currents as disturbances, averaging the four dual-switch states. Three insights of that analysis were preserved. The equilibrium requires $D_A = D_B = D$ with $\bar{V}_{C2}/\bar{V}_{C1} = D/(1-D)$. Without parasitics the plant is marginally stable, with an undamped resonance at 5.56 kHz, and even with the winding resistance the damping ratio is only about 0.0025, which is why the early classical designs did not work. Finally, the module voltages alone make only three of the four states observable (the difference current is hidden), so per-phase current sensing is mandatory, which fixed the sensor set (section 3.5). The coupling between the states also ruled out three cascaded single-loop controllers. The problem is multiple-input multiple-output. It was first approached with an LQR and integral regulator and then with the decoupling PI controller that was deployed.

Where the winding equations come from

Figure 4(a) replaces each leg with its averaged switch-node voltage $\phi_{j+2} + d\bar{V}_j$, where $\bar{V}_j = v_j + v_{j+1}$ is the voltage across the pair, so each winding sees $d\bar{V}_j - v_{j+1}$ less its resistive drop. The windings are anti-series as built (turns $1{:}-1$), so $v_A = L\dot{i}_A - M\dot{i}_B$ and $v_B = L\dot{i}_B - M\dot{i}_A$. Adding and subtracting the two winding equations, and using $\bar{V}_j - 2v_{j+1} = v_j - v_{j+1} = v_{d,j}$, gives the two mode equations of the deployed model:

$$(L-M)\,\dot{i}_{m,j} = \bar{V}_j\,u_{c,j} + v_{d,j} - R_m\,i_{m,j} \tag{1}$$

$$(L+M)\,\dot{i}_{c,j} = \bar{V}_j\,u_{d,j} - R_c\,i_{c,j} \tag{2}$$

The coupled inductor therefore splits into two independent modes (fig. 4(b, c)). The shuffled current sees only the leakage inductance $L-M = 2.05\,\mu$H and the circulating current the full $L+M = 13.75\,\mu$H ($L = 7.9\,\mu$H and $M = 5.85\,\mu$H per winding as built, from the ERUC23's 2.05 μH leakage and 5.85 μH magnetising inductance [36]). The common-mode duty drives the voltage mode, the differential-mode duty drives the current mode, and to first order the two do not interact, which is the structure the decoupling controller exploits. In the averaged model $R_m = R_c = R_w$; on the switching hardware R_c also contains the switching resistance R_{sw} (section 3.6).

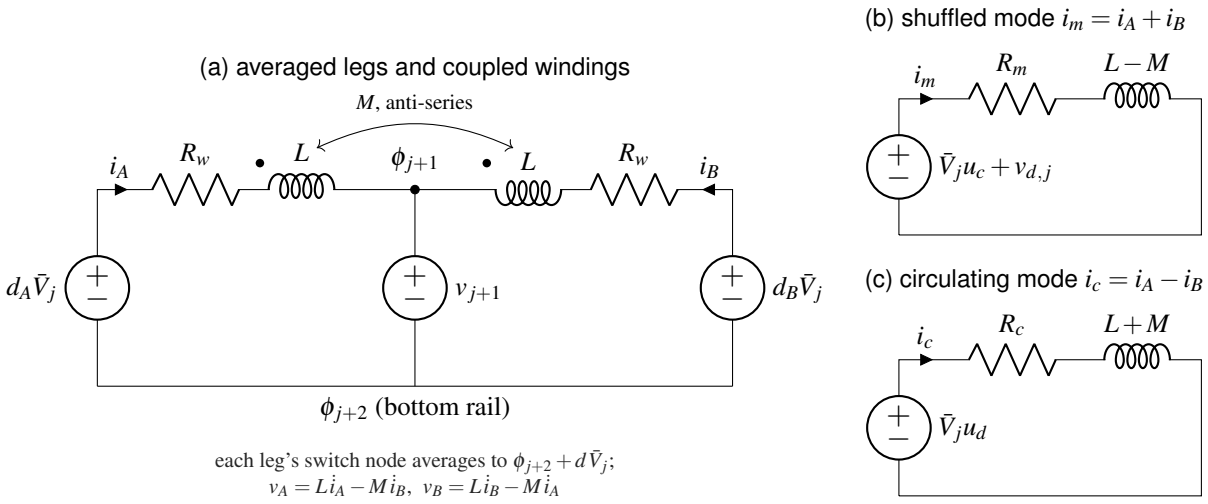


Figure 4: Where the winding equations come from. (a) Each leg averaged to $d\bar{V}_j$ above the bottom rail, driving one of the anti-series coupled windings into the midpoint. (b, c) Adding and subtracting the two winding equations splits them into the shuffled mode, which sees $L-M$, and the circulating mode, which sees $L+M$ (eqs. (1) and (2)).

The string as a nodal network

Figure 5 draws the small-signal network of a three-module string. Each module k is linearised about its operating point into a current source I_k in parallel with its slope conductance $g_k = -\mathrm{d}I/\mathrm{d}V$ and the capacitance C_k across it. Node 4 is ground, and the DL3021 in constant-voltage mode appears as a large conductance G_{load} across the string. Each shuffler enters only through its injections: averaged over a period, it pushes $i_{m,j}$ into its midpoint and draws $\frac{1}{2}i_{m,j}$ from each rail. Kirchhoff's current law at the nodes gives

$$C_n\,\dot{\phi} = -G_n\,\phi + S\,i_m + w, \qquad v_d = D_v\,\phi, \tag{3}$$

which for $N = 3$, with $w = [I_1,\ I_2 - I_1,\ I_3 - I_2]^T$ the module current injections, reads

$$C_n = \begin{bmatrix} C_1 & -C_1 & 0 \\ -C_1 & C_1{+}C_2 & -C_2 \\ 0 & -C_2 & C_2{+}C_3 \end{bmatrix}, \qquad G_n = \begin{bmatrix} g_1{+}G_{load} & -g_1 & 0 \\ -g_1 & g_1{+}g_2 & -g_2 \\ 0 & -g_2 & g_2{+}g_3 \end{bmatrix},$$

$$S = \begin{bmatrix} -\frac{1}{2} & 0 \\ 1 & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{bmatrix}, \qquad D_v = \begin{bmatrix} 1 & -2 & 1 \\ 0 & 1 & -2 \end{bmatrix}.$$

The same pattern holds for any N: C_n and G_n are tridiagonal Laplacians of the module capacitances and conductances, each shuffler adds one column $[-\frac{1}{2}, 1, -\frac{1}{2}]$ to S, and each row of D_v forms $v_{d,j} = \phi_j - 2\phi_{j+1} + \phi_{j+2}$. For the solar string, $C = [130, 260, 130]\,\mu$F (the middle module carries the decoupling capacitors of both shufflers), $g = 0.19$ S per module and $G_{load} = 20$ S.

Equilibrium

At $D = \frac{1}{2}$, charge balance on the ladder gives $T(N)\,I_m = \delta I$ with $T = \mathrm{tridiag}(-\frac{1}{2}, 1, -\frac{1}{2})$ and $(\delta I)_j = I_j - I_{j+1}$, whose inverse has the closed form $T^{-1}_{jk} = \frac{2}{N}\min(j,k)\,(N - \max(j,k))$. Each shuffler therefore carries a weighted sum of all the mismatches, and the string current settles at the mean module current, $I_{str} = \frac{1}{N}\sum_k I_k$. For $N = 3$, $T^{-1} = \frac{1}{3}\left[\begin{smallmatrix} 4 & 2 \\ 2 & 4 \end{smallmatrix}\right]$. At the QSPICE bench point, where the three supply channels limit at 0.8, 1.0 and 1.3 A, $\delta I = [-0.2, -0.3]$ A gives $I_m = [-0.467, -0.533]$ A against -0.464 and -0.539 A in QSPICE, and $I_{str} = 1.033$ A against 1.024 A. In open loop at $D = 0.5$ the only voltage difference left is the winding drop, $v_{d,j} = -R_m i_{m,j}$.

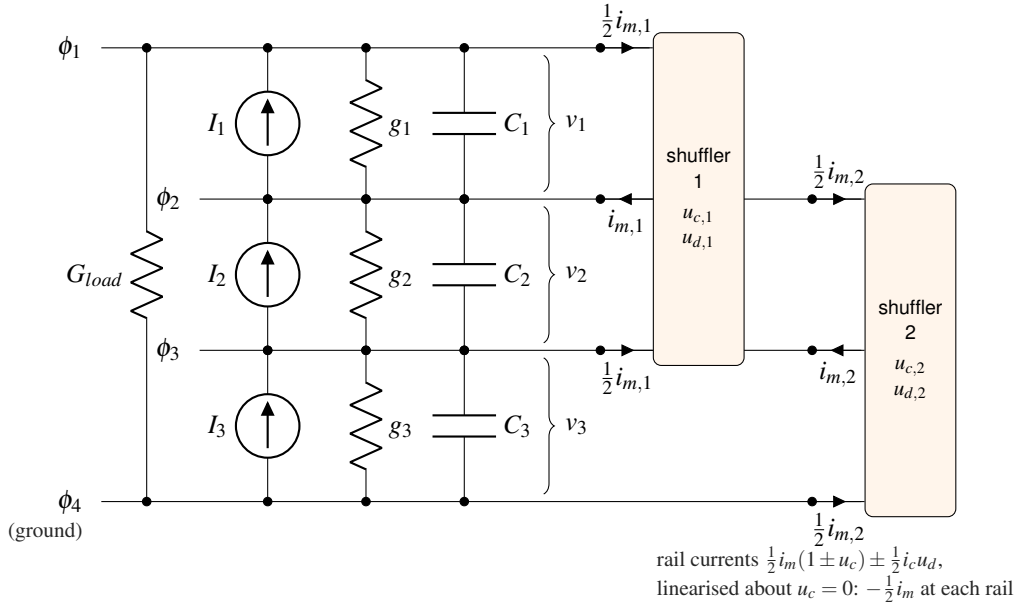


Figure 5: Small-signal nodal model of a three-module string with two shufflers: each module is $I_k \parallel g_k \parallel C_k$ between node potentials, the CV load is G_{load} across the string, and each shuffler injects i_m at its midpoint and draws $\frac{1}{2}i_m$ from each rail (the columns of S in eq. (3)).

DC gain, and what Z_e means

At DC the capacitors carry no current, so eq. (3) gives $\phi = G_n^{-1}(S\,i_m + w)$, and the voltage differences respond to the shuffled currents through

$$v_d = -Z_e\,i_m + \text{const}, \qquad Z_e = -D_v\,G_n^{-1}S. \tag{4}$$

Z_e is the string's DC transfer impedance: the drop in $v_{d,j}$ per ampere of shuffled current in shuffler k. Pushing current into a midpoint lifts it, which lowers that pair's voltage difference by an amount set by how stiff the modules are. For the three-module solar string with a stiff CV load,

$$Z_e = \frac{1}{g}\,T(3) = \begin{bmatrix} 5.26 & -2.63 \\ -2.63 & 5.26 \end{bmatrix}\,\Omega.$$

The diagonal is just the module's own I–V slope resistance $1/g$, and the off-diagonal term is the coupling to the neighbouring shuffler through the $\frac{1}{2}i_m$ rail currents. Setting the derivative in eq. (1) to zero, $\bar{V}\,u_c + v_d = R_m i_m$, and eliminating i_m with eq. (4) gives the voltage block of the DC gain; eq. (2) gives the current block directly:

$$G(N) = \begin{bmatrix} G_v & 0 \\ 0 & \operatorname{diag}(\bar{V}_j/R_c) \end{bmatrix}, \qquad G_v = -(R_m I + Z_e)^{-1} Z_e \operatorname{diag}(\bar{V}_j). \tag{5}$$

Because Z_e is almost two orders of magnitude larger than R_m (61 mΩ), $G_v \approx -\operatorname{diag}(\bar{V}_j)$. For $N = 3$ the diagonal is $-40.4\,\text{V}$ per unit duty and the neighbour terms only $0.31\,\text{V}$ per unit, so the string barely couples the voltage loops. On the bench, where the supplies sit in current limit ($g = 0$), Z_e is unbounded and $G_v = -\operatorname{diag}(\bar{V}_j)$ exactly: $-46.5\,\text{V}$ per unit at 23.3 V modules. The current block is $\bar{V}/R_c = 41\,\text{V}/0.061\,\Omega = 672\,\text{A}$ per unit in the averaged model; section 3.6 explains why the hardware value is five times smaller.

Controller

The deployed controller inverts the DC gain, $M = G^{-1}\operatorname{diag}(k_I)$, with $k_{I,v} = 2\pi \cdot 150\,\text{s}^{-1}$ for the voltage loops and $k_{I,i} = 2\pi \cdot 400\,\text{s}^{-1}$ for the current loops. The control law is $u = k\,(M_p e + M\xi)$ with $\xi \mathrel{+}= T_s e$,

$M_p = M\,\mathrm{diag}(k_{p,v}, k_{p,i})$, $T_s = 100\,\mu$s and a 2 kHz first-order filter on the measurements (appendix C.2). Because MG is diagonal, every loop behaves as a first-order lag with time constant $\tau = 1/(k\,k_I)$: at $k = 1$, $\tau_v = 1.06$ ms and $\tau_i = 0.40$ ms; at the boot value $k = 0.3$, 3.5 and 1.3 ms; at $k = 0.1$, 10.6 and 4.0 ms. The gain scale k (`kscale`) is the single run-time knob: it scales the bandwidth without changing the decoupling. The open-loop phase portraits in fig. 23 (appendix C) show why the real plant turned out easier to control than the Semester 1 model suggested: once the switching resistance is included, the voltage mode damps itself.

3.4 Model-in-the-Loop (MIL)

The MIL model (fig. 6) is the controller architecture in Simulink. The power circuit is idealised with an averaged plant of four AIKO-class modules (single-diode Simscape solar cells) and three shufflers with averaged switches. For the average switches, each leg is a controlled voltage source $d\,V_{bus}$ and a controlled current source $d\,i_L$. It uses $L_{self} = 7.9\,\mu$H, $M = 5.85\,\mu$H, a 1.85 mΩ winding resistance, a 61 mΩ per-phase damping resistor and 100 μF capacitors, with a controlled current sink driven by the MPPT reference as the load. The controller runs every 50 μs and the supervisor and MPPT every 1 ms, on a variable-step solver with a 100 μs maximum step. The controller, supervisor and MPPT blocks are reproduced in fig. 29 and their pseudocode in appendix D.

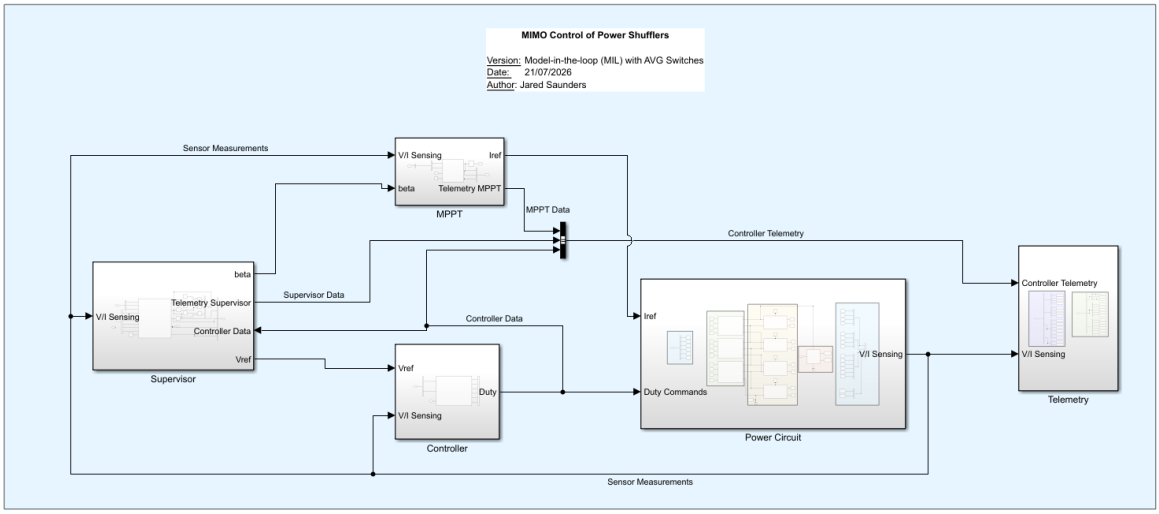


Figure 6: Simulink MIL model (`4_Simulink/1_MIL_Model_In_The_Loop`), top level: controller, supervisor, MPPT, power circuit and telemetry. The plant and the controller, supervisor and MPPT blocks are in figs. 28 and 29.

Validation used a seven-test suite (table 15): uniform irradiance, decoupling, half shade, shade with references, deep shade, MPPT under uniform irradiance and MPPT under shade. The pass criteria were $|v_d - r_v| < 0.25$ V and $|i_c| < 1$ A when tracking, an MPPT power of at least 98.5 % of the ceiling, and $|i_{diff}| \leq 10$ A in protection. All seven tests passed (table 15; two are shown in fig. 27).

Software-in-the-loop (SIL) is an extension of MIL: the MATLAB Function blocks are replaced by C Caller blocks running the firmware's control core (`6_RTOS/ctrl_core`), and a bit-identical run against the MATLAB version, the golden replay, is the acceptance test. SIL is the cornerstone of the gain refinement pipeline, because the C that passes here is the C that the QSPICE DLL and the firmware compile.

3.5 Component Selection

Figure 7 shows the controller's data flow. The user reaches the system through the Raspberry Pi: over SSH and a parameter web page, with an IR remote, and through Grafana dashboards backed by InfluxDB, while an RGB LED and an LCD screen show the system state. The Pi flashes the Nucleo, exchanges commands and telemetry with it over UART, and reads the electronic load over USB. The

Nucleo closes the loop on its own: it drives the eight PWM outputs, reads the current sensors on its ADCs and the voltage sensors over SPI, and the Pi never sits inside the control loop.

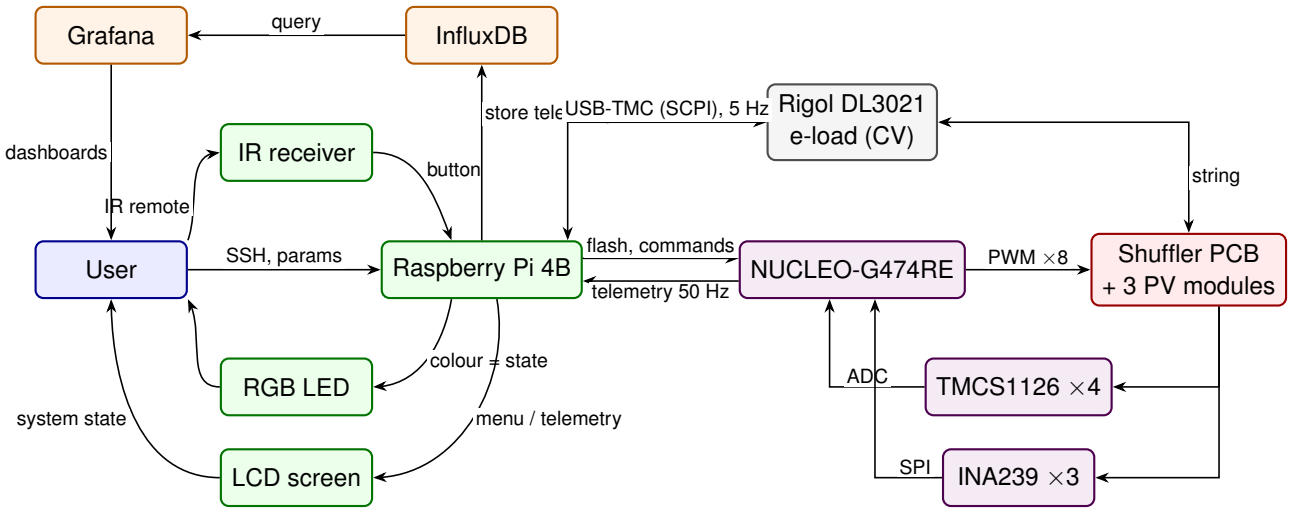


Figure 7: Controller data flow (after the presentation's data-flow diagram): the Raspberry Pi is the user interface, logger and flasher; the Nucleo closes the loop on its own.

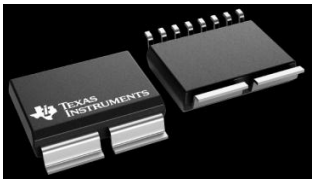
Figure 8 and table 2 summarise the four controller components, with every figure taken from the manufacturers' datasheets. The STM32G474RE on the NUCLEO-G474RE board was chosen for its 170 MHz clock, its five 12-bit ADCs and its high-resolution timer (HRTIM, 184 ps resolution, with hardware dead time and complementary outputs) [37, 38], together with its many general timers and its price; the ADC and PWM capability were the driving factors, and STM32CubeMX was used to allocate the peripherals (table 11). The TMCS1126 Hall-effect sensor is galvanically isolated, so it can safely sit in series with the windings; the B4 variant gives 100 mV/A about a 1.65 V reference, a $\pm$15.5 A linear range on a 3.3 V supply and a 500 kHz signal bandwidth [39]. The INA239 was chosen because the higher-resolution INA229 was out of stock; it measures bus voltages up to 85 V with a 16-bit ADC and 3.125 mV resolution over SPI [40]. The three devices share SPI3 with separate chip selects, and their shunt inputs are tied off, so only the bus voltage and die temperature are used. The Raspberry Pi 4B [41] was the safest way to add telemetry, remote control, a state display, connectivity and 5 V / 3.3 V distribution without adding complexity to the PCB; it hosts the web services and flashes the Nucleo, so the laptop never plugs into the system.



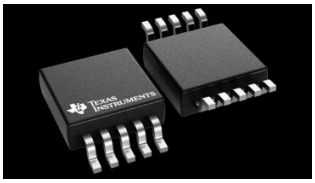
(a) Raspberry Pi 4B [41]



(b) NUCLEO-G474RE [38]



(c) TMCS1126 [39]



(d) INA239 [40]

Figure 8: The controller components: microcomputer, microcontroller, Hall-effect current sensor and power monitor.

On the board (fig. 25, appendix E) the ADC traces take the most direct path and are routed on the bottom layer to stay away from the switching nodes, the SPI forms a bus up and down the board, and the PWM and chip-select traces went wherever was convenient. One limitation of the Version 0 board was known before testing: the top INA239 (U10, chip select 3) sits behind an ISO7762 digital isolator

Table 2: Controller components and the datasheet figures behind their selection.

Role	Part	Key datasheet figures	Source
Microcontroller	STM32G474RE (NUCLEO-G474RE)	Cortex-M4 at 170 MHz; HRTIM with 184 ps resolution, hardware dead time; five 12-bit ADCs	[37, 38]
Current sensor ×4	TMCS1126B4AQDVGRQ1	100 mV/A; ±15.5 A linear at 3.3 V; 500 kHz bandwidth; reinforced isolation, 5 kV_{RMS}	[39]
Voltage sensor ×3	INA239AQDGSRQ1	0–85 V bus input; 16-bit ΔΣ ADC, 3.125 mV LSB; 10 MHz SPI	[40]
Microcomputer	Raspberry Pi 4B	quad-core Cortex-A72 at 1.5 GHz; Gigabit Ethernet, 802.11ac; 5 V, 3 A USB-C supply	[41]

that was wired backwards, so it cannot be read. The top module voltage is therefore reconstructed from the electronic load's readback, $V_{M1} = V_{str} - V_{M2} - V_{M3}$, which the Pi forwards to the Nucleo at 2 Hz.

3.6 Gain Refinement Pipeline

The gains are not tuned by hand. Each gain set starts with an identification, in which every input is stepped on a plant and the 4×4 DC gain matrix G is measured (fig. 9). The design step turns G into $M = G^{-1}\mathrm{diag}(k_I)$ and stores it, with the proportional ratios, the filter cut-off, the duty limits and a version string, in `shuffler_params.json`. The generator `gen_params.py` then writes `params.h`, which is compiled together with the same control core into all three harnesses. The version string travels in the telemetry, so every logged sample records which gain set produced it.

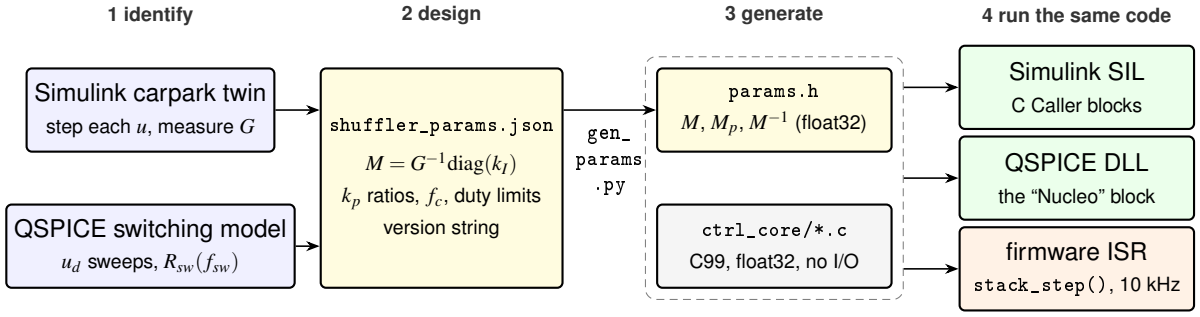


Figure 9: Gain refinement pipeline: identification in Simulink or QSPICE, one JSON gain set, a generated `params.h`, and the same C core in the Simulink SIL harness, the QSPICE DLL and the firmware.

Two identification plants were used. The Simulink carpark twin (fig. 30, appendix H) was updated to match the experiments: power sources for the NGE103 channels or the KAUSSP80 cells, the DL3021 in constant-voltage mode, a plant reduced to three modules, and a sensing block that reproduces the INA round robin (1 kHz, 3.125 mV resolution), the 12-bit hall ADC, optional noise (2 mV and 20 mA rms) and the Pi's 2 Hz V_{M1} path. The control architecture was also integrated into the hardware teammate's QSPICE model (fig. 31): a controller component wraps the unchanged C core as a DLL, sampling at 10 kHz with the INA values held at 1 kHz and the hall readings quantised, and drives HRTIM-style comparators. The sensors are modelled too, including the TMCS1126's 500 kHz pole and the reversed B-phase sensors. The two plants agree on the voltage block of G ($v_d/u_c \approx -46$ V per unit on both) but not on the current block: 742 against 142 A per unit at 200 kHz (table 4). The difference is an effective resistance in the switching plant. During each dead time the switch-node

capacitance must be commutated, which costs volt-seconds in proportion to the current and appears as

$$R_{sw} = 64\,C_p\,f_{sw}^3\,(L+M)^2, \tag{6}$$

0.29, 0.80 and 1.70 Ω at 200, 300 and 400 kHz (appendix C.3). The Simulink bench gain set was therefore rejected for hardware: QSPICE is the arbiter and Simulink the source. A new switching frequency needs a new identification and gain set (appendix F). The switching frequency is a parameter in QSPICE and a compile-time define (`HW_PWM_KHZ`) in the firmware, and `run_scenarios.m` and `run_scenarios.py` run every setup and gain scale.

3.7 Firmware / Software

The firmware is a bare-metal C application on the STM32G474, built with the STM32CubeMX HAL, CMake, Ninja and the `arm-none-eabi` toolchain at `-O2`. It contains the portable control core (`ctrl_core`: controller, supervisor, MPPT, reconstruction and the stack that schedules them), drivers for the HRTIM, ADC, SPI and UART, a supervisory state machine and a line-based telemetry and command protocol (appendix E). FreeRTOS was planned, but the shipped firmware is a superloop with interrupts: the time-critical work runs in interrupts with fixed priorities, and the slower tasks run from the main loop on millisecond ticks. Moving to a real-time operating system is left as future work (section 5).

In order of priority, the firmware runs the E-stop interrupt, the 10 kHz control interrupt triggered by the HRTIM master timer, INA polling at 1 kHz, the safety checks and state machine at 100 Hz, telemetry at 50 Hz and command parsing. The safety path (the over-current latch inside the control interrupt, the E-stop interrupt and the 0.3 s independent watchdog) never depends on the main loop. The peripheral settings are in table 11. The hall currents are sampled by four HRTIM-triggered ADC conversions per switching period and box-averaged over the whole control tick (fig. 26, appendix E); the average acts as a comb filter with nulls at f_{sw} and its harmonics, so the controller sees the average winding current free of switching ripple.

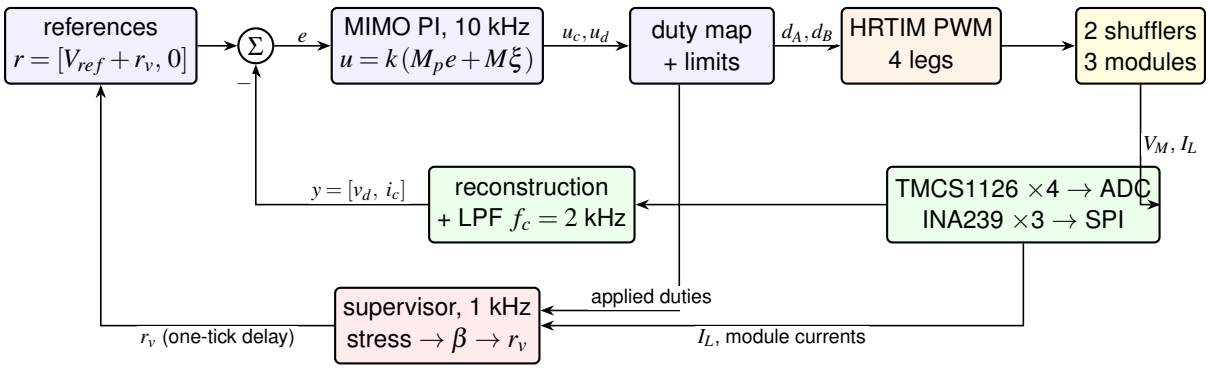


Figure 10: Deployed control architecture: the 10 kHz forward path, the measurement feedback, and the 1 kHz supervisor that sets the voltage-difference references.

Every 100 µs the control interrupt reconstructs v_d and i_c from the sensors, filters them at 2 kHz, runs the MIMO PI and maps u_c and u_d to the leg duties $d_A = \frac{1}{2}(1+u_c+u_d)$ and $d_B = \frac{1}{2}(1+u_c-u_d)$, with limits and a slew of 0.01 per tick (fig. 10). Every millisecond the supervisor forms a stress index from the filtered differential currents and the duty excursion, integrates its governor gain β and relaxes the voltage-difference references r_v, which reach the controller one tick later. The pseudocode for the interrupt, soft-start, supervisor and state machine is in appendix D.

The state machine (fig. 11) starts in BOOT and runs SELFTEST, which initialises the INA239s and zero-calibrates the hall sensors; a failure latches FAULT. From IDLE, an enable command (`C,EN,1`) with a non-idle mode starts SOFTSTART, which ramps the duties to their target over 150 ms before entering RUN. Disabling or selecting IDLE passes through STOP, which turns the legs off, and returns to IDLE on the next tick. Any trip (over-current, over- or under-voltage, E-stop, over-temperature,

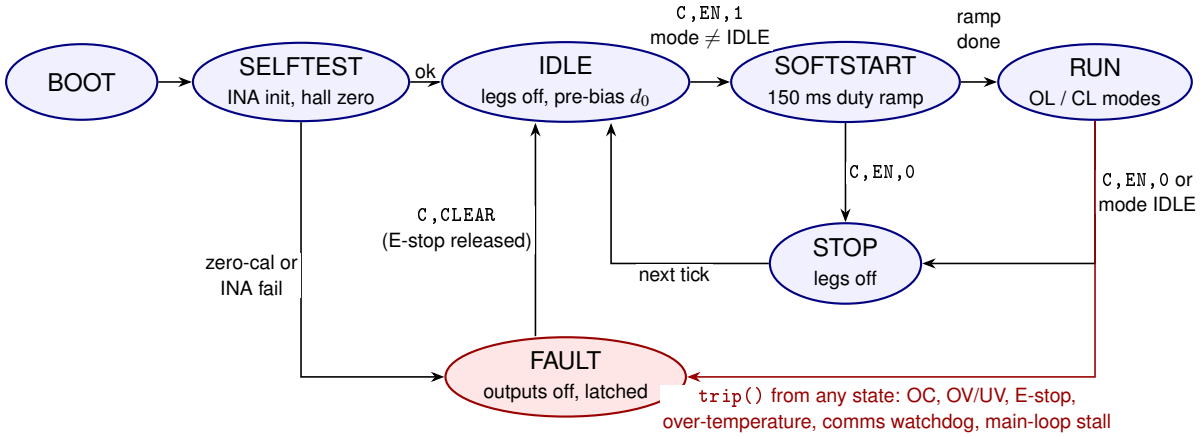


Figure 11: Supervisory state machine of the firmware (`6_RTOS/App/app.c`, 100 Hz task).

the communications watchdog or a main-loop stall) switches the outputs off first and latches FAULT, and `C,CLEAR` returns to IDLE only once the E-stop is released. On the Pi, Docker runs InfluxDB and Grafana, `pod.py` bridges the serial telemetry and drives the LCD, LED and IR receiver, and `eload.py` polls the DL3021 at 5 Hz and forwards V_{str} to the Nucleo at 2 Hz (appendix E). The physical signals were checked on the oscilloscope: the dead time and the 180° interleave at 200, 300 and 400 kHz, and the alignment of the ADC triggers (fig. 36).

3.7.1 Experimental setup and test plan

The two set-ups are compared in table 3 and photographed in fig. 37 (appendix M). The tests answer five questions: (i) does the open loop behave as modelled ($V_{M2}/(V_{M2}+V_{M3}) = 1-d$, $v_d = -R_m i_m$, a duty-step gain of $-\bar{V}$); (ii) does the closed loop regulate ($v_d \to 0$, $i_c \to 0$, with the steady-state error and noise measured against k and f_{sw}); (iii) does it reject disturbances (supply and load steps on the bench, shade in the sun); (iv) how much stability margin is there (raising k until the loop misbehaves); and (v) how much energy is returned under shade (delivered against returned power).

Table 3: Bench and solar experimental set-ups.

	Bench, 16 Oct 2026 (lab)	Solar, 20 Oct 2026 (carpark)
Modules	R&S NGE103 three-channel supply [42], 25 V set, current limits 0.8 / 1.0 / 1.3 A	3 × KickAss KAUSSP80 [35]: 80 W, V_{oc} 25.1 V, I_{sc} 4.35 A, $V_{mp} \approx 20.5$ V
Load	Rigol DL3021 [43] in CV at 70 V (steps 60–125 V)	DL3021 in CV at 61 V ($\approx 3V_{mp}$)
Mismatch	supply current limits	cardboard or a shirt over one module
Switching frequency	200, 300, 400 kHz, one gain set per f_{sw}	200 kHz only
Gain scale	per run	sweep $k = 0.1$ to 0.8
Instruments	oscilloscope (dead time, interleave)	telemetry and e-load log only
Supply for Pi and laptop	mains	battery and inverter
Windows	10 (B01–B09 and B01s, table 22)	11, peak sun 15:04–15:13, minimal sun after 15:48

The two set-ups test different things. The supply channels are stiff current limits with no I–V curve and no irradiance, so the bench exercises the loop and the gain set for each switching frequency. The sun exercises the plant the controller was designed for, but only at 200 kHz and at the irradiance the day gave. The full test matrix is table 22 (21 windows); the pre-flight checklist and power-up order are in the repository (`0_Documentation/experiment_checklist.md`).

Every metric has one definition for every phase (`stepmetrics.py`, appendix K).

The hardware cannot resolve the loop dynamics. Telemetry arrives at about 44 Hz (11–14 Hz on three 200 kHz bench runs) while the loop time constants are 1–10 ms, so hardware rise and settling times are bounds ($\leq$ 23 ms, one telemetry frame). The hardware does resolve steady-state error, noise, second-scale disturbances, duty-step amplitudes and energy, and the step response is taken from the models.

3.8 Risks

The risk register from the proposal (table 8, appendix B) was kept and updated through the project, and the earlier shuffler's failure (section 2.2) is why protection was designed in from the start rather than added later. Several risks materialised. Scope creep compromised the PCB revisions and features, leaving a single board. Firmware faults were contained by the E-stop, fault latching and state telemetry: faults 0x10 (module under-voltage) and 0x2000 (main-loop stall) tripped three times on 16 Oct when gains were applied live (table 18), and each time the outputs switched off safely. The averaged model was inaccurate, over-estimating the current-loop gain fivefold, but this was caught in QSPICE before any hardware ran. The top voltage sensor (U10) could not be read and was replaced by the e-load readback. Electrical safety (component ratings and a 15 A string fuse) and outdoor testing (sun, fire and rain) were managed by procedure, and no risk led to injury or to component damage (U10's isolator was wired backwards at assembly).

3.9 Ethics

Electrical safety followed QUT's HSE protocols for electrical workspaces: PPE, a 15 A string fuse, hardware and software E-stops, over-current, over- and under-voltage and over-temperature envelopes in the firmware, and a rule that the system was never left energised unattended. For data security and privacy, all services are hosted locally on the Raspberry Pi with authentication and no cloud connection, only sensed data and internal states are streamed, and credentials are excluded from version control. For integrity, AI assistance (Claude) was used for derivation checking, scripting and the results pipeline; every result can be reproduced from the repository (`make all`).

3.10 Sustainability

The project was sustained by constant collaboration (bi-weekly team meetings, a shared GitHub repository and defined hand-over milestones with the hardware teammate). The future of the project is supported by every folder of the repository containing a README, videos recordings expaining the project and regenerable results are all in the repository. Because the shuffler processes only the mismatch power, its component ratings and losses scale with the shading rather than with the string power, which supports more efficient use of existing solar capacity (UN SDG 7.3).

4 Results, Analysis and Discussion

4.1 Proposed Solution

The delivered system (fig. 12) is a self-contained power shuffler for a three-module string: one Version 0 PCB carrying two interleaved buck-boost shufflers, the NUCLEO-G474RE that closes the loop, and a Raspberry Pi pod with an LCD, an RGB LED and an IR receiver, in a printed enclosure with a heatsink. Against the objectives, the N-module averaged model gives the plant and its DC gain in closed form (section 3.3); the decoupling MIMO controller is written once in C and runs unchanged in SIL, in QSPICE and in the firmware; the controller board senses the four winding currents and the module voltages; and the system was validated on the bench and on a real string in the sun. In use, the operator picks a preset with the IR remote or the parameter page, the firmware soft-starts the legs and closes the loop, and the module-voltage differences and circulating currents are held at zero while

Grafana shows the live telemetry. Any trip latches FAULT with the legs switched off.

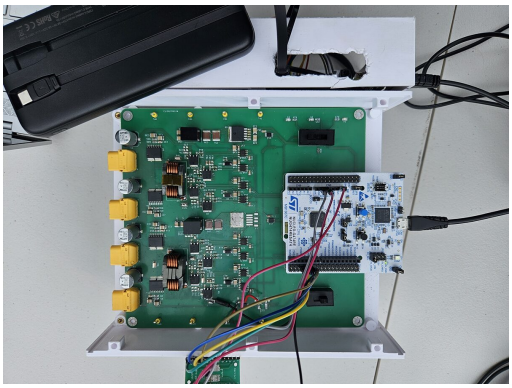


Figure 12: The built power shuffler on 20 Oct 2026: PCB, Nucleo, Pi pod, LCD and heatsink (the CAD render is on the title page).

4.2 Analytical

The analytical $N = 3$ model sets the expectation for every later phase. Table 4 reads its DC gain as what one per cent of duty does. On every plant and in every model, 1 % of common-mode duty moves the voltage difference by about 0.4 V, close to $\bar{V}/100$, because $Z_e \gg R_m$ (section 3.3). The current block is where the plants differ. With averaged switches, 1 % of differential-mode duty moves the circulating current by 6.7 A; once the switching resistance is included it is 1.4 A at 200 kHz and 0.24 A at 400 kHz. The gain identified in QSPICE on the bench operating point (1.43 / 1.39 A) sits within 13 % of the analytical bench value (1.61 A), which is the evidence that R_{sw} is real. The averaged Simulink plant over-estimated the current-loop gain fivefold (7.42 A), and its gain set was rejected for the hardware. The last column of table 4 is the one term the averaged model cannot predict: a 0.14–0.25 V cross-coupling from differential-mode duty into $v_{d,1}$, caused by the dead time, which the identified gain matrix cancels.

Table 4: DC gain matrix $G(N)$ read as "what 1 % of duty does": the voltage block (common-mode duty u_c to the voltage difference v_d) and the current block (differential-mode duty u_d to the circulating current i_c), shuffler 1 / shuffler 2 where they differ. The last column is the dead-time cross-coupling that only the switching model shows. The gain sets store $M = G^{-1}\mathrm{diag}(k_I)$, so $G = \mathrm{diag}(k_I)\,M^{-1}$.

Obtained from	Plant	Δv_d [V] per 1 % of u_c	Δi_c [A] per 1 % of u_d	cross: Δv_{d1} [V] per 1 % of u_{d1}
Analytical	Solar string, averaged switches ($R = 61\,\mathrm{m\Omega}$)	-0.40	6.72	0.00
Analytical	Solar string, 200 kHz ($R_{sw} = 0.29\,\Omega$)	-0.38	1.41	0.00
Analytical	Solar string, 400 kHz ($R_{sw} = 1.72\,\Omega$)	-0.29	0.24	0.00
Analytical	Bench supplies in current limit, 200 kHz	-0.47	1.61	0.00
Simulink step tests	Bench, averaged switches (rejected)	-0.46 / -0.47	7.42 / 7.42	-0.01
QSPICE step tests	Bench, 200 kHz (deployed)	-0.46 / -0.47	1.43 / 1.39	-0.25
QSPICE step tests	Bench, 300 kHz (deployed)	-0.45	0.59 / 0.57	-0.17
QSPICE step tests	Bench, 400 kHz (deployed)	-0.42 / -0.43	0.23	-0.14

The root locus (fig. 24, appendix C) shows the margin of the deployed gains. With the deployed M, the voltage-loop poles stay real up to $k \approx 1$ and the circulating-current loops become complex first. The same M on the 61 mΩ averaged plant crosses into the right half-plane near $k \approx 1.5$, a second reason the averaged gain set could not be used. The open-loop phase portraits (fig. 23) show the same trend: as the plant resistance grows with f_{sw}, the voltage mode changes from a lightly damped focus to an over-damped node.

4.3 Simulink

The MIL model passed all seven validation tests (table 15, appendix G). Figure 13 shows the carpark twin, which adds the sensing model, closing the loop on the three-module solar plant. In (a) a $+1\,\mathrm{V}$ step on the $v_{d,2}$ reference is followed with the first-order shape of the design, with τ of 11.6, 3.5 and 0.56 ms at $k = 0.1$, 0.3 and 1 against the design 10.6, 3.5 and 1.06 ms, while $v_{d,1}$ does not move: the voltage loops are decoupled. In (b) and (c) module 3 loses a third of its irradiance. In open loop at $D = 0.5$ the shufflers already carry most of the deficit but leave a residual $v_d = -R_m i_m$ of 0.06 and 0.12 V; the closed loop removes it within 15 ms at $k = 0.3$. Panel (c) shows why the string recovers: the shuffled currents step to 0.97 and 1.92 A, the weighted sums of the mismatch predicted by $T^{-1}\delta I$ (0.96 and 1.92 A), and the string current settles at the mean module current, 3.82 A. The only overshoot in the twin is on the bench plant at $k = 1$ (8.8 %).

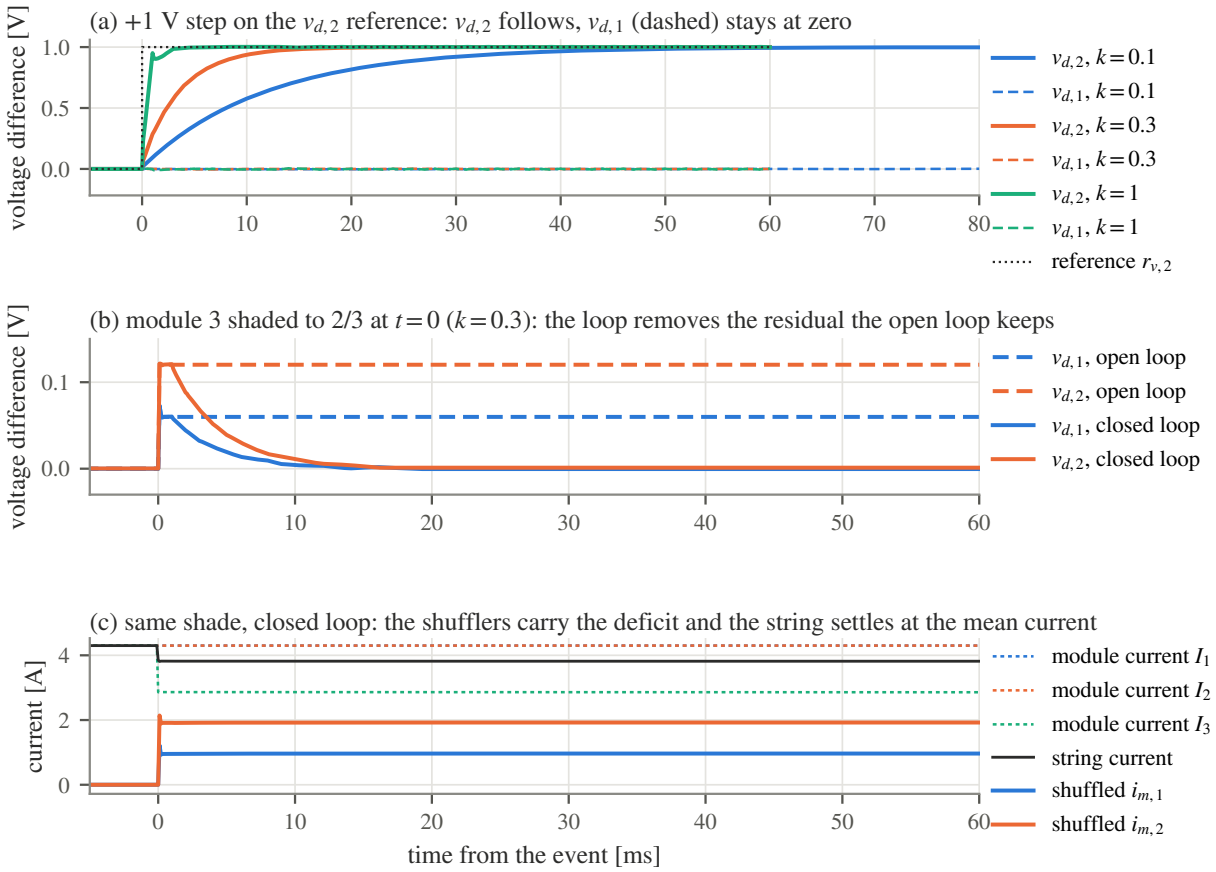


Figure 13: Simulink carpark twin (averaged switches, sensing model, solar plant): (a) $+1\,\mathrm{V}$ reference step at three gain scales, (b, c) module 3 shaded to two thirds at $t = 0$, closed loop against open loop.

4.4 QSPICE

Figure 14 is the strongest simulated evidence, because it runs the unchanged firmware control core as a DLL against real switches, drivers, dead time and the sensor models. A single run at $k = 0.3$ on the solar plant closes the loop on the open-loop residual, follows a $+1\,\mathrm{V}$ step on the $v_{d,2}$ reference with $v_{d,1}$ held, and then rejects a shade on module 3. The shuffled currents pick up the mismatch, the circulating currents stay within $\pm 0.1\,\mathrm{A}$, and the string current falls to the mean module current.

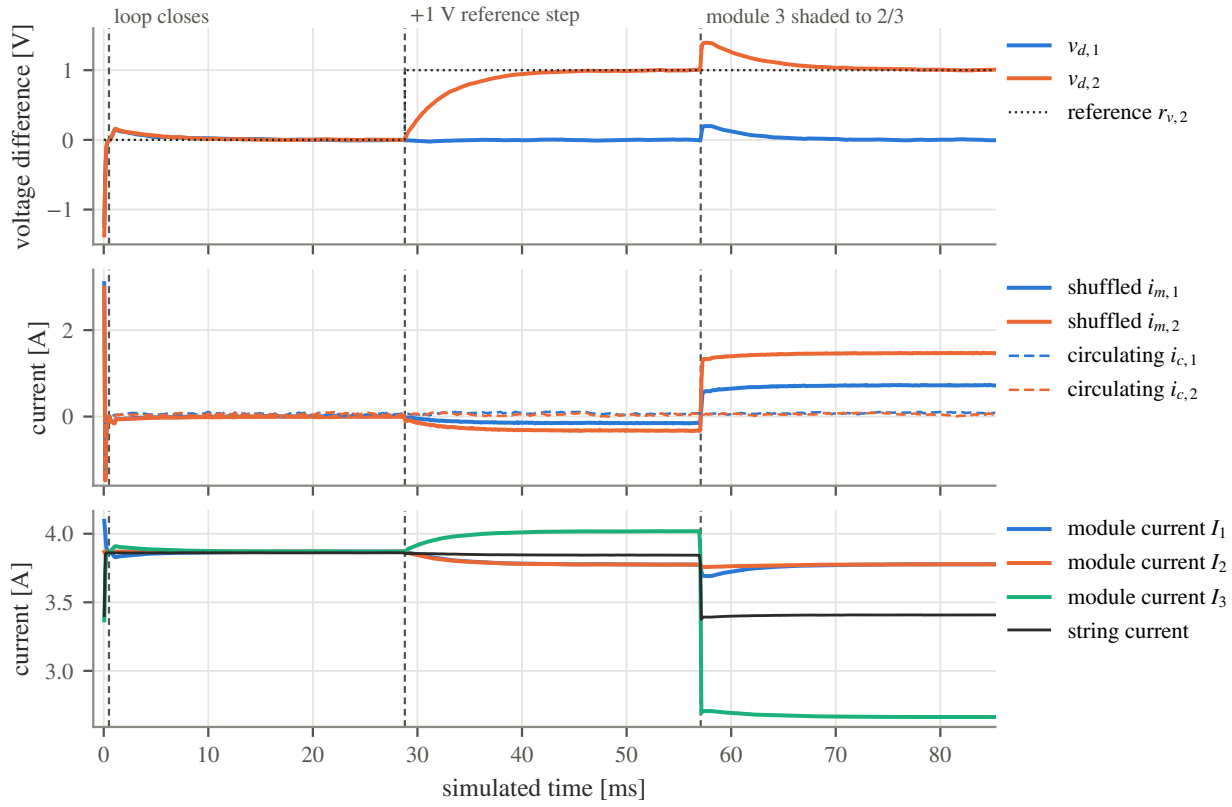


Figure 14: QSPICE switching model with the firmware control DLL, solar plant, $k = 0.3$ (100 μs bin means of the switching waveforms): loop closure, +1 V reference step on $v_{d,2}$, then module 3 shaded to two thirds.

The 200 kHz gain matrix identified from this model, and deployed on the hardware, is

$$G_{200} = \begin{array}{c} \\ v_{d1} \\ v_{d2} \\ i_{c1} \\ i_{c2} \end{array} \begin{array}{c} \begin{array}{cccc} u_{c1} & u_{c2} & u_{d1} & u_{d2} \end{array} \\ \begin{bmatrix} -46.3 & -0.2 & -24.7 & -0.1 \\ -0.2 & -46.9 & -0.1 & -27.4 \\ -1.1 & -0.3 & 142.5 & 1.4 \\ -1.4 & -0.9 & 0.1 & 139.0 \end{bmatrix} \end{array} \tag{7}$$

in volts or amperes per unit duty. Its structure is the analytical one: a voltage block of $-\bar{V} \approx -46.6$ V with neighbour terms below 0.5 %, a current block of $\bar{V}/R_c$, and almost no current response to the common-mode duty. The only new terms are the v_d/u_d cross terms of -25 and -27 V, the dead-time effect. The matrix is well conditioned ($\operatorname{cond} G = 3.2$), so its inverse, the stored gain matrix M, is well behaved (appendix F).

The device model also reproduces the switching ripple (table 5). QSPICE matches $\Delta i_c = \bar{V}/(2 f_{sw}(L+M))$ to within 1–3 % at all three frequencies. Each winding carries half of the circulating ripple, and in the shuffled current the two interleaved windings cancel, leaving only 0.2–0.4 A. Had the circulating mode seen $L-M$ instead, the ripple would be 28–57 A, so this result also confirms the as-built anti-series coupling. The telemetry cannot see the ripple: its samples are box-averaged over a whole control tick, and their spread is sensing noise. QSPICE was not used to judge milliamp and millivolt levels of i_c and v_d, which depend on milliohm parasitics and nanosecond edges.

4.5 Bench Testing

Bench testing was a success. The set-up was the three-channel supply (NGE103), the electronic load (DL3021), an oscilloscope, a laptop and the full system (table 3). Testing started at 200 kHz in open

Table 5: Peak-to-peak switching ripple in A. The circulating mode sees $L+M$, so the analytical law is $\Delta i_c = \bar{V}/(2 f_{sw}(L+M))$ with $L+M = 13.75\,\mu\text{H}$ and $\bar{V} = 46.6\,\text{V}$. Each winding carries half of it (Δi_A); in the shuffled current the two interleaved windings cancel (Δi_m). The 44 Hz telemetry cannot see the ripple: its $\sigma(i_c)$ is sensing noise.

f_{sw} [kHz]	Δi_c analytical	Δi_c QSPICE	Δi_A per winding	Δi_m QSPICE	QSPICE vs analytical	telemetry $\sigma(i_c)$
200	8.47	8.39	4.25	0.19	-1 %	0.30
300	5.65	5.58	2.85	0.20	-1 %	0.05
400	4.24	4.09	2.18	0.39	-3 %	0.08

loop to get the baseline behaviour, then closed the loop at 0.1, 0.2 and 0.3 of the QSPICE gains, then ran 300 kHz at 0.3 and 0.5 and 400 kHz at 0.3. 300 kHz was the most favourable: as the switching frequency rises the open-loop residual and the current-loop gain fall, but the heatsink temperature rises (appendix L). An open-loop test at 100 kHz was also explored on 13 Oct; the system ran, but with erratic behaviour. The supplies cannot deliver as much as the panels, so the bench exercised the loop rather than the power stage.

Open loop. The three open-loop duty steps (fig. 15, table 16 in appendix I) confirm the model's DC gain on hardware: every measured change in v_d is within 0.02 V of $-\bar{V}\Delta u_c$, and the shuffler whose duty did not change stays put. At duties of 0.5 the residual $v_{d,2}$ is $-0.17\,\text{V}$ at 200 kHz, $-0.58\,\text{V}$ at 300 kHz and $-0.94\,\text{V}$ at 400 kHz (table 17). This is the $-R_m i_m$ drop, growing with R_{sw}, that the closed loop has to remove.

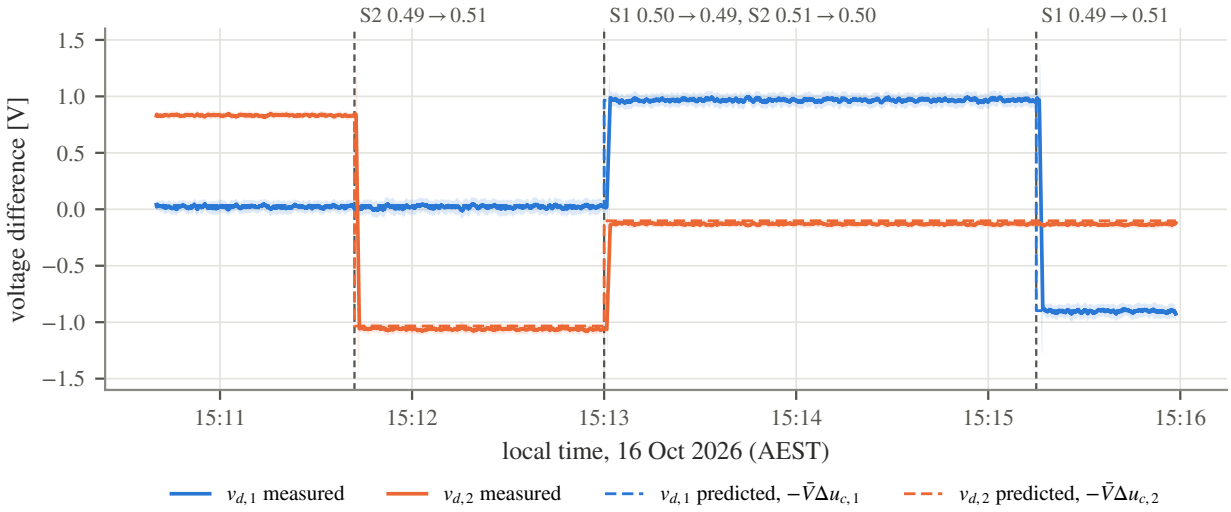


Figure 15: Open-loop duty steps on the bench (16 Oct, 200 kHz): measured voltage differences (1 s mean, $\pm 1\sigma$ band) against the averaged-model prediction $-\bar{V}\Delta u_c$ (dashed), which overlays the measurement.

Closed loop. Figure 16 shows 300 kHz at $k = 0.3$ through seven step changes of the supply channels. The shuffled currents move to carry each new mismatch while the 1 s means of both voltage differences stay within $\pm 0.008\,\text{V}$ and of both circulating currents within $\pm 0.03\,\text{A}$; the raw samples scatter by 0.016 V and 0.055 A. Over all 22 settled closed-loop segments, $|\bar{v}_{d,2}| \le 0.010\,\text{V}$ and $|\bar{i}_{c,2}| \le 0.066\,\text{A}$, with sample noise $\sigma(v_{d,2})$ of 0.013–0.17 V (table 23): the references are held with no measurable offset, and the residual is sensing noise. The 150 ms soft-start leaves 0–0.8 V on $v_{d,2}$, which the loop removes within one telemetry frame in 10 of 23 closures and within 3 s, limited by noise, in the rest (table 19).

Stability limit and faults. At 300 kHz and $k = 0.6$ the loop oscillated ($\sigma(v_{d,2}) = 1.41\,\text{V}$, 11.8 V

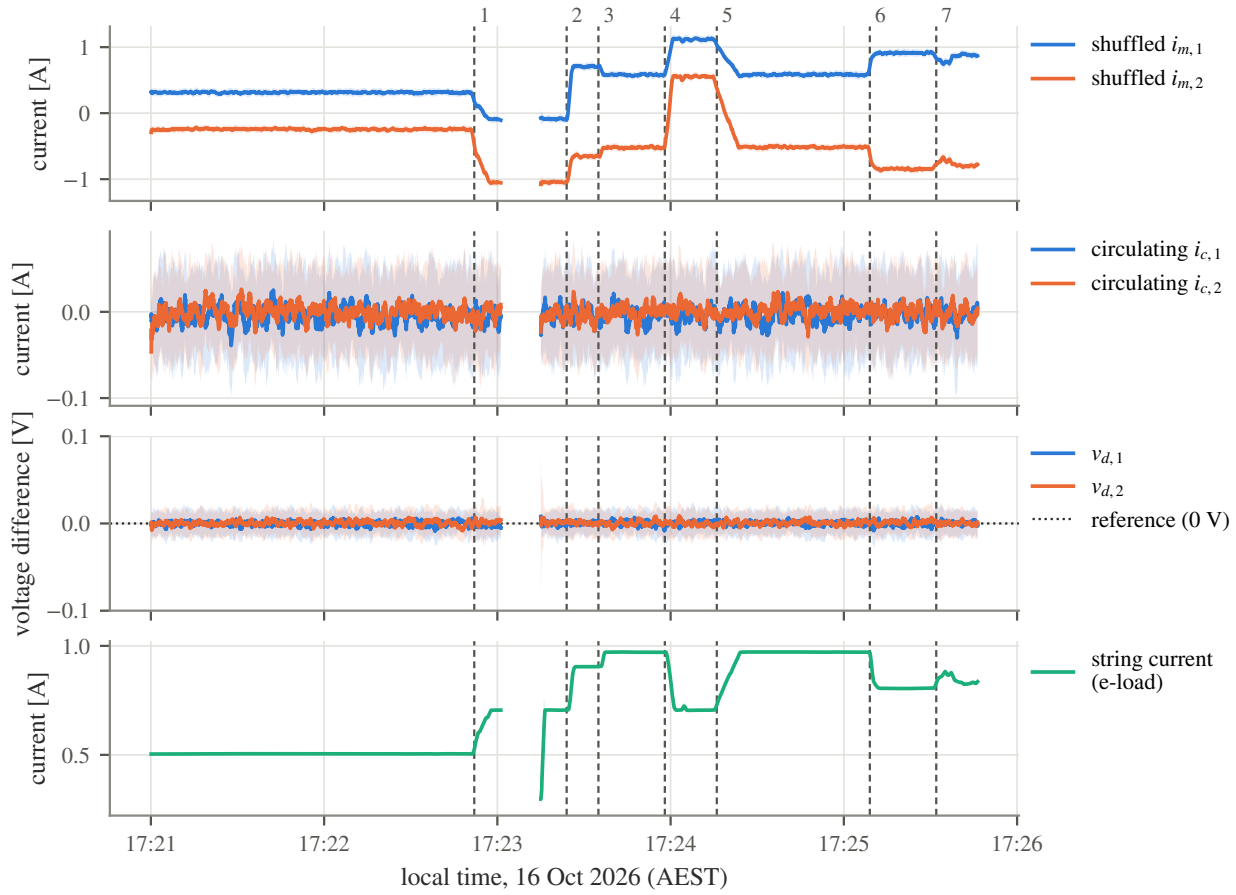


Figure 16: Bench, 300 kHz closed loop, $k = 0.3$ (B06): 1 s means with the $\pm 1\sigma$ spread of the 44 Hz samples. The dashed lines 1–7 are step changes of the supply channels; the shuffled currents move, the voltage differences and circulating currents do not.

peak to peak) for 6 s before it was stopped, so the 300 kHz gain set loses margin above $k \approx 0.5$. Applying a new gain scale while the loop was running tripped the firmware three times (table 18): once on module under-voltage and twice on a main-loop stall. Each trip switched the legs off and latched FAULT as designed, but gains should be set before the loop is closed. Three 200 kHz runs (B02–B04) logged at only 11–14 Hz, so every time metric there is a bound of at least 70 ms.

Limitations. The board carries only two shufflers, and boards cannot be stacked, so scalability cannot be tested. The two shufflers are hard-wired together and cannot be driven independently. The top INA239 does not work, so V_{M1} is inferred from the e-load. And a change of switching frequency needs a new identification and gain set.

4.6 Solar Test

Solar testing was also a success: eleven windows and 36 min of run time, with the loop closed from $k = 0.1$ to 0.8 without a fault (table 22). Figure 17 shows the peak-sun window at $k = 0.1$. As the sun came out and the mismatch grew, the shuffled currents swung to -3.3 and -1.5 A while the 1 s means of the voltage differences stayed within ± 0.06 V. The open-loop residual of the same afternoon (-0.32 V at peak sun, -0.07 V in minimal sun) was removed in closed loop. During the shade events the current mismatch changed by 0.5–1.5 A over 2–6 s, $v_{d,2}$ deviated by less than 0.3 V and recovered, and the duties never left 0.45–0.55, far from their limits (table 21).

The top module was shaded to almost zero current (a shirt over about a third of the panel): by KCL it carried ≈ 0 A in every solar window, while modules 2 and 3 carried up to 2.5 A. The string current

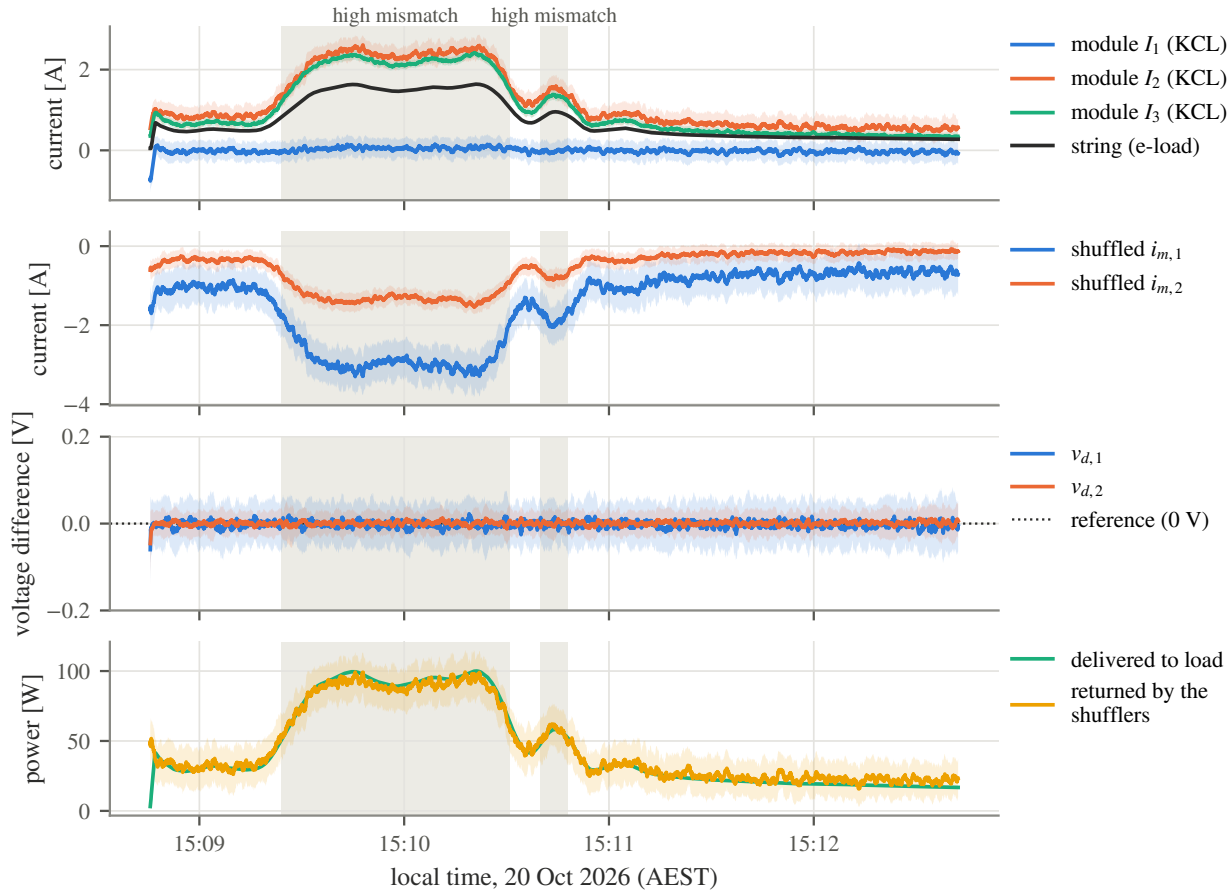


Figure 17: Carpark string at peak sun, 200 kHz closed loop, $k = 0.1$ (S02, 20 Oct 2026): module currents rebuilt by KCL, shuffled currents, voltage differences and the power delivered to the 61 V load and returned by the shufflers (1 s means, $\pm 1\sigma$ bands; shaded spans are the detected high-mismatch periods).

settled at the mean module current, as the model predicts, so the shufflers delivered the other two modules' power to the 61 V load, up to 99 W during S02. A plain series string could not have supplied that load at all, so the energy returned equals the energy delivered: $E_{ret} = 10.6\,\text{Wh} \approx E_{load}$ over the 36 min (table 20). The measured ratio $E_{load}/E_{modules}$ of 99.2–100 % closes by construction, because V_{M1} comes from the e-load and the module currents from KCL, so it is not a converter efficiency.

The weather was the main limitation. After 15:48 the sun was minimal (P_{load} 2–13 W), which allowed the gain sweep but not enough mismatch power to stress the loop. The closed-loop response to panel dynamics is therefore validated at low power, and otherwise rests on the models.

4.7 Synthesising Results

Figure 18 (and table 24, appendix K) compare the +1 V reference step across the three models. The fitted time constant follows the design $1/(k\,2\pi\,150)$ at every gain scale: within 0.86–1.22× in the analytical model, 0.52–1.17× in QSPICE and 0.38–1.10× in the Simulink twin. The models drift faster than the design at high k, where the plant's own dynamics add to the integrator. Every one of these time constants is shorter than one 23 ms telemetry frame, so the hardware cannot resolve the step response. What it does resolve agrees with the models: the DC gain (open-loop duty steps), zero steady-state error, a noise-limited residual, and closures within one frame.

Most of the overshoot entries are zero (fig. 18(b), table 24), and that is the design working rather than missing data. Because the controller inverts the DC gain, MG is diagonal and each loop is an integrator acting on a plant of unit DC gain, which is a first-order closed loop, and a first-order response

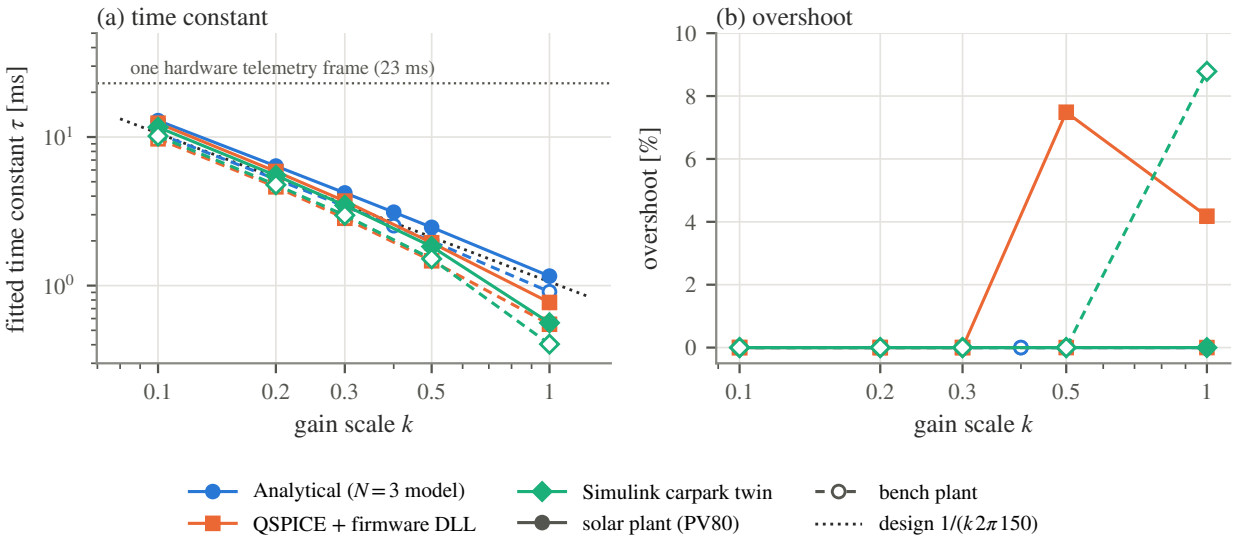


Figure 18: +1 V reference step on $v_{d,2}$ (200 kHz gain set) against gain scale: (a) fitted time constant, with the first-order design and one hardware telemetry frame; (b) overshoot. Filled markers and solid lines are the solar plant, hollow markers and dashed lines the bench plant.

cannot overshoot. Overshoot needs the dynamics the design neglects: the 2 kHz measurement filter, the one-tick delay of the 100 μs sampling, and the plant's LC modes. These only matter once the loop's time constant falls to a few times their 0.1–0.3 ms. That happens at $k \geq 0.5$ ($\tau \leq 2$ ms), where QSPICE shows 7.5 % at $k = 0.5$ and 4.2 % at $k = 1$ on the solar plant and the Simulink twin 8.8 % on the bench plant at $k = 1$. The analytical model includes the filter but not the sampling delay, so it never overshoots.

Together these results support the gap claim: the same gain matrix decouples the voltage and current loops in an averaged model, in a switching model with sensing quantisation and dead time, and on the board at three switching frequencies. Where the phases disagree there is a reason. The Simulink twin is faster than the design at $k \geq 0.5$ because its averaged plant has no R_{sw}. QSPICE overshoots at high k through the sampling delay and the filter. And the 300 kHz gain set, identified at the bench operating point, loses margin on hardware earlier than the models suggest ($k = 0.6$ oscillation).

The risk, ethics and sustainability outcomes were met. There was no injury and no damaged component; U10's isolator was wired backwards before any testing. Every fault was latched by the firmware, all data stayed local on the Pi, and the string returned 10.6 Wh with one module producing nothing. The supervisor's feedback to push the controller algorithms into QSPICE, start rapidly testing early on in either simulink / qspice, support of learning from textbooks and papers and a strong emphasis on safety shaped the methods and direction of the project.

5 Conclusion

This project asked how the coupled dynamics of several interleaved buck-boost power shufflers in one string can be modelled and controlled. The answer is the N-module averaged model. It splits each shuffler into a shuffled mode and a circulating mode, and it gives the string's DC gain in closed form, $v_d \approx -\bar{V} u_c$ and $i_c = (\bar{V}/R_c) u_d$, with neighbour coupling below one per cent. Inverting that gain gives a decoupling integral controller on $[v_d, i_c]$: it commands the shuffled currents so the string equalises, and a partially shaded string reaches its maximum power point when the module voltages are equal, which the loop enforces.

The controller was carried from the analytical model through Simulink and QSPICE to the firmware, as one C core and one results pipeline, and validated on the bench and in the sun. On hardware it holds

the module-voltage differences within $\pm 0.010\,\mathrm{V}$ and the circulating currents within $\pm 0.07\,\mathrm{A}$, limited by sensing noise. Its bandwidth follows the design $\tau = 1/(k\,2\pi\,150)$ to within the spread of the three simulations (0.4–1.2×). Across 21 test windows on two days the loop closed from $k = 0.1$ to 0.8 in the sun without a fault. The only instability was deliberately provoked ($k = 0.6$ at 300 kHz), and the only trips came from live gain changes. With the top panel shaded to almost zero current, the shufflers delivered up to 99 W and 10.6 Wh over the afternoon to a 61 V load that a plain series string could not have supplied. The averaged model's one real error, a fivefold over-estimate of the current-loop gain, was traced to a switching resistance $R_{sw} = 64\,C_p f_{sw}^3 (L+M)^2$ and caught in QSPICE before any hardware ran. The result is a complete, reproducible system and a starting point for further work on MIMO control of DPP strings.

Future work. Scalability comes first: determine how the relationships change when more shufflers act in one string, or when a shuffler handles a varying number of panels, which needs boards that can be chained and driven independently. The gain calculation for a new switching frequency can now be streamlined, since $R_{sw}(f_{sw})$ is a formula. Other controllers should be generated and compared against this one (LQR, MPC, an observer-based design, and the extended-Kalman-filter approach of [22]), as should a cost-function or machine-learning search for optimal gains. The firmware should move from its superloop to a real-time operating system, as FreeRTOS was planned, so that the control, sensing, safety and telemetry tasks are scheduled with explicit priorities and deadlines. Three firmware items were found in testing: refuse `C,APPLY` while the loop is running, add the COMP/DAC hardware over-current trip, and rework U10's isolator. Further work should also test other panel arrangements and dynamic configurations, and applications outside solar (mining, computing, industrial power and batteries), including whether the technology would win a solar car race.

Recommendations. The most immediate work is more data: stress-test the system on a very sunny day with every panel connected. To measure efficiency properly, add an MPPT algorithm, run it on the Raspberry Pi, which already controls the load over USB, and keep the load's limits in mind if the same set-up is reused. The electrical safety limits restrict which experiments can be run, so they should be designed around from the start. The system favours high switching frequencies, so GaN devices are worth revisiting. Mechanically, downsize the heatsink, improve the fasteners, reconfigure the PCB and redesign the enclosure, and consider consolidating the microcontroller and the microcomputer onto one board. Finally, hands-on development was the fastest route to understanding: anyone continuing the project should start from the GitHub repository, its manufacturing notes and its user guides.

References

[1] "World Energy Outlook 2025 – Analysis," Nov. 2025.

[2] Clean Energy Council, "Clean Energy Australia Report 2025," industry Report, Clean Energy Council, Melbourne, May 2025.

[3] "Quarterly Carbon Market Report December quarter 2025 | Clean Energy Regulator," Feb. 2026.

[4] M. Kasper, D. Bortis, and J. W. Kolar, "Classification and Comparative Evaluation of PV Panel-Integrated DC–DC Converter Concepts," *IEEE Transactions on Power Electronics*, vol. 29, pp. 2511–2526, May 2014.

[5] C. Olalla, C. Deline, D. Clement, Y. Levron, M. Rodriguez, and D. Maksimovic, "Performance of power-limited differential power processing architectures in mismatched PV systems," *IEEE Transactions on Power Electronics*, vol. 30, no. 2, pp. 618–631, 2014.

[6] G. R. Walker, J. Xue, and P. Sernia, "PV string per-module maximum power point enabling converters," in *Australasian Universities Power Engineering Conference, AUPEC'03*, 2003.

[7] T. Esram and P. L. Chapman, "Comparison of photovoltaic array maximum power point tracking techniques," *IEEE Transactions on energy conversion*, vol. 22, no. 2, pp. 439–449, 2007.

[8] M. Sarvi and A. Azadian, "A comprehensive review and classified comparison of MPPT algorithms in PV systems," *Energy Systems*, vol. 13, pp. 281–320, May 2022.

[9] N. Belhaouas, M.-S. A. Cheikh, P. Agathoklis, M.-R. Oularbi, B. Amrouche, K. Sedraoui, and N. Djilali, "PV array power output maximization under partial shading using new shifted PV array arrangements," *Applied Energy*, vol. 187, pp. 326–337, 2017.

[10] O. Bingöl and B. Özkaya, "Analysis and comparison of different PV array configurations under partial shading conditions," *Solar Energy*, vol. 160, pp. 336–343, 2018.

[11] I. M. Mehedi, Z. Salam, M. Z. Ramli, V. J. Chin, H. Bassi, M. J. H. Rawa, and M. P. Abdullah, "Critical evaluation and review of partial shading mitigation methods for grid-connected PV system using hardware solutions: The module-level and array-level approaches," *Renewable and Sustainable Energy Reviews*, vol. 146, p. 111138, 2021.

[12] J. M. Riquelme-Dominguez and S. Martinez, "Systematic evaluation of photovoltaic MPPT algorithms using state-space models under different dynamic test procedures," *IEEE Access*, vol. 10, pp. 45772–45783, 2022.

[13] P. S. Rao, G. S. Ilango, and C. Nagamani, "Maximum power from PV arrays using a fixed configuration under different shading conditions," *IEEE journal of Photovoltaics*, vol. 4, no. 2, pp. 679–686, 2014.

[14] H. Jeong, S. Park, J.-H. Jung, T. Kim, A.-R. Kim, and K. A. Kim, "Segmented differential power processing converter unit and control algorithm for photovoltaic systems," *IEEE Transactions on Power Electronics*, vol. 36, no. 7, pp. 7797–7809, 2020.

[15] N. Hui, "Development of MPPT Control Algorithm," EGH400-2 Final Report, Bachelor of Engineering (Mechatronics), School of Electrical Engineering and Robotics, Queensland University of Technology, 2020.

[16] M. Kasper, S. Herden, D. Bortis, and J. W. Kolar, "Impact of PV string shading conditions on panel voltage equalizing converters and optimization of a single converter system with overcurrent protection," in *2014 16th European Conference on Power Electronics and Applications*, pp. 1–10, IEEE, 2014.

[17] C. Olalla, C. Deline, and D. Maksimović, "Modeling and simulation of conventionally wired photovoltaic systems based on differential power processing SubMIC-enhanced PV modules," in *2014 IEEE 15th Workshop on Control and Modeling for Power Electronics (COMPEL)*, pp. 1–9, IEEE, 2014.

[18] H. Jeong, H. Lee, Y.-C. Liu, and K. A. Kim, "Review of differential power processing converter techniques for photovoltaic applications," *IEEE Transactions on Energy Conversion*, vol. 34, no. 1, pp. 351–360, 2018.

[19] C. Li and J. A. Cobos, "Classification of differential power processing architectures based on VA area modeling," *IEEE Journal of Emerging and Selected Topics in Power Electronics*, vol. 10, no. 6, pp. 7849–7866, 2021.

[20] N. G. F. Dos Santos, J. R. R. Zientarski, and M. L. da Silva Martins, "A two-switch forward partial power converter for step-up/down string PV systems," *IEEE Transactions on Power Electronics*, vol. 37, no. 6, pp. 6247–6252, 2021.

[21] A. M. Milasi, R. M. Milasi, and S. N. Lordejani, "Robust control of an interleaved DC-DC buck converter with coupled inductors," *International Journal of Dynamics and Control*, vol. 12, pp. 3769–3777, Oct. 2024.

[22] E. Kabalci and A. Boyar, "Highly efficient interleaved solar converter controlled with extended kalman filter MPPT," *Energies*, vol. 15, no. 21, p. 7838, 2022.

[23] X. Huang, F. C. Lee, Q. Li, and W. Du, "High-frequency high-efficiency GaN-based interleaved CRM bidirectional buck/boost converter with inverse coupled inductor," *IEEE Transactions on Power Electronics*, vol. 31, no. 6, pp. 4343–4352, 2015.

[24] M. Guacci, J. A. Anderson, K. L. Pally, D. Bortis, J. W. Kolar, M. J. Kasper, J. Sanchez, and G. Deboy, "Experimental characterization of silicon and gallium nitride 200 V power semiconductors for modular/multi-level converters using advanced measurement techniques," *IEEE Journal of Emerging and Selected Topics in Power Electronics*, vol. 8, no. 3, pp. 2238–2254, 2019.

[25] E. I. Batzelis, G. Anagnostou, and B. C. Pal, "A state-space representation of irradiance-driven dynamics in two-stage photovoltaic systems," *IEEE Journal of Photovoltaics*, vol. 8, no. 4, pp. 1119–1124, 2018.

[26] D. Lin, X. Li, and S. Ding, "The state-space design research of MPPT based on reinforcement learning in PV system," in *2021 IEEE 1st International Power Electronics and Application Symposium (PEAS)*, pp. 1–6, IEEE, 2021.

[27] M. U. Khan, A. F. Murtaza, A. M. Noman, H. A. Sher, and M. Zafar, "State-Space Modeling, Design, and Analysis of the DC-DC Converters for PV Application: A Review," *Sustainability*, vol. 16, p. 202, Jan. 2024.

[28] M. Assabaa, A. Bouchahed, A. Draidi, B. Mehimmedetsi, and A. Belhani, "Enhancing linear quadratic regulator and proportional-integral linear quadratic regulator controllers for photovoltaic systems," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 15, no. 3, pp. 2713–2725, 2025.

[29] R. D. Middlebrook and S. Ćuk, "A general unified approach to modelling switching-converter power stages," in *IEEE Power Electronics Specialists Conference (PESC)*, pp. 18–34, 1976.

[30] J. G. Kassakian, D. J. Perreault, G. C. Verghese, and M. F. Schlecht, *Principles of Power Electronics*. Cambridge: Cambridge University Press, 2 ed., 2024.

[31] MathWorks, "Simulink: simulation and model-based design," 2026.

[32] D. Seal, "The System Engineering "V" - Is It Still Relevant In the Digital Age?," 2018.

[33] Qorvo, "QSPICE circuit simulator," 2023.

[34] X. Li, H. Wen, Y. Hu, Y. Du, and Y. Yang, "A comparative study on photovoltaic MPPT algorithms under EN50530 dynamic test procedure," *IEEE Transactions on Power Electronics*, vol. 36, no. 4, pp. 4153–4168, 2021.

[35] KickAss Products, "KAUSSP80 80 W 12 V fixed solar panel, specification," 2024.

[36] TDK Electronics, "ERU chokes – ERUC23, SMT flat wire coupled inductor – B82559A*M023." https://product.tdk.com/system/files/dam/doc/product/inductor/inductor/high/data_sheet/b82559sxm023.pdf, 2023. Accessed Oct. 22, 2026.

[37] STMicroelectronics, "STM32G4 series reference manual (RM0440) and STM32G474xE datasheet." https://www.st.com/en/microcontrollers-microprocessors/stm32g474re.html, 2023. Accessed Oct. 22, 2026.

[38] STMicroelectronics, "MB1367 NUCLEO-G474RE schematic pack." https://www.st.com/resource/en/schematic_pack/mb1367-g474re-d01-schematic.pdf, 2020. Accessed Oct. 22, 2026.

[39] Texas Instruments, "TMCS1126-Q1 AEC-Q100, precision 500 kHz Hall-effect current sensor with reinforced working voltage, overcurrent detection and ambient field rejection, datasheet rev. C." https://www.ti.com/lit/ds/symlink/tmcs1126-q1.pdf, 2024. Accessed Oct. 22, 2026.

[40] Texas Instruments, "INA239-Q1 AEC-Q100, 85-V, 16-bit, high-precision power monitor with SPI interface, datasheet rev. A." https://www.ti.com/lit/ds/symlink/ina239-q1.pdf, 2022. Accessed Oct. 22, 2026.

[41] Raspberry Pi Ltd, "Raspberry Pi 4 Model B datasheet (RP-008341-DS-1)." https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008341-DS-1-raspberry-pi-4-datasheet.pdf, 2024. Accessed Oct. 22, 2026.

[42] Rohde & Schwarz, "R&S NGE100 power supply series, user manual," 2022.

[43] Rigol Technologies, "DL3000 series programmable DC electronic load, datasheet and programming guide," 2021.

A Research Project Timeline

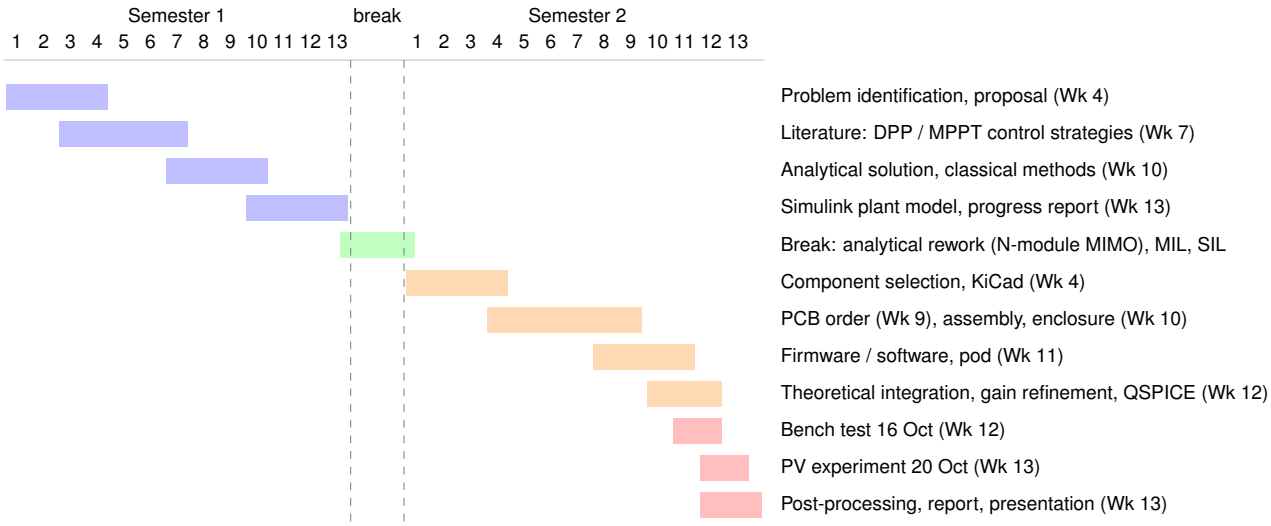


Figure 19: Project timeline across the two semesters (teaching weeks) with the milestone week of each phase.

Table 6: Development phases and their V-model counterpart.

Phase	Pipeline	V-model	Primary artefact
0	Background research	Concept of operations	proposal, literature review
1	Analytical solution	System requirements	`nstring.py`, notebooks, appendix C
2	Model-in-the-loop	High-level design	`power_shuffler_sim.slx` (4 modules)
3	Component selection	Detailed design	KiCad Version 0 PCB, BOM
4	Firmware / software	Development	`6_RTOS` (C), `7_Pi` (Python, Docker)
5	Software-in-the-loop	Unit verification	C Caller blocks, golden replay
6	QSPICE integration	Subsystem verification	QSPICE integrated model + control DLL
7	Bench test	System verification	16 Oct 2026, 10 windows (table 22)
8	Solar test, partial shade	System validation	20 Oct 2026, 11 windows (table 22)
9	Post-processing / synthesis	Acceptance	`8_Results/analysis` (`make all`)

Three things changed from the proposal. The first PCB moved from Semester 1 Week 10 into Semester 2 (progress report, Appendix A), and the planned Version 1 and Version 2 boards collapsed into a single Version 0 board, which meant compromising on PCB revisions and features. String MPPT integration was deferred, because the electronic load performs it. Interim milestones were reviewed weekly with the supervisor.

B Risk Register

Likelihood and consequence are rated Low (L), Medium (M), High (H) or Critical (C); the overall rating is the higher of the two. The register is the proposal's, with a *Status* column recording what happened.

Table 7: Milestones and progress achieved (proposal → progress report → delivery).

Semester	Week	Milestone	Progress / change
1	4	Problem identification, proposal	Research question, objectives, risk register, Gantt (34 weeks)
1	7	Existing DPP / MPPT control strategies	Literature review (Appendix A of the proposal), gap statement
1	10	Analytical solution using classical methods	Single-shuffler averaged model, equilibrium, linearisation, cascaded PI; marginal stability found
1	13	Simulink plant model	4-module PV string, P–V validation, inner current loop attempt (instability under shade)
Break	–	Analytical rework, MIL, SIL	MIMO reformulation (LQR + integral, then decoupling PI), N-module string model, MIL 7-test suite, SIL C core + golden replay
2	4	Component selection	STM32G474 Nucleo, TMCS1126, INA239, Raspberry Pi 4B; KiCad Version 0
2	9	PCB order	PCBWay board + stencil, components, reflow
2	10	Enclosure development	Enclosure, heatsink, pod wiring (IR, LCD, LED, DS18B20)
2	11	Firmware / software	Bare-metal C application, control core, telemetry contract, Pi stack (Docker InfluxDB + Grafana)
2	12	Theoretical integration / gain refinement	QSPICE integrated model with the control DLL; gain sets per f_{sw}; R_{sw} identified
2	12	Bench test	16 Oct 2026: 200/300/400 kHz, open and closed loop (table 22)
2	13	PV experiment	20 Oct 2026: 3 × KAUSSP80 in the carpark, $k = 0.1 \ldots 0.8$
2	13	Post-processing	Results pipeline (`make all`), REPORT.md, this report, presentation

Table 8: Project risk register with outcome.

#	Risk	Category	L	C	Rating	Mitigation / monitoring	Status at delivery
1	Scope creep	Project	H	H	H	Strict change control; new requirements deferred to the next revision; supervisor sign-off for additions	Materialised: one PCB revision only, MPPT deferred to the load, features compromised (slide: "had to compromise on PCB revisions and features")
2	No PV hardware available	Resource	M	C	H	Procure panels early; fallback QSPICE validation	Mitigated: 3 × KAUSSP80 80 W panels, DL3021 load; EcoFlow not used (V_{oc} 75 V > 60 V input)
3	Electronics failure	Hardware	M	H	H	Overload protection; spares; one-month buffer	Partly: U10 (top INA239) unusable (isolator wired reversed); worked around with V_{M1} from the e-load
4	Firmware failure	Control sys.	M	H	H	Simulation first (SIL, QSPICE DLL); unit tests; fault latching	Materialised in a contained way: 3 trips on live gain changes (UV, main-loop stall), all latched safely; E-stop and state telemetry worked
5	Monitoring software inaccuracy	Software	L	M	L	Sensor noise filtering; cross-validate against known points	Telemetry at 44 Hz is noise-limited; rise/settle reported as bounds; B02–B04 logged at 11–14 Hz
6	Data loss	Data	L	H	M	GitHub, OneDrive, local backups	No loss; every window re-exportable from InfluxDB
7	Unable to access data	Data	L	M	L	Shared cloud repositories, two admins	OK
8	Electrical hazard (50 V DC class)	Safety	M	C	C	QUT HSE protocols, PPE, isolation, formal risk assessment	No incident; 15 A fuse, OC/OV/UV envelopes, hardware + software E-stop
9	Sunburn (outdoor testing)	Safety	H	L	M	Sunscreen; sessions outside 10–14 h where practical	Carpark session 12:30–16:30 with shade for the operators

#	Risk	Category	L	C	Rating	Mitigation / monitoring	Status at delivery
10	Fire risk	Safety	L	C	H	Extinguisher; rated fuses; never unattended	No incident; heatsink and INA die temperatures logged (overtemperature trip at 85 °C, Pi E-stop at 80 °C)
11	Assignment deadline collision	Project	H	M	H	One-month buffer; weekly review	Buffer consumed by the PCB deferral; delivery on time
12	Skill gap between team members	Team	M	M	M	Division by domain; knowledge transfer at reviews	Worked: control vs power electronics split; QSPICE as the hand-over interface
13	Control–hardware integration failure	Control sys.	M	H	H	QSPICE validates firmware–hardware interaction first	Mitigated: the firmware control DLL in QSPICE caught the 5× current-gain error of the averaged model before hardware
14	MPPT fails to track global MPP	Control sys.	M	H	H	Validate across all shading scenarios in simulation	Descoped: string MPPT delegated to the load; equalisation (the enabling condition) validated
15	Intellectual property dispute	Legal	L	H	M	Timestamped GitHub; monitor IP Australia	No issue
16	Poor commercial adoption	Strategic	L	L	L	Outside scope	–
17	Data breach / cybersecurity	Data	L	M	L	No personal data; private repos; credentials excluded	Pi services local-only; params page has no authentication (note for future work)
18	Team member illness	Team	M	M	M	Buffer; control work proceeds in isolation	No impact
19	Safe outdoor testing (rain, wind)	Safety	M	M	M	Weather check; enclosure; abort criteria	Dry day; minimal sun after 15:48 limited the closed-loop stress test
20	Inaccurate model	Control sys.	M	H	H	Identify the plant at each f_{sw}; compare three models	Materialised and resolved: R_{sw} found (0.29 / 0.80 / 1.70 Ω), gain sets re-identified per f_{sw}

#	Risk	Category	L	C	Rating	Mitigation / monitoring	Status at delivery
21	Sensor faults	Hardware	M	M	M	Zero-cal at boot; plausibility checks; redundancy via KCL	U10 unusable; ADC phase-B fix on 16 Oct; KCL reconstruction cross-checks the module currents

C Analytical Solution

C.1 Single shuffler averaged model (lead-up, Semester 1)



(a) Topology and sign conventions.



(b) Coupled inductor model: $L_k = 0.5L_3$, $L_m = L_2 - 0.5L_3$; ERUC23-4R1K: R_{DC} 1.85 mΩ, L_k 2.05 µH, L_m 5.85 µH [36].

Figure 20: Single-shuffler model elements.

State, inputs. $x = [I_{L1}, I_{L2}, V_{C1}, V_{C2}]^T$, $u_d = [d_A, d_B]^T$, $u_e = [I_{PV1}, I_{PV2}, I_{LOAD}]^T$. $I_{Lk} > 0$ flows $V_{mid} \to L_k \to$ switch node; $V_{C1} = V_{top} - V_{mid}$, $V_{C2} = V_{mid}$.

Switching states. $2^4 = 16$ combinations, 7 forbidden (shoot-through), 9 valid; the 4 dual-switch states form the averaged basis with dwell fractions $d_A d_B$, $d_A(1-d_B)$, $(1-d_A)d_B$, $(1-d_A)(1-d_B)$; the single-switch and all-off states are dead-time transients.

Averaged model (with $\mathbf{L} = \begin{bmatrix} L & M \\ M & L \end{bmatrix}$):

$$\mathbf{L}\dot{\mathbf{I}} = \begin{bmatrix} -d_A V_{C1} + (1-d_A)V_{C2} \\ -d_B V_{C1} + (1-d_B)V_{C2} \end{bmatrix} - R\mathbf{I} \tag{8}$$

$$C_1 \dot{V}_{C1} = d_A I_{L1} + d_B I_{L2} + I_{PV1} - I_{LOAD} \tag{9}$$

$$C_2 \dot{V}_{C2} = I_{PV2} - (1-d_A) I_{L1} - (1-d_B) I_{L2} \tag{10}$$

Rule: C_1 receives current from each phase whose high-side switch conducts; C_2 supplies current to each phase whose low-side switch conducts.

Equilibrium. Volt-second balance forces $D_A = D_B \equiv D$ and

$$\frac{\bar{V}_{C2}}{\bar{V}_{C1}} = \frac{D}{1-D}, \qquad D = \frac{I_{LOAD} - I_{PV1}}{I_{LOAD} - I_{PV1} + I_{PV2}}, \qquad \bar{I} = \frac{I_{PV2}}{2(1-D)}. \tag{11}$$

The differential duty $d_A - d_B$ is a transient input only; a finite load is required for a non-trivial equilibrium.

Linearisation and modes. $\tilde{A}$ has identical rows 1,2 and columns 1,2 ($L_1 = L_2$). Common-mode duty $\tilde{u}_c = (\tilde{d}_A + \tilde{d}_B)/2$ drives $(I_\Sigma = I_{L1} + I_{L2}, V_{C1}, V_{C2})$; differential duty $\tilde{u}_\delta = \tilde{d}_A - \tilde{d}_B$ drives $I_\Delta = I_{L1} - I_{L2}$ through a pure integrator $L\dot{\tilde{I}}_\Delta = -V_\Sigma \tilde{u}_\delta$.

Stability. $\sigma(\tilde{A}) = \{0, 0, \pm j\omega_0\}$, $\omega_0^2 = 2D^2/(LC_1) + 2(1-D)^2/(LC_2)$; with $L = 8.2\,\mu$H, $C = 100\,\mu$F, $D = 0.5$: $\omega_0 \approx 34.9$ krad/s (5.56 kHz), $Z_0 = 0.286\,\Omega$. With the coupled inductor and $R_w = 1.85$ mΩ: $-67.3 \pm j26968$, -902, 0 (f_0 4.29 kHz, $\zeta = 0.0025$). The open-loop plant is marginally stable; the controller must provide all damping.

Controllability / observability. Controllable if $\bar{I} \neq 0$ (no DCM), $V_\Sigma \neq 0$ and $D \in (0,1)$. Observability: $\{V_{C1}, V_{C2}\}$ gives rank 3 of 4 (differential current hidden); adding I_{L1} gives rank 4. Per-phase current sensing is therefore mandatory (sensor set of section 3.5).

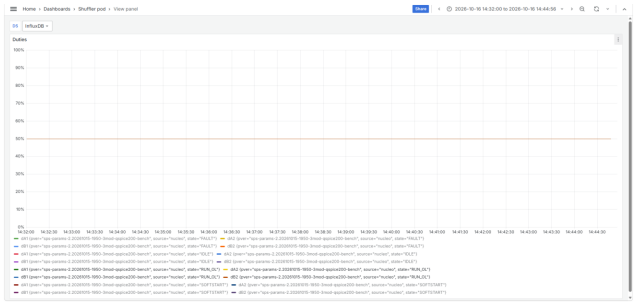
Discretisation. ZOH at $T_s = 5\,\mu$s: poles $0.9848 \pm 0.1737\,j$, 1, 1 on the unit circle.

Cascaded PI (Semester 1 design, superseded). Crossovers 10 kHz / 1 kHz / 5 kHz, PM 84°: $K_{p,i} = 0.00678$, $K_{i,i} = 42.6$; $K_{p,v} = 0.628$, $K_{i,v} = 395$; $K_{p,\delta} = 0.00339$. Three SISO loops treated the

coupled states as independent; the fixed-current-reference inner loop drove the shaded module to open circuit (progress report Fig. 8). Figure 21 shows the pole map and phase portraits that motivated the MIMO reformulation.



(a) Open-loop eigenvalues: idealised model on the $j\omega$ axis; winding resistance and coupling pull the resonant pair into the LHP.



(b) Full 4-state MIMO plant: LQR + integral pulls the resonant pair off the $j\omega$ axis into a well-damped region.



(c) Phase portraits: (a) ideal, centre; (b) passive damping, $\zeta \approx 0.003$; (c) inner current loop active, stable focus.

Figure 21: Single-shuffler stability pictures (Semester 1 and break).

Re-interpretation after the hardware convention was fixed. The sensed $I_{LA}+I_{LB}$ *is* the circulating current; relabelled, the Appendix-B model gives -77 V per unit against -76.5 measured and -200 A per unit with the $0.386\,\Omega$ battery-path resistance, so the Semester 1 derivation was right in kind (DESIGN_LOG 6.1).

C.2 N-module string model (deployed)

Variables (fig. 22). Node potentials ϕ_k, $k = 1..N+1$ (node $N+1$ is ground); module voltage $v_k = \phi_k - \phi_{k+1}$. Shuffler j spans modules j, $j+1$: rails ϕ_j, ϕ_{j+2}, midpoint ϕ_{j+1}, $\bar{V}_j = v_j + v_{j+1}$. Modal duties $u_c = d_A + d_B - 1$, $u_d = d_A - d_B$ (inverse $d_A = \frac{1}{2}(1+u_c+u_d)$, $d_B = \frac{1}{2}(1+u_c-u_d)$). Modal currents $i_m = i_A + i_B$ (shuffled; telemetry $I_{LA} - I_{LB}$ because the B sensor is reversed) and $i_c = i_A - i_B$ (circulating; telemetry $I_{LA}+I_{LB}$). Outputs $v_{d,j} = v_j - v_{j+1} = \phi_j - 2\phi_{j+1} + \phi_{j+2}$ and $i_{c,j}$.

State equations (eqs. (1) to (3)): S injects $+1$ at the midpoint and $-\frac{1}{2}$ at each rail of shuffler j; G_n is the nodal conductance of the module slopes g_k with G_{load} added at node 1; w the module short-circuit current injections. Rail currents are $\frac{1}{2} i_m(1 \pm u_c) \pm \frac{1}{2} i_c u_d$.

Inductances. As built (ERUC23 windings 1:$-$1) the circulating mode sees $L+M = 13.75\,\mu$H and the shuffled mode $L-M = 2.05\,\mu$H ($L_{self} = 7.9\,\mu$H, $M = 5.85\,\mu$H). The ripple law $\Delta i_c = \bar{V}/(2 f_{sw}(L+M))$ confirms the assignment (table 5).

Equilibrium. $D_j \bar{V}_j = v_{j+1} + \frac{1}{2} R_m I_{m,j} \Rightarrow D_j/(1-D_j) = v_{j+1}/v_j$. Charge balance on the ladder: $T(N)\, I_m = \delta I$, $T = \text{tridiag}(-\frac{1}{2}, 1, -\frac{1}{2})$, $(\delta I)_j = I_j - I_{j+1}$, with the closed form $T^{-1}_{jk} = \frac{2}{N}\min(j,k)(N-$

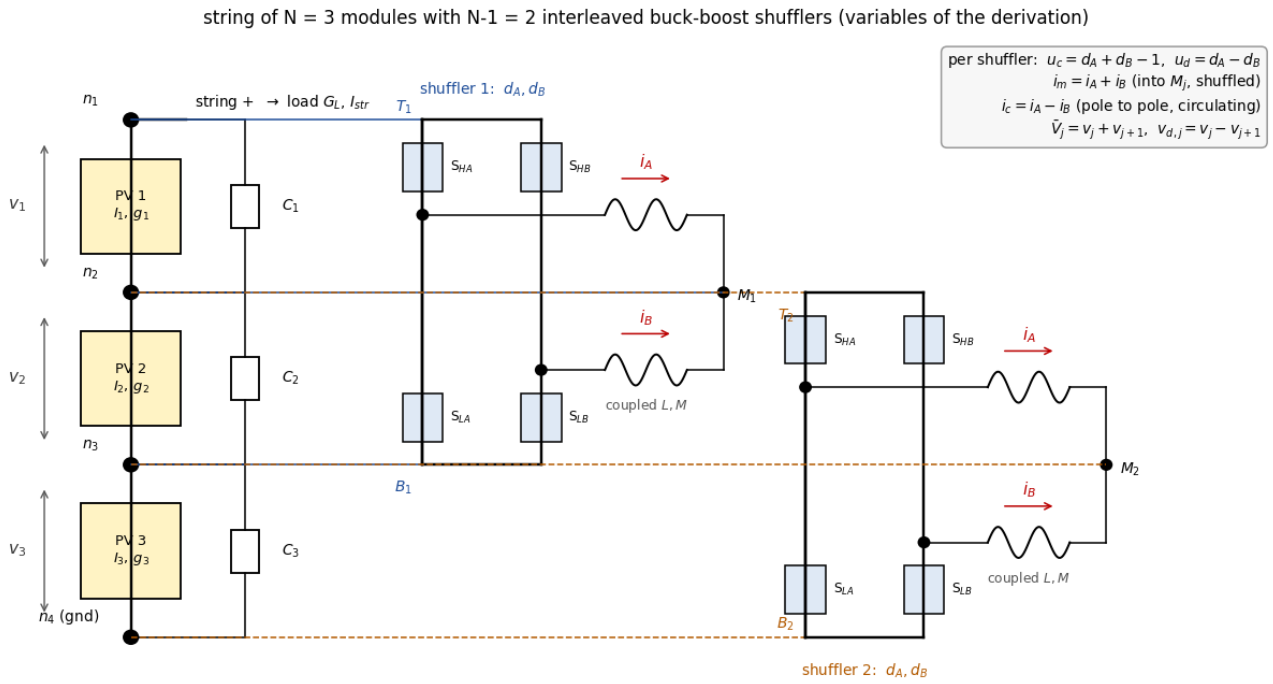


Figure 22: Derivation drawing: $N = 3$ modules and $N - 1 = 2$ interleaved buck-boost shufflers with the node potentials and per-shuffler variables of the N-module model (the small-signal version is fig. 5).

$\max(j,k)$); string current $I_{str} = I_1 - D_1 I_{m,1} = \frac{1}{N}\sum_k I_k$. For $N = 3$: $i_{m1} = (4I_1 - 2I_2 - 2I_3)/3$. Open loop at $D = 0.5$: $v_{d,j} = -R_m i_{m,j}$. Check against QSPICE S10i ($R_m = 0.29\,\Omega$): $i_m = -0.466/-0.534$ A (model) vs $-0.464/-0.539$ A (QSPICE); I_{str} 1.032 vs 1.024 A; $R_m i_m = -0.135$ V vs $v_{d,1} = -0.135$ V.

DC gain.

$$G(N) = \begin{bmatrix} G_v & 0 \\ 0 & \mathrm{diag}(\bar{V}_j/R_c) \end{bmatrix}, \qquad G_v = -(R_m I + Z_e)^{-1} Z_e \,\mathrm{diag}(\bar{V}_j), \qquad Z_e = -D_v G_n^{-1} S, \tag{12}$$

where D_v maps $\phi \mapsto v_d$. $G(2)$: $-\bar{V}/(R_m g + 1)$. $G(3)$ voltage block: diagonal $-\bar{V}(4R_m g + 3)/(4R_m^2 g^2 + 8R_m g + 3)$, off-diagonal $2R_m \bar{V} g/(\cdot)$: diagonal $\approx -\bar{V}$ with $O(R_m g)$ neighbour terms. Numbers in table 4.

Controller. $M = G^{-1}\mathrm{diag}(k_I)$ with $k_{I,v} = 2\pi \cdot 150\,\mathrm{s}^{-1}$, $k_{I,i} = 2\pi \cdot 400\,\mathrm{s}^{-1}$; $u = k\,(M_p e + M\xi)$, $M_p = M\,\mathrm{diag}(k_{p,v}, k_{p,v}, k_{p,i}, k_{p,i})$, $k_{p,v} = 1.58 \times 10^{-4}$ s, $k_{p,i} = 1.08 \times 10^{-4}$ s, $\xi \mathrel{+}= T_s e$, $T_s = 100\,\mu$s, first-order LPF on y with $f_c = 2$ kHz ($a = 2\pi f_c T_s/(1 + 2\pi f_c T_s) = 0.557$), one tick of delay; back-calculation anti-windup $\xi = M^{-1}(u_{act} - M_p e)$. Firmware layout $u = [u_{c,1}, u_{c,2}, u_{d,1}, u_{d,2}]$, $e = [e_{vd1}, e_{vd2}, e_{ic1}, e_{ic2}]$. Design: $\tau_v = 1/(k k_{I,v})$, $\tau_i = 1/(k k_{I,i})$, rise $= 2.197\tau$, 2 % settle $= 3.91\tau$.

Table 9: Design time constants vs gain scale and the closed-loop pole frequencies at $k = 1$.

k	τ_v [ms]	τ_i [ms]	rise $2.197\tau_v$ [ms]	closed-loop poles at $k = 1$
0.1	10.61	3.98	23.3	model M: 160 / 164 Hz (v_d), 593 Hz (i_c)
0.3	3.54	1.33	7.8	deployed M: 127 / 135 Hz (v_d), 582 / 604 Hz (i_c)
1.0	1.06	0.40	2.3	

Plants used in the analysis. PV80 ($N = 3$): $V = 20.5$ V, $g = 0.19$ S per module, $C = [130, 260, 130]\,\mu$F, $G_{load} = 20$ S (DL3021 CV), $R_w = 0.061\,\Omega$ (averaged) or R_{sw}. BENCH-QSPICE: $V = 23.3$ V, $g = 0$, $I = [0.8, 1.0, 1.3]$ A (all NGE103 channels in current limit, the 16 Oct point),

same C. Reduced voltage mode: $f_0 = 1/(2\pi\sqrt{L_m C_e})$ with $C_e = 173\,\mu$F gives 8.4 kHz; $\zeta = \frac{1}{2}(R_m\sqrt{C_e/L_m} + g_e\sqrt{L_m/C_e}) = 0.29$ at 61 mΩ, 1.34 at 0.29 Ω, 7.8 at 1.7 Ω.

PBH ranks (PV80). $n = 7$ states; controllable from u (7), from u_c alone (5), from u_d alone (2); observable from y (7), from v_d alone (5): circulating modes are reachable only from u_d and seen only by the current sensors; the string-sum mode (176 Hz with a current sink) is unreachable/unobservable with equal capacitors and belongs to the load / MPPT. $\mathrm{cond}(G(N))$ for PV80 grows only from 16.6 ($N = 2$) to 18.9 ($N = 8$), i.e. the decoupling scales.

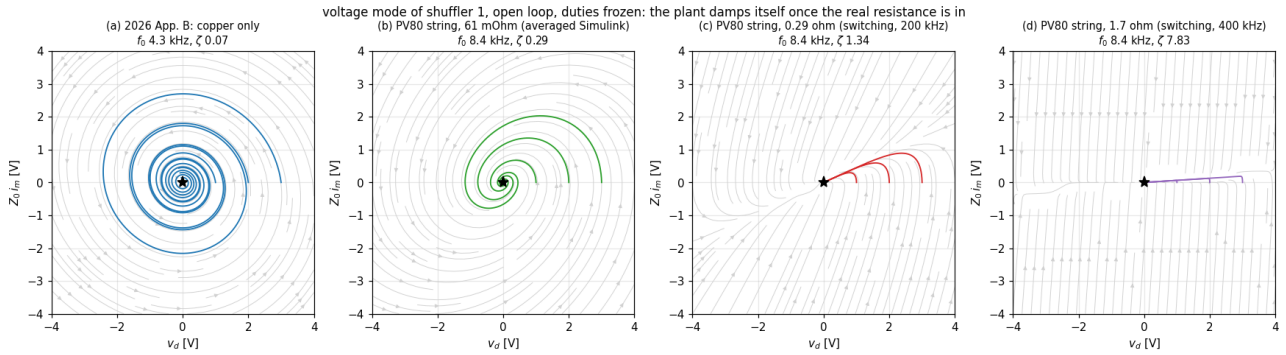


Figure 23: Open-loop phase portraits of the voltage mode in the $(v_d, Z_0 i_m)$ plane: (a) 2026 App. B copper-only model ($\zeta \approx 0.07$), (b) PV80 plant with the 61 mΩ averaged resistance (focus, $\zeta \approx 0.3$, 7–9 kHz), (c) 0.29 Ω switching plant at 200 kHz and (d) 1.7 Ω at 400 kHz (over-damped nodes). The plant damps itself once the real switching resistance is included.

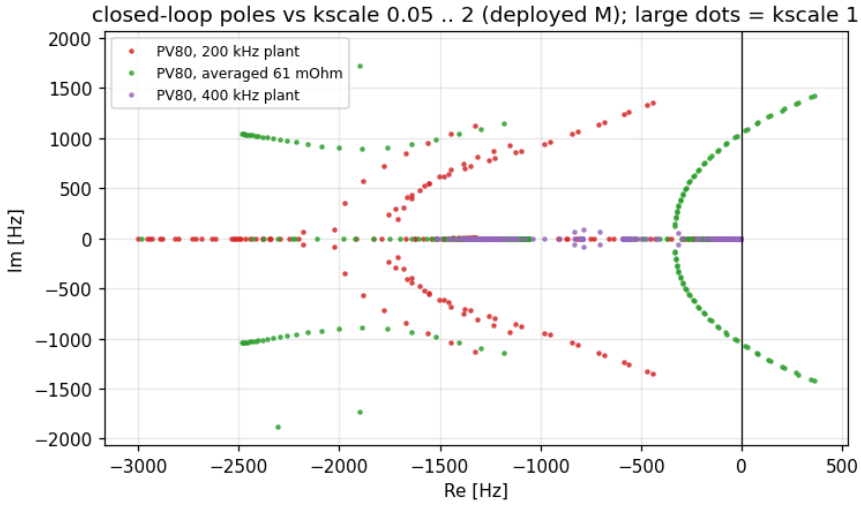


Figure 24: Closed-loop poles of the analytical $N = 3$ model with the deployed gain set as k is swept from 0.05 to 2, for the 200 kHz (0.29 Ω), averaged (61 mΩ) and 400 kHz (1.7 Ω) plants; large dots mark $k = 1$.

C.3 Switching resistance R_{sw}

The averaged 61 mΩ plant over-predicts the current-loop gain 5×. Open-loop u_d sweeps on the QSPICE device model (S10i / S10i3 / S10i4) with and without reverse recovery and device capacitances isolate the mechanism: pole-capacitance commutation during the dead time costs volt-seconds proportional to the current,

$$R_{sw} = \frac{4\bar{V}^2 C_p f_{sw}}{\Delta i^2} = 64\, C_p\, f_{sw}^3\, (L+M)^2, \tag{13}$$

which with $C_p = 2.2$ nF gives 0.21 / 0.72 / 1.70 Ω at 200 / 300 / 400 kHz; the identified gains (142 / 58 / 23 A per unit) imply 0.27 / 0.79 / 1.66 Ω; the analysis uses 0.29 / 0.80 / 1.70 Ω. The switching

twin (`nswitch.py`, exact per-configuration integration with $C_p = 2C_{oss}$) reproduces QSPICE point for point (170/165/148/111/74/66 vs 172/163/148/112/71/66 A per unit).

C.4 Open items

Unexplained 6.34 kHz / ζ 0.53 resonance seen in June 2026; C_{oss} is voltage dependent so R_{sw} should be re-measured per operating point; the $u_d \to v_d$ cross term ($-10 \ldots -25$ V per unit) is not in the model; at 400 kHz the dead time sits at the ZVS boundary and the linear model stops near $u_d \approx 0.02$.

D Controller, Supervisor and Firmware Pseudocode

Listing 1: MIMO decoupling PI, one 100 μs tick (`shuffler_ctrl.c`).

```
function ctrl_step(VM[1..N], IL_A[1..NS], IL_B[1..NS], rv[1..NS], Vref[1..NS]):
    # outputs: module-voltage differences and circulating currents
    y = [VM[1]-VM[2], ..., VM[N-1]-VM[N],  IL_A[1]+IL_B[1], ..., IL_A[NS]+IL_B[NS]]
    y_f += a * (y - y_f)                        # LPF, a = 2*pi*fc*Ts/(1+2*pi*fc*Ts), fc
        ↪  = 2 kHz
    r   = [rv + Vref, 0 ... 0]  # supervisor relaxation + manual offsets; ic refs are
        ↪ zero
    e   = r - y_f
    xi_try = xi + Ts * e                        # integrator candidate
    u   = kscale * (Mp @ e + M @ xi_try)        # u = [uc1..ucNS, ud1..udNS]; M = inv(G)
        ↪  diag(kI)
    for each shuffler j:
        dA[j] = clamp(0.5 + 0.5*(u[j] + u[NS+j]), dmin, dmax)
        dB[j] = clamp(0.5 + 0.5*(u[j] - u[NS+j]), dmin, dmax)
    u_act = [dA + dB - 1, dA - dB]              # what was actually applied
    xi = Minv @ (u_act / kscale - Mp @ e)       # back-calculation anti-windup (every
        ↪ tick)
    return dA, dB
```

Listing 2: Feasibility governor / supervisor, 1 ms (`shuffler_supervisor.c`).

```
function supervisor_step(IM[1..N], IL_A, IL_B, d[1..NL], en):
    if not en: beta = 0; rv = 0; return rv
    id   = IL_A - IL_B                           # shuffled current per shuffler
    imf += alpha * (IM - imf);  idf += alpha * (id - idf)      # alpha = Ts3/TF = 0.2
    dl   = max(imf) - imf                        # deficit of each module vs the
        ↪ strongest
    stress = max( max|idf| / I_RATED,  max|d - 0.5| / 0.45 )  # I_RATED = 10 A
    if   stress > S_HI: beta += Ts3 * KB   * (stress - S_HI)   # S_HI 0.9, KB 60
    elif stress < S_LO: beta -= Ts3 * KREL * (S_LO - stress)   # S_LO 0.7, KREL 6
    beta = clamp(beta, 0, BMAX)                   # BMAX 4
    for each shuffler k: rv[k] = clamp(beta * (dl[k+1] - dl[k]), -RVMAX, RVMAX)   # 30
        ↪ V
    return rv                                    # relaxes equalisation when a shuffler
        ↪ is stressed
```

Listing 3: Module-current reconstruction by KCL (`shuffler_recon.c`); the string current is unobservable so $I_N = 0$ is the reference.

```
function recon(IL_A, IL_B, dA, dB):
    for each shuffler k:
        ia = IL_A[k]; ib = -IL_B[k]             # B sensor is reversed on the board
        itop[k] = dA[k]*ia + dB[k]*ib           # averaged current drawn at the top rail
        ic[k]   = ia + ib                       # injected at the midpoint
        ibot[k] = ic[k] - itop[k]               # returned at the bottom rail
```

```
    IM[N] = 0
    for j = N-1 down to 1:
        IM[j] = IM[j+1] + ic[j] - (j < NS ? itop[j+1] : 0) - (j >= 2 ? ibot[j-1] : 0)
    return IM                                   # relative module currents (offset-free
        ↪ deficits)
```

Listing 4: Adaptive-step perturb-and-observe MPPT, 1 ms (`shuffler_mppt.c`); compiled in but disabled on hardware because the load performs the string MPPT.

```
function mppt_step(Vstr, Istr, beta, en):
    if not en: Iref = IREF0; dir = +1; boot = 1; return Iref      # IREF0 8 A
    acc += Vstr * Istr; n += 1
    if n == NWIN:                                                  # 20 ticks = 20 ms
        P = acc / NWIN; acc = 0; n = 0
        if |beta - beta0| > 1e-3: boot = 1          # governor moved: hold the
        ↪ reference
        elif boot: Pprev = P; Iref += dir * S0; boot = 0           # S0 0.25 A
        else:
            dP = P - Pprev
            step = clamp(KA * |dP|, SMIN, SMAX)                    # 0.01, 0.05 .. 1 A
            if   dP >  DB: Iref += dir * step                      # DB 1 W
            elif dP < -DB: dir = -dir; Iref += dir * step
            Pprev = P
        beta0 = beta; Iref = clamp(Iref, IMIN, IMAX)               # 2 .. 8.5 A
    return Iref
```

Listing 5: Control ISR, 10 kHz (HRTIM master repetition interrupt, `app.c`).

```
on HRTIM_master_repetition:                     # every 20 / 30 / 40 PWM periods
    Sensors_Tick()  # 80-sample means per channel; OC / rail / DMA-stall latch
    IL = Sensors_Current_A()                    # (code - zero) * 3300/4096 mV / (100
        ↪ mV/A)
    if Sensors_FaultLatched(): trip(FLT_OC_LATCH)
    prt_isr_service()  # pending C,APPLY: swap gains, xi = Minv * u_act(d)
    VM = snapshot of the 1 kHz INA task (VM1 from vstr_pi - VM2 - VM3 when vm1_src = 2)
    if state == SOFTSTART:
        d = ramp(d_start, target, n_tick / 1500)          # 150 ms linear ramp
    elif state == RUN and mode is CL:
        out = stack_step({VM, IL, Vstr = sum VM, Vref, en_sup, en_mppt = 0})
        d = clamp(out.d, d_prev - slew, d_prev + slew)    # slew 0.01 per tick
    elif state == RUN:  d = open-loop duties (C,DUTY / C,DUTYS)
    else:               d[i] = Vbot / (Vtop + Vbot)       # pre-bias while idle (0.5 if
        ↪ unknown)
    for each leg: hw_set_duty(leg, d)          # clamp to the f_sw limits, write CMP1 (
        ↪ preloaded)
```

Listing 6: Stack scheduler (`stack.c`): controller every tick, supervisory layers every 10th.

```
function stack_step(in):
    if tick % STACK_DECIM == 0:                  # 10 -> 1 kHz
        IM   = recon(in.IL_A, in.IL_B, dA, dB)
        rv   = supervisor_step(IM, in.IL_A, in.IL_B, d, in.en_sup)
        Iref = mppt_step(in.Vstr, in.Istr, beta, in.en_mppt)
    dA, dB = ctrl_step(in.VM, in.IL_A, in.IL_B, rv_delayed, in.Vref)   # rv through a
        ↪ unit delay
    tick += 1
    return {dA, dB, rv, beta, stress, IM, Iref}
```

Listing 7: Safety task and state machine, 100 Hz (`app.c`); `trip()` may run from any context.

```
every 10 ms:
    kick IWDG                                       # 0.3 s window
    if state in {RUN, SOFTSTART}:
        if E-stop pin low:                          trip(FLT_ESTOP)
        OV / UV / string-OV counters: 3 consecutive samples -> trip(code)   # 30 V, 3 V
        ↪  (CL), 85 V
        if CL and now - last_rx > wdg_s:            trip(FLT_COMMS_WDG)         # 5 s
        ↪ without a C line
        if CL and vm1_src == 2 and VM1 stale:       trip(FLT_VSTR_STALE)
        if any INA die > 85 C:                      trip(FLT_OVERTEMP)
        if CL and a needed VM unreadable:           trip(FLT_INA)
    switch state:
        IDLE:      if enable and mode != IDLE: pins -> HRTIM; arm synchronised enable;
        ↪ state = SOFTSTART
        SOFTSTART: if not enable: state = STOP;  elif ramp done: stack_init(); state =
        ↪ RUN
        RUN:       if not enable or mode == IDLE: state = STOP
        STOP:      legs off; enable = 0; state = IDLE
        FAULT:     hw_outputs_off()                 # every 10 ms until C,CLEAR
    send S line on every state change; send F line once per latched fault

function trip(code):
    hw_outputs_off(); gates low                     # first, from any context
    faults |= code; enable = 0; state = FAULT
```

Listing 8: Soft start (`app.c` + `hw_legs_enable_sync`).

```
while IDLE: every leg pair is held at d0 = Vbot / (Vtop + Vbot)    # matches the
        ↪ natural midpoint
on enable: outputs switch on at master CMP1 = PER/4 (centred first pulse, 16-try
        ↪ fallback window)
SS_TICKS = 150 ms * 10 kHz = 1500
tick 0: d_start[i] = d_applied[i]
each tick n: frac = min(n / SS_TICKS, 1)
    target = 0.5 if CL else OL duty[i]
    d[i] = d_start[i] + (target - d_start[i]) * frac
n == SS_TICKS: ss_done -> safety task moves to RUN; stack_init() so the controller
        ↪ starts from zero integrators
```

Listing 9: Pi pod main loops (`pod.py`, `eload.py`).

```
pod.py:
    RX thread: read line from /dev/ttyACM0 @ 921600 -> telemetry_parser.parse_line(line
        ↪ )
              -> InfluxDB points (telemetry / status / fault / note / cmd), batched
        ↪ 200 pts / 500 ms
    TX thread: one byte every 2 ms (firmware RX is one IRQ per byte); C,PING every 1 s
        ↪ (comms watchdog)
    main loop (0.2 s): DS18B20 heatsink temperature -> Influx (C,ESTOP if > 80 C); LCD
        ↪ state/values; RGB LED by state
    control socket 127.0.0.1:5017: podctl / params_web / remote.py send commands; IR
        ↪ keys -> presets (actions.py)
eload.py:
    5 Hz:  :MEAS:VOLT? :MEAS:CURR? :MEAS:POW?  -> Influx (source = eload)
    1 Hz:  mode, levels, ranges, questionable status
    2 Hz:  C,SET,vstr_pi,<V>  -> Nucleo (VM1 source when U10 is unreadable)
```

E Firmware, Peripherals and Telemetry Contract

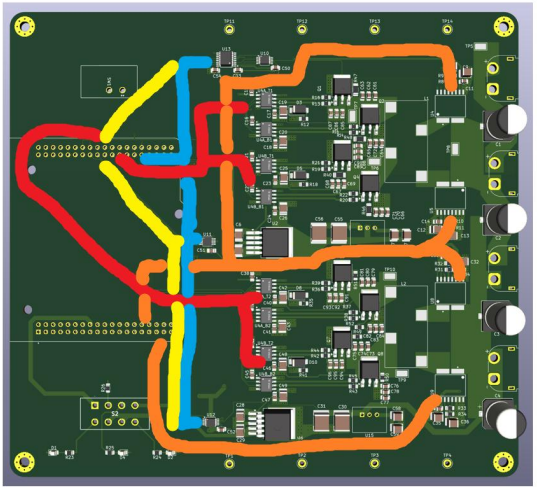
E.1 Scheduling

Table 10: Execution contexts of the firmware (bare-metal superloop plus interrupts; no RTOS).

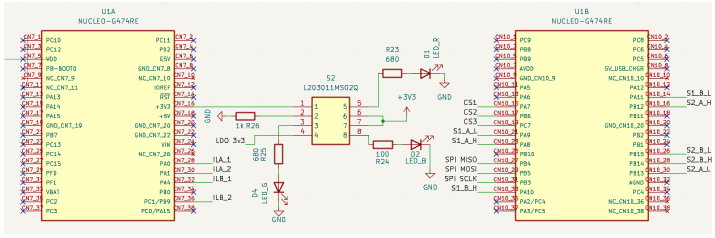
Context	Trigger / priority	Rate	Work
`app_ctrl_isr`	HRTIM master repetition IRQ, NVIC prio 1	10 kHz (T_s = 100 μs)	hall averages + OC latch, gain swap, soft-start / `stack_step` / OL duties / pre-bias, slew limit, write CMP1
`app_tick_1ms`	SysTick, prio 0	1 kHz	ms counter; main-loop stall check ($>$ 20 ms while switching $\Rightarrow$ fault 0x2000)
`app_task_sense_v`	main loop	1 kHz	INA239 $\times$3 over SPI3 $\rightarrow V_{M1..3}$, die temperatures
`app_task_safety`	main loop, every 10th pass	100 Hz	IWDG kick, OV/UV/string-OV, watchdogs, state machine
`app_task_telemetry`	every 20th pass	50 Hz	`T` frame; `S` line at 1 Hz and on state change
`app_task_cmd`	every pass	1 kHz	parse `C,...` lines, answer `A,...`
E-stop	EXTI0 on PC0, prio 0	event	`trip(FLT_ESTOP)`
LPUART1 RX	prio 6, 1 byte per IRQ	event	512-byte ring buffer

Clock: HSI 16 MHz $\rightarrow$ PLL $\rightarrow$ SYSCLK 170 MHz (AHB/APB 170 MHz, flash latency 4, boost scale); the 400 kHz build feeds the ADCs from PLL-P at 56.7 MHz. IWDG 0.3 s. ISR cost is measured with the DWT cycle counter against the 100 μs budget (Release `-O2`; `-O0` reached 75 μs idle and $>$ 100 μs in CL at 300 kHz). Toolchain: CMake $\geq$ 3.22, Ninja, arm-none-eabi-gcc (Cortex-M4F, hard float, nano specs), presets Debug/Release; `build_wsl.sh` uses the Windows STM32Cube bundle from WSL.

E.2 Board routing and current sampling

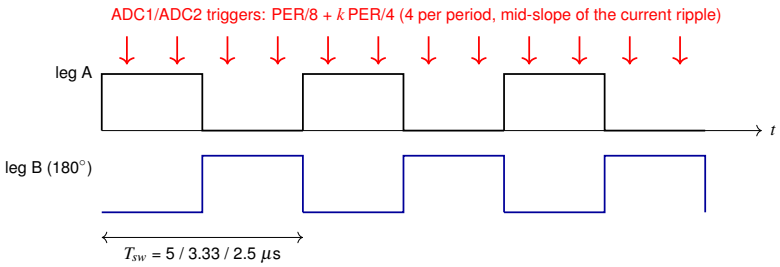


(a) Routing on the Version 0 board: PWM (red), SPI (blue), chip selects (yellow), ADC (orange), top and bottom layers.



(b) Nucleo header pinout used by the PCB.

Figure 25: Controller routing on the Version 0 PCB.



Control tick = 20 periods at 200 kHz (30 at 300 kHz, 40 at 400 kHz) = 100 μs; DMA ring of 160 half-words = one tick.
Per channel: 80-sample box average ⇒ nulls at f_{sw}, $2f_{sw}$, ... and every 10 kHz multiple (comb filter), so the controller sees the average winding current.
OC latch: $|I| > 12$ A for 2 ms, ADC code < 40 or > 4055 for 3 ticks, or a stalled DMA ⇒ outputs off within one tick.

Figure 26: Hall-current sampling scheme: four HRTIM-triggered ADC conversions per switching period, box-averaged over one control tick.

E.3 Peripherals

Table 11: Peripheral configuration (`shuffler_hw.c/h`, `sensors.c`, `ina2x9.c`, `comms.c`).

HRTIM1	$f_{HRTIM} = 170\,\text{MHz} \times 32 = 5.44\,\text{GHz}$ (184 ps). Master PER 27200 (200 kHz) / 18133 (300) / 13600 (400). Legs A–D: A legs reset on master PER, B legs on master CMP2 = PER/2 (180° interleave). Duty = leg CMP1, preloaded, update on reset. Output 1 set on the master event, reset on CMP1; output 2 is the complement from the dead-time generator. Dead time `HW_DEADTIME_NS` 100 (136 steps × 0.735 ns, both edges; was 200 ns until 16 Oct). Pins: leg 0 S1A PA8/PA9, leg 1 S1B PA10/PA11, leg 2 S2A PB12/PB13, leg 3 S2B PB14/PB15 (AF13); a constructor drives them low before `main()`. Duty limits 0.05–0.95 (200 kHz), 0.08–0.95 (300), 0.12–0.88 (400). Synchronised enable at master CMP1 = PER/4. Hardware fault inputs not used (COMP/DAC OC trip reserved, bit 0x002).
ADC1 / ADC2	12-bit, regular group of 2 ranks, trigger HRTIM ADC trigger 1 = TA CMP3/CMP4 + TB CMP3/CMP4 (4 per period at PER/8 + kPER/4). Sample time 6.5 cycles, clock PCLK/4 = 42.5 MHz, no oversampling, offset calibration at start. ADC1 IN1 = PA0 (I_{LA1}), IN2 = PA1 (I_{LA2}); ADC2 IN17 = PA4 (I_{LB1}), IN7 = PC1 (I_{LB2}). DMA1 ch1/ch2 circular half-word, 160 samples = 4 triggers × 20 periods × 2 ranks = one control tick; the ISR polls the TC flags.
Hall conversion	$I = (\overline{\text{code}} - \text{zero}) \cdot (3300/4096\,\text{mV})/S$, $S = 100$ mV/A on all four channels (B phases were 25 before 16 Oct; `params.yaml` still carries 25 for channels 1/3); nominal zero 2048 (1.65 V); boot zero-cal averages 64 ticks, falls back to 2048 and sets `FLT_ZEROCAL` if off by $> \pm 124$ codes. B-phase sensors are physically reversed. Run-time `hall_mva0..3` (± 400, $\lvert x \rvert \geq 10$), `hall_zero0..3` (1800–2300). Box average of 80 samples per channel per tick (nulls at f_{sw}, $2f_{sw}$, ... and every 10 kHz multiple). OC latch: $\lvert I \rvert >$ `oc_a` (12 A) for 20 ticks, code < 40 or > 4055 for 3 ticks, or DMA not wrapping for 3 ticks.
SPI3	Master, mode 1, 8-bit MSB-first, software NSS, prescaler 128 ⇒ 1.33 MHz; PB3 SCK, PB4 MISO, PB5 MOSI; CS1 PA7 (U12, PV3+ to GND), CS2 PB6 (U11, PV2+ to GND), CS3 PC7 (U10 behind ISO7762 U13, PV1+ to PV3+). INA239 ADC_CONFIG 0xF248 (continuous, 84 μs, AVG 1); VBUS 3.125 mV/LSB, die temperature 125 m°C/LSB; INA229 auto-detected (195.3 μV/LSB); absent devices re-probed at 1 Hz. $V_{M3} = U12$, $V_{M2} = U11 - U12$, $V_{M1} = -U10 - V_{M2}$ (U10 wired reversed) or, by default (`vm1_src` = 2), $V_{M1} = v_{str,pi} - U11$ (stale after 2 s).
LPUART1	ST-LINK VCP PA2/PA3, 921600 8N1, blocking TX, protocol version 2, firmware `shuffler-fw-0.4.1`.
GPIO	E-stop PC0 (pull-up, falling-edge EXTI, prio 0); B1 toggles enable; LD2 PA5.
Controller filter	IIR $y_f \mathrel{+}= a(y - y_f)$, $a = 0.5569$ at $f_c = 2$ kHz; optional moving average of 1–8 ticks (`filt_ma`). Supervisor filter $\alpha = 0.2$.

E.4 Fault bits

E.5 Telemetry and command contract

Telemetry frame (50 Hz, 27 fields):

```
T,ms,state,fault(hex),VM1,VM2,VM3,ILA1,ILB1,ILA2,ILB2,dA1,dB1,dA2,dB2,rv1,rv2,beta,
    ↪ stress,IM1r,IM2r,T1,T2,T3,kscale,pset,pver
S,ms,state,fault,mode,fw,pver        F,ms,code,channel,value,raw        A,<cmd>,OK|ERR
    ↪ ,<why>        P,name,val[,min,max,class]
```

States: `BOOT`, `SELFTEST`, `IDLE`, `SOFTSTART`, `RUN_OL`, `RUN_CL`, `RUN_OL1`, `RUN_CL1`, `RUN_OL2`, `RUN_CL2`, `STOP`, `FAULT`. Outside RUN_CL the fields `rv`, `beta`, `stress`, `IM1r`, `IM2r` repeat their last value (the "stale values" seen on the Grafana screenshots); the analysis keeps RUN rows only.

Table 12: Fault bitmask (`app.h`); OV/UV/string-OV need 3 consecutive 10 ms samples; V_{M1} is excluded unless `vm1_prot` = 1.

Bit	Meaning	Bit	Meaning
0x001	OC latch / ADC rail / DMA stall	0x080	over-temperature (INA die $>$ 85 °C)
0x002	hardware OC (COMP, not implemented)	0x100	watchdog reset
0x004	E-stop	0x200	software E-stop (`C,ESTOP`)
0x008	module OV $>$ 30 V	0x400	string OV $>$ 85 V
0x010	module UV $<$ 3 V (closed loop only)	0x800	comms watchdog (CL, 5 s without a command)
0x020	zero-calibration	0x1000	$v_{str,pi}$ stale
0x040	INA bus / VM unreadable	0x2000	main loop stalled $>$ 20 ms while switching

Commands (Pi → Nucleo): `C,MODE,<IDLE|OL|CL|OL1|CL1|OL2|CL2>`; `C,TEST,0..3`; `C,EN,0|1`; `C,DUTY,leg,d`; `C,DUTYS,dA1,dB1,dA2,dB2`; `C,VREF,rv1,rv2`; `C,SUP,0|1`; `C,SET,name,value`; `C,SETM,m00..m33`; `C,GET,name`; `C,LIST`; `C,APPLY`; `C,REVERT`; `C,CAL,ch,mV/A,zero`; `C,ESTOP`; `C,CLEAR`; `C,PING`. Parameter classes: STAGED (take effect on `C,APPLY`: `kscale`, `kp_v`, `kp_i`, `fc_lpf`, `filt_ma`, `dmin`, `dmax`, `m00..m33`); LIVE (`sup_*`, `vref1/2`, `hall_mva*`, `oc_a`, `vm_ov`, `vm_uv`, `vstr_ov`, `vstr_param`, `vstr_pi`, `vm1_prot`, `slew`, `wdg_s`); IDLE-only (`hall_zero*`, `vm1_src`). `pset` increments on every swap; a swap rebases the integrators ($\xi = M^{-1}u_{act}$) so the duties do not jump.

E.6 Firmware images

Table 13: Firmware variants used on 16 and 20 Oct (`7_Pi/firmware/VERSION.txt`).

Image	f_{sw} [kHz]	PER	periods / tick	duty limits	gain set (`pver`)
`shuffler.bin`	200	27200	20	0.05–0.95	`20261015-1950-3mod-qspice200-bench`
`shuffler_300k.bin`	300	18133	30	0.08–0.95	`20261016-1628-3mod-qspice300-bench`
`shuffler_400k.bin`	400	13600	40	0.12–0.88	`20261016-1751-3mod-qspice400-bench`

E.7 Raspberry Pi stack

Docker: `influxdb:2.7` on :8086 (org `boost`, bucket `shuffler`, infinite retention) and `grafana/grafana-oss:11.2.0` on :3000 (provisioned Flux datasource, dashboard "Shuffler pod": mode, active, init check, temperatures, state, module voltages, inductor currents, duties, supervisor, reconstructed module currents, string voltage INA-sum vs e-load, e-load status/V/I, string power, command log, run-time parameters). Services (systemd): `pod.service` (`pod.py`, 921600 baud), `eload.service` (Rigol DL3021 over USB-TMC, 5 Hz measurements, 2 Hz V_{str} forward, limits CC 15 A / CV 150 V / CP 200 W), `params_web.service` (stdlib HTTP on :8080, no authentication, allow-listed commands). IR: Elegoo NEC remote via `gpio-ir` on GPIO17 (POWER = E-stop, FUNC = disable, digits = presets, PLAY = run, VOL± = kscale ±0.1, EQ = status pages). Presets (`actions.py`): 0 init check, 1 open loop, 2 closed loop, 3/4 load on/off, 5/6 load remote/local, 7/8 kscale 0.3/1.0, 9 clear fault. Flashing: OpenOCD (`interface/stlink.cfg`, `target/stm32g4x.cfg`) with the pod stopped around the flash; `stage_firmware.sh` builds Release and copies the `.bin`; `sync.sh` rsyncs `7_Pi` to the Pi; `setup.sh` installs packages, udev rules, overlays (I^2C LCD 0x27/0x3F, 1-wire DS18B20 GPIO4, IR GPIO17), the venv and the services.

F Gain Sets

Every gain set is a JSON file in `6_RTOS/ctrl_core/params/` (`version`, `n_mod`, `ctrl`, `sup`, `mppt`); `gen_params.py` renders it to `params.h` (M, M_p, M^{-1} as float32 literals). All 3-module sets use $T_s = 100\,\mu$s, $f_c = 2$kHz, $d_{min/max} = 0.08/0.92$, $k_{p,v} = 1.58 \times 10^{-4}$, $k_{p,i} = 1.08 \times 10^{-4}$. Rows are $[u_{c,1}, u_{c,2}, u_{d,1}, u_{d,2}]$, columns $[e_{vd1}, e_{vd2}, e_{ic1}, e_{ic2}]$.

Table 14: Base gain matrices M of the deployed and candidate gain sets.

Set (`version`, sps-params-2…)	row	e_{vd1}	e_{vd2}	e_{ic1}	e_{ic2}
`20261015-1754-3mod-simulink-bench` (rejected)	$u_{c,1}$	−20.477	0.010	−0.046	0.092
	$u_{c,2}$	−0.008	−19.903	−0.089	0.045
	$u_{d,1}$	0	0	3.390	0
	$u_{d,2}$	0	0	0	3.385
`20261015-1950-3mod-qspice200-bench` (200 kHz, deployed)	$u_{c,1}$	−20.284	0.122	−9.384	0.105
	$u_{c,2}$	0.207	−20.010	0.055	−10.521
	$u_{d,1}$	−0.146	−0.044	17.569	−0.202
	$u_{d,2}$	−0.195	−0.131	−0.102	18.019
`20261016-1628-3mod-qspice300-bench` (300 kHz, deployed)	$u_{c,1}$	−20.739	0.400	−15.980	0.196
	$u_{c,2}$	0.127	−20.602	0.581	−18.513
	$u_{d,1}$	−0.125	−0.167	42.703	−1.068
	$u_{d,2}$	−0.052	−0.066	−1.532	44.231
`20261016-1751-3mod-qspice400-bench` (400 kHz, deployed)	$u_{c,1}$	−22.683	0.684	−36.942	−0.076
	$u_{c,2}$	−0.306	−22.038	−0.106	−40.278
	$u_{d,1}$	0.152	0.197	107.293	−3.903
	$u_{d,2}$	0.246	0.191	−3.838	109.866

The voltage rows of the Simulink and QSPICE sets agree ($u_c/e_{vd} \approx -20$, which is $k_{I,v}/\bar{V}$). The current rows differ fivefold (3.39 against 17.6) because the switching plant's current-loop gain is $\bar{V}/R_{sw}$, not $\bar{V}/R_{phase}$. The QSPICE sets also carry u_c/e_{ic} cross terms of -9 to -40 that cancel the dead-time coupling v_d/u_d.

The telemetry field `pver` records which set was running. On 16 Oct the bench ran `qspice200` from 12:59 to 16:29, `qspice300` from 16:29 (with brief returns to 200 kHz at 16:56–17:03 and 17:13–17:14) and `qspice400` from 17:52. On 20 Oct the solar test ran `qspice200` throughout, with k from 0.1 to 0.8. The identification script refuses to write a set whose $u_d \to i_c$ diagonal is not positive and warns if $\mathrm{cond}(G) > 100$; the previous JSON is archived under its version name.

G Simulink Model and Validation Suite

The validation suite (`shuffler_string_validate.m`) runs seven tests on the MIL model (4 modules, 3 shufflers, $k = 1$, cold start) and the first five on the carpark twin (3 modules, 2 shufflers, sensing model). Pass criteria: settled ($|\Delta v_d| < 0.05$ V, $|\Delta i_c| < 0.5$ A between two 10 ms windows), track ($|v_d - r_v| < 0.25$ V, $|i_c| < 1$ A), MPPT (module power above 98.5 % of the per-module ceiling) and protect ($|i_c| < 1$ A, $|i_{diff}| \le 10$ A). Figure 27 shows one tracking test and the protection test as examples; every test's figures are regenerated by the script and are in `8_Results/analysis/out/figures/`.

Table 15: Validate test matrix.

#	Test	irradiance on the shaded module	r_v	T_{stop}	mode	result (MIL / carpark)
01	uniform-zero	1000 W/m^2	0	0.06 s	track	PASS / PASS
02	decoupling	1000	[0 2 0] (carpark [0 2])	0.06	track	PASS / PASS
03	shade-balanced	500	0	0.06	track	PASS / PASS
04	shade-refs	500	[0 2 −2] (carpark [2 −2])	0.06	track	PASS / PASS
05	shade-deep	200	0	0.50	protect	PASS / PASS
06	mppt-uniform	1000	0	0.60	mppt	PASS / (CC load only)
07	mppt-shade	500	0	0.60	mppt	PASS / (CC load only)

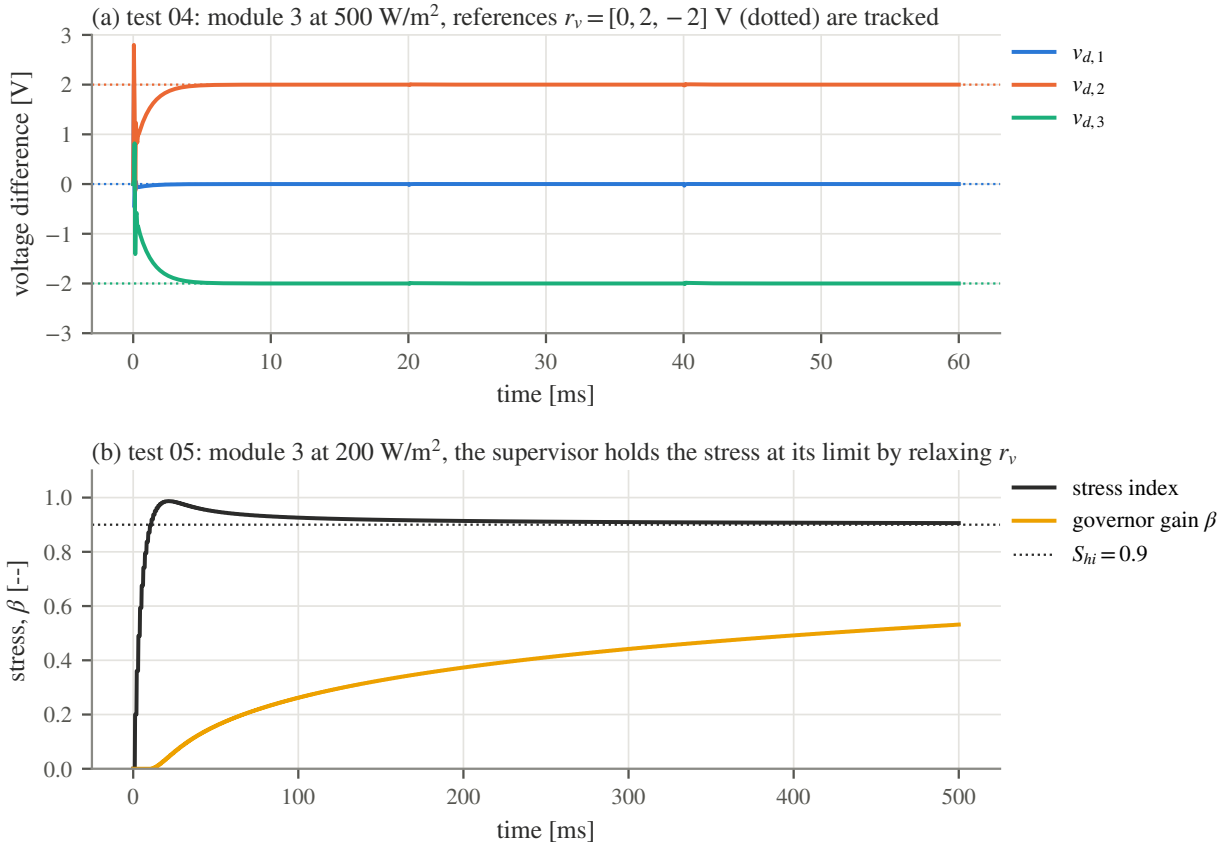


Figure 27: Two of the MIL validation tests: (a) tracking non-zero references under shade, (b) deep shade, where the feasibility governor raises β to relax the references and keep the shuffler inside its current rating.

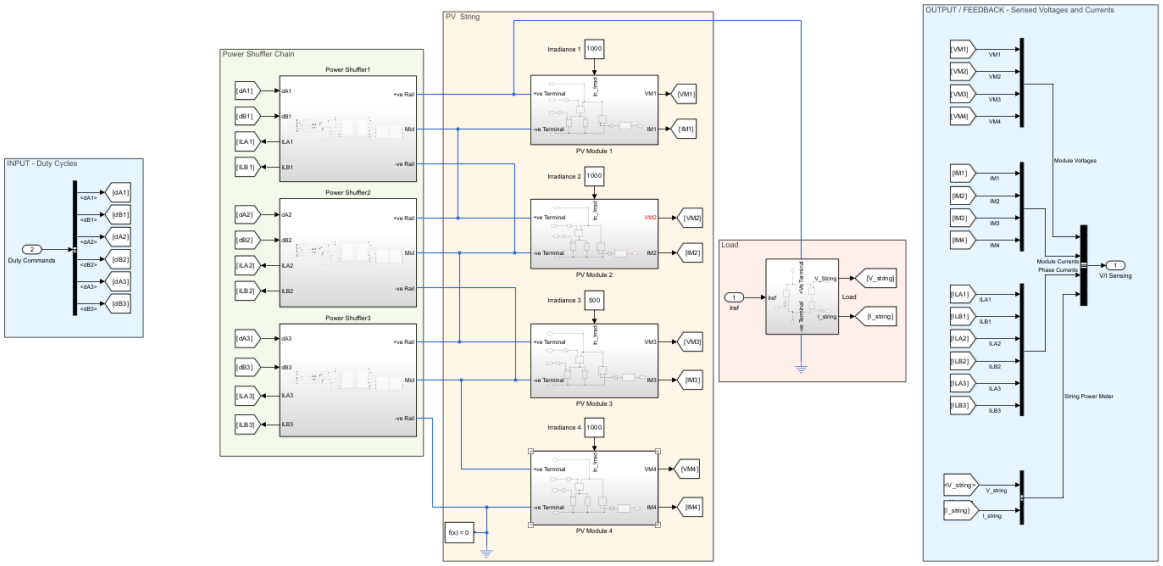
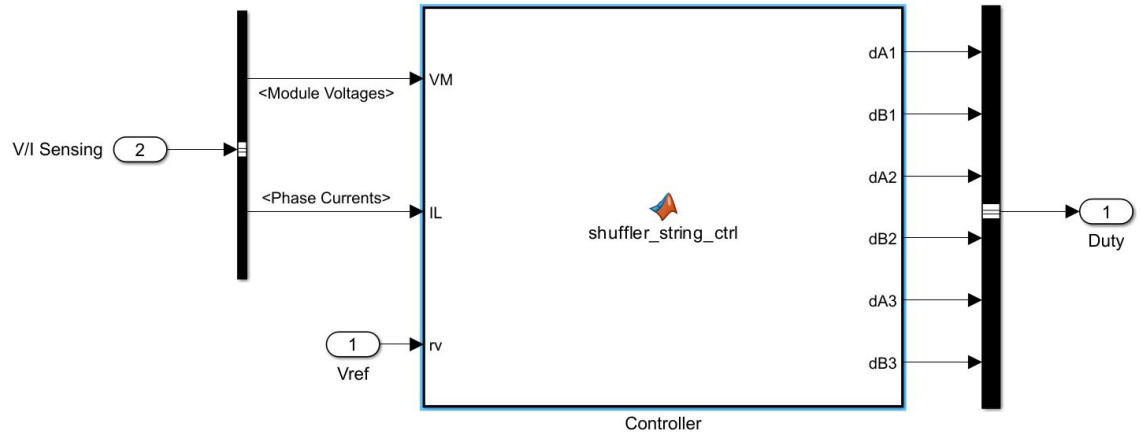
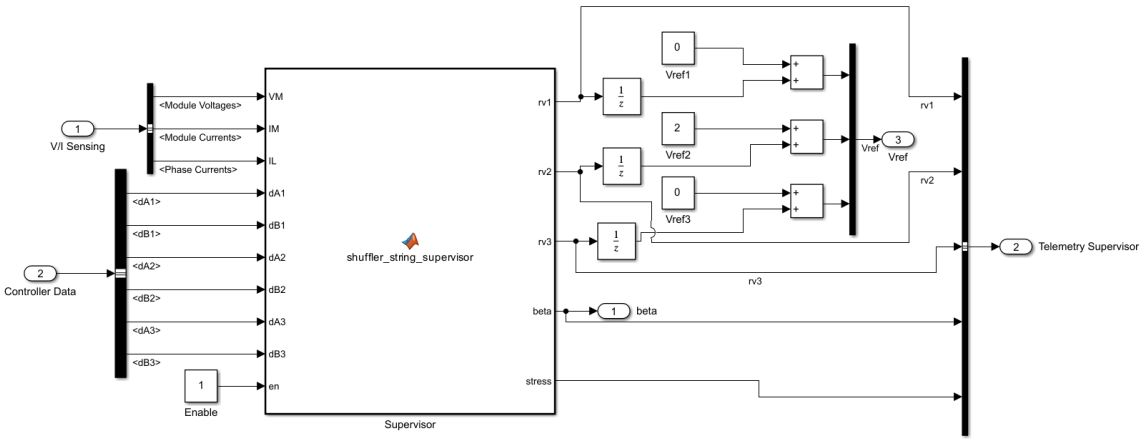


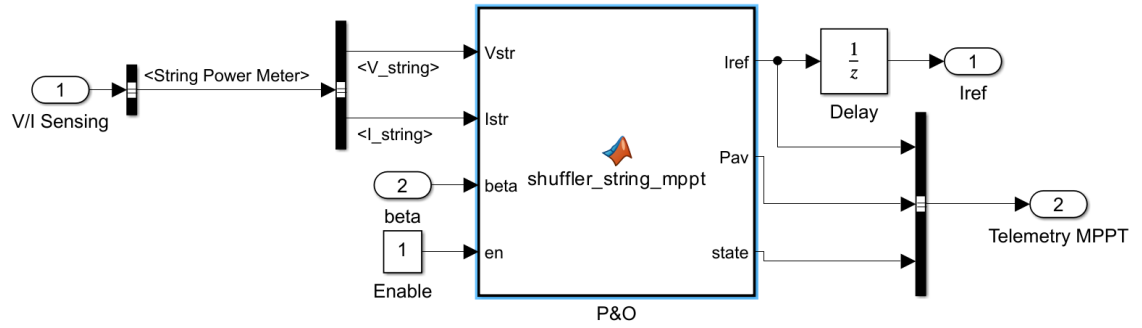
Figure 28: Inside the MIL plant: four AIKO-class modules and three averaged-switch shufflers.



(a) Controller block.



(b) Supervisor block.



(c) MPPT (P&O) block.

Figure 29: The controller, supervisor and MPPT blocks of the Simulink MIL model (fig. 6).

H Simulation Model Sheets

The scenarios run on every model are S1 (loop closure on the open-loop residual), S3 (module 3 loses a third of its current), S4 (+1 V step on the $v_{d,2}$ reference) and S5 (the open-loop duty steps as sent on the bench), each at $k = 0.1$, 0.2, 0.3, 0.5 and 1 on the bench and solar plants: 28 analytical, 25 QSPICE and 25 carpark-twin runs. Their metrics are in appendix K; figs. 13 and 14 show representative runs, and all figures are regenerated by `make all` in `8_Results/analysis`.

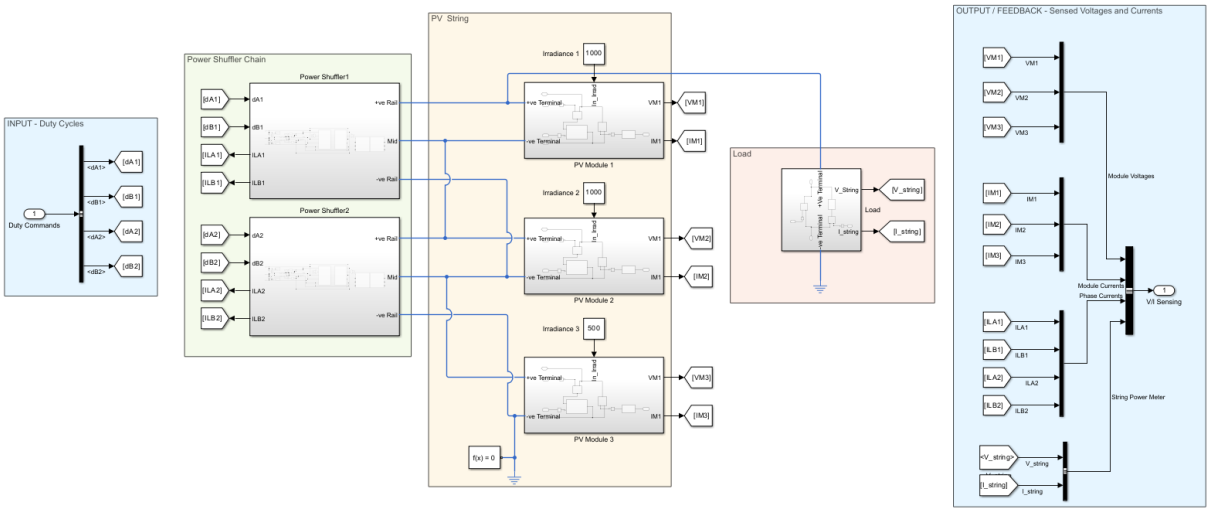


Figure 30: Simulink carpark twin: three modules, two averaged-switch shufflers, the sensing block (noise and delay) and the DL3021 CV load. The power-source and load variants are in fig. 32.

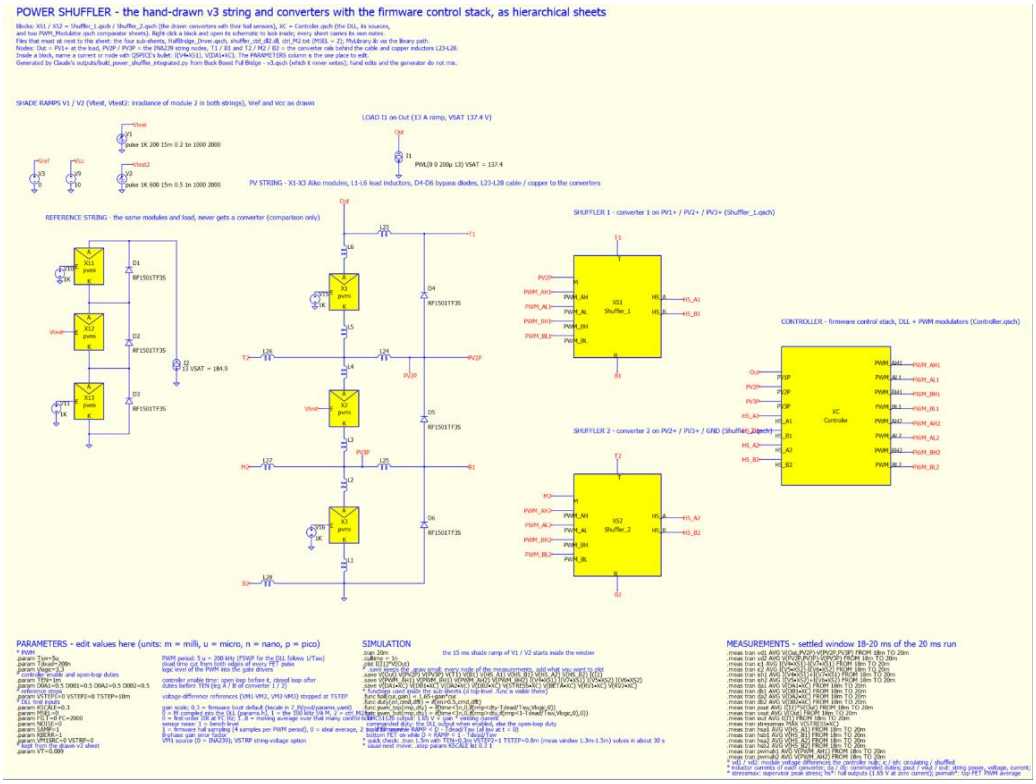
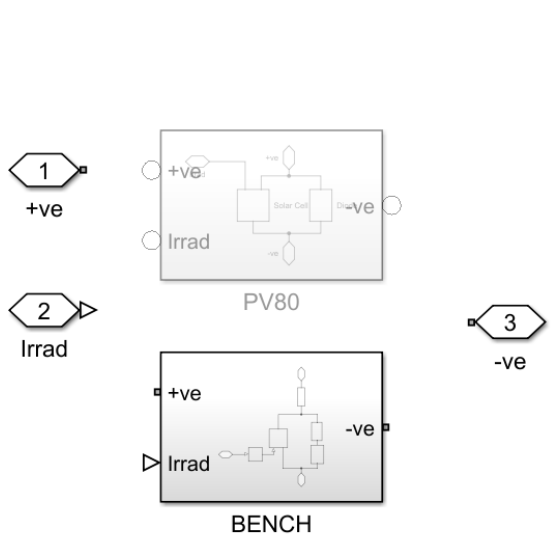
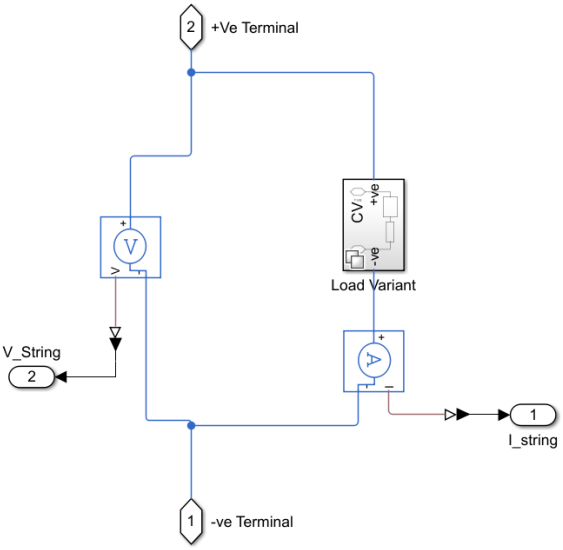


Figure 31: QSPICE integrated model, top sheet: PV string, two shufflers with real switches, drivers and dead time, and the controller. The controller sheet with the firmware DLL is in fig. 33.



(a) Power-source variants (PV80 / BENCH).



(b) Load variants (CV / CC).

Figure 32: Simulink carpark twin (fig. 30): power-source and load variants.

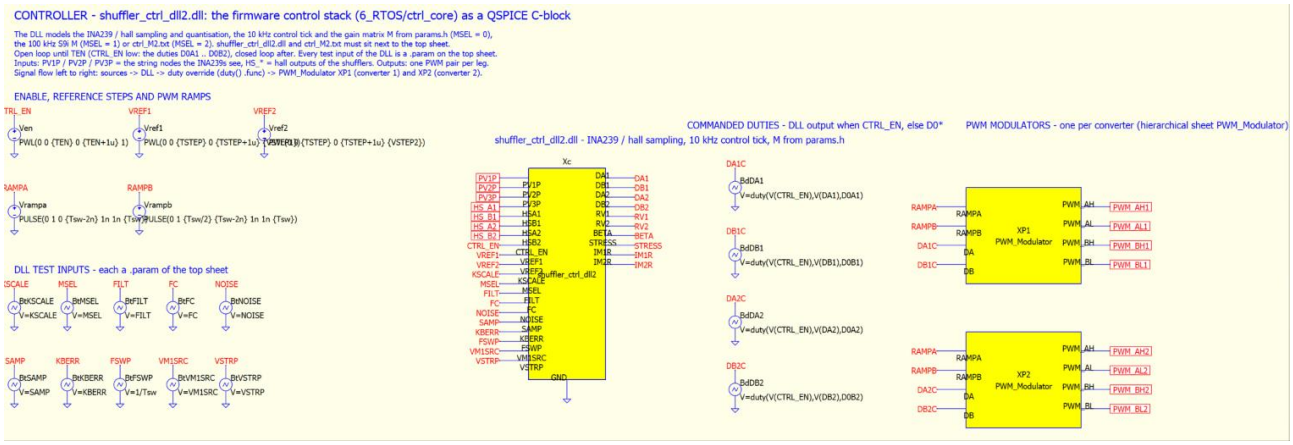


Figure 33: QSPICE integrated model (fig. 31), controller sheet: the firmware control DLL ("the Nucleo") and the HRTIM-style PWM modulators.

I Bench Test Records (16 Oct 2026)

Ten bench windows were exported from InfluxDB (table 22); only RUN_OL and RUN_CL rows are analysed. The tables below are the evidence behind the open-loop, loop-closure and fault statements of section 4.5; the per-window overview and event figures are regenerated by the results pipeline (`8_Results/analysis`, `make all`).

Table 16: Open-loop duty steps on the bench (B01s, 16 Oct, 200 kHz): both legs of a shuffler were moved together, so $\Delta u_c = 2\Delta d$, and the averaged model predicts $\Delta v_d = -\bar{V}\Delta u_c$ with $\bar{V} = 46.6$ V. Voltages in V.

Time	Leg duties changed	Δu_{c1}	Δu_{c2}	Δv_{d1} pred.	Δv_{d1} meas.	Δv_{d2} pred.	Δv_{d2} meas.
15:11:42	S2 0.49→0.51	0.00	+0.04	0.00	-0.03	-1.86	-1.87
15:13:00	S1 0.50→0.49, S2 0.51→0.50	-0.02	-0.02	+0.93	+0.91	+0.93	+0.94
15:15:15	S1 0.49→0.51	+0.04	0.00	-1.86	-1.86	0.00	0.00

Table 17: Open-loop steady state on hardware (duties 0.5): the residual $v_d = -R_m i_m$ and the circulating current the loop has to remove.

Case	f_{sw}	T [s]	$\bar{v}_{d1}$	$\bar{v}_{d2}$	$\bar{i}_{c1}$	$\bar{i}_{c2}$	$\bar{i}_{d1}$	$\bar{i}_{d2}$	V_{M1}	V_{M2}	V_{M3}	P_{load} [W]
B01	200	321	0.01	-0.17	0.66	3.54	0.47	-2.92	23.3	23.3	23.4	67.9
B01	200	380	0.00	-0.17	0.68	3.55	0.42	-2.92	23.3	23.3	23.5	66.9
B01s	200	409	0.24	0.05	0.46	2.62	0.77	-2.03	23.5	23.3	23.2	65.6
B05	300	280	0.08	-0.58	0.51	0.05	0.55	-0.57	23.2	23.1	23.7	69.1
B05	300	18	0.08	-0.58	0.51	0.05	0.54	-0.58	23.2	23.1	23.7	69.1
B08	400	149	0.23	-0.94	0.34	0.12	0.32	-0.19	23.2	22.9	23.9	62.9
S01	200	184	-0.25	-0.32	-0.75	0.96	-2.20	-0.88	20.1	20.3	20.6	67.7
S03	200	171	-0.04	-0.07	0.08	1.00	-0.38	-0.08	20.3	20.3	20.4	9.5

Table 18: Gain changes applied while the loop was running (`C,APPLY` during RUN_CL) and the 300 kHz stability limit (from T1/T3 and the pod command log).

Case	Time	Action	Outcome
B04	16:07:50	kscale 0.3 → 0.4 applied in RUN_CL	fault 0x10 (module UV) within 1 s
B07	17:37:56	kscale 0.5 → 0.6 applied in RUN_CL	fault 0x2000 (main-loop stall) within 2 s
B07	17:38:34	kscale 0.6 run (6 s)	oscillation: $\sigma(v_{d2})$ 1.41 V, pk-pk 11.8 V, 3.2 Hz aliased
B07	17:39:41	kscale 0.6 → 0.3 applied in RUN_CL	fault 0x2000 (main-loop stall) within 2 s
B08	18:03:40	PSU change + gain set 400 kHz	OC fault 0x01, restarted

Table 19: Loop closures on hardware (IDLE → SOFTSTART → RUN_CL): the residual left by the 150 ms soft-start ramp and how the loop removes it. $t_{s,abs}$ is the time to stay inside ± 0.1 V / ± 0.2 A.

Case	Time	k	Signal	IDLE level	y at RUN_CL	y_∞	SSE	Rise	$t_{s,abs}$ [s]	Noise
B02	15:31:37	0.1	vd2	-4.97	-0.01	0.000	0.000	-	0.00	0.030
B02	15:31:37	0.1	ic2	0.01	1.58	-0.399	-0.399	$\leq$ 89 ms	2.84	0.980
B02	15:34:19	0.1	vd2	-4.95	0.06	0.000	0.000	-	0.00	0.015
B02	15:34:19	0.1	ic2	0.03	-0.98	-0.458	-0.458	$\leq$ 88 ms	2.91	1.629
B03	16:02:08	0.3	vd2	-4.95	-0.09	0.015	0.015	$\leq$ 71 ms	1.90	0.044
B03	16:02:08	0.3	ic2	0.01	0.10	0.158	0.158	-	2.99	0.528
B04	16:07:03	0.3	vd2	-4.95	-0.07	0.030	0.030	-	1.48	0.044
B04	16:07:03	0.3	ic2	-0.00	-0.07	-0.116	-0.116	-	2.99	0.586
B06	17:17:01	0.3	vd2	-4.98	-0.21	0.000	0.000	$\leq$ 23 ms	0.00	0.000
B06	17:17:01	0.3	ic2	-0.02	-0.04	-0.012	-0.012	-	0.00	0.056
B06	17:23:14	0.3	vd2	-4.98	-0.13	0.000	0.000	$\leq$ 23 ms	0.00	0.015
B06	17:23:14	0.3	ic2	-0.01	-0.01	-0.015	-0.015	-	0.00	0.066
B07	17:37:20	0.5	vd2	-4.96	-0.19	0.010	0.010	$\leq$ 23 ms	2.69	0.074
B07	17:37:20	0.5	ic2	-0.02	-0.04	0.007	0.007	-	0.00	0.063
B07	17:38:34	0.6	vd2	-4.96	-0.20	0.035	0.035	$\leq$ 23 ms	2.51	0.037
B07	17:38:34	0.6	ic2	-0.01	-0.04	-0.002	-0.002	-	1.26	0.044
B07	17:39:20	0.4	vd2	-4.94	-0.19	-0.010	-0.010	$\leq$ 23 ms	0.00	0.059
B07	17:39:20	0.4	ic2	-0.02	-0.05	-0.019	-0.019	-	0.00	0.062
B07	17:40:07	0.3	vd2	-4.95	-0.20	0.000	0.000	$\leq$ 23 ms	0.00	0.015
B07	17:40:07	0.3	ic2	-0.01	-0.02	0.008	0.008	-	0.00	0.061
B09	18:05:20	0.3	vd2	-4.96	-0.46	0.000	0.000	$\leq$ 23 ms	0.00	0.015
B09	18:05:20	0.3	ic2	0.02	0.06	0.082	0.082	-	0.73	0.091
B09	18:07:37	0.3	vd2	-4.97	-0.80	0.000	0.000	$\leq$ 23 ms	0.00	0.015
B09	18:07:37	0.3	ic2	0.02	0.14	0.049	0.049	-	1.85	0.068
B09	18:08:24	0.3	vd2	-4.96	-0.17	0.010	0.010	$\leq$ 23 ms	0.00	0.015
B09	18:08:24	0.3	ic2	0.01	0.10	0.048	0.048	-	2.49	0.068
B09	18:12:32	0.3	vd2	-4.96	-0.80	-0.010	-0.010	$\leq$ 23 ms	0.18	0.015
B09	18:12:32	0.3	ic2	0.01	0.08	0.050	0.050	-	1.48	0.087
S02	15:08:45	0.1	vd2	0.00	-0.17	0.000	0.000	$\leq$ 43 ms	0.00	0.015
S02	15:08:45	0.1	ic2	-0.03	0.29	-0.049	-0.049	$\leq$ 40 ms	2.97	0.305
S04	15:52:05	0.1	vd2	-0.03	0.13	0.000	0.000	$\leq$ 23 ms	3.00	0.119
S04	15:52:05	0.1	ic2	-0.05	0.30	0.070	0.070	-	3.00	0.326
S05	15:56:54	0.2	vd2	-0.01	-0.26	-0.030	-0.030	$\leq$ 37 ms	2.99	0.119
S05	15:56:54	0.2	ic2	-0.04	0.34	-0.023	-0.023	$\leq$ 23 ms	2.97	0.245
S06	16:01:08	0.3	vd2	-0.02	-0.17	-0.015	-0.015	$\leq$ 23 ms	2.91	0.037
S06	16:01:08	0.3	ic2	-0.04	-0.03	-0.154	-0.154	-	2.99	0.302
S07	16:05:37	0.4	vd2	-0.03	-0.03	-0.030	-0.030	-	2.91	0.052
S07	16:05:37	0.4	ic2	-0.03	0.53	-0.226	-0.226	$\leq$ 23 ms	2.97	0.270
S08	16:09:44	0.5	vd2	-0.03	-0.13	0.010	0.010	$\leq$ 23 ms	2.66	0.059
S08	16:09:44	0.5	ic2	-0.04	0.06	-0.027	-0.027	-	2.97	0.412
S09	16:13:19	0.6	vd2	-0.03	-0.15	-0.010	-0.010	$\leq$ 23 ms	2.86	0.089

Case	Time	k	Signal	IDLE level	y at RUN_CL	y_∞	SSE	Rise	$t_{s,abs}$ [s]	Noise
S09	16:13:19	0.6	ic2	-0.04	-0.01	-0.182	-0.182	-	2.99	0.386
S10	16:19:11	0.7	vd2	-0.03	0.07	-0.015	-0.015	-	2.77	0.104
S10	16:19:11	0.7	ic2	-0.05	-0.03	-0.006	-0.006	-	2.97	0.178
S11	16:23:11	0.8	vd2	-0.06	0.00	-0.010	-0.010	-	2.80	0.052
S11	16:23:11	0.8	ic2	-0.04	0.22	-0.099	-0.099	37.2 ms	3.00	0.279

J Solar Test Records (20 Oct 2026)

Carpark string of three KAUSSP80 modules into the DL3021 at 61 V CV, 200 kHz. Peak sun 15:04–15:13 (S01 open loop, S02 closed loop $k = 0.1$), minimal sun from 15:48 (S03 open loop, S04–S11 closed loop, $k = 0.1$ to 0.8). Module currents are reconstructed by KCL from the winding currents, duties and the e-load current.

Table 20: Solar test (20 Oct 2026, KAUSSP80 string into a 61 V CV load): powers in W, module currents in A (KCL reconstruction), energy delivered and energy returned above a weakest-module-limited string.

Case	Loop	k	T [s]	$\bar{P}_{load}$	$\bar{P}_{shuf}$	$\bar{P}_{ret}$	$P_{ret,max}$	I_{M1}	I_{M2}	I_{M3}	E_{load} [Wh]	E_{ret} [Wh]	η [%]
S01	OL	–	188	67.3	62.3	65.9	96.1	0.00	1.76	1.54	3.51	3.44	99.3
S02	CL	0.1	238	45.1	42.3	43.9	98.9	-0.00	1.21	1.03	2.98	2.90	99.2
S03	OL	–	175	9.4	9.3	9.3	11.1	-0.04	0.31	0.19	0.46	0.45	99.9
S04	CL	0.1	266	8.2	8.2	8.0	9.0	-0.04	0.28	0.16	0.60	0.59	99.9
S05	CL	0.2	230	10.9	10.6	10.7	12.4	-0.04	0.35	0.22	0.69	0.68	99.9
S06	CL	0.3	189	13.4	12.5	13.1	14.2	-0.03	0.42	0.27	0.70	0.69	99.9
S07	CL	0.4	156	13.5	13.0	13.3	14.2	-0.04	0.42	0.28	0.59	0.58	99.9
S08	CL	0.5	151	10.6	10.6	10.4	11.8	-0.04	0.34	0.22	0.44	0.44	99.9
S09	CL	0.6	279	7.3	7.3	7.2	8.5	-0.04	0.27	0.13	0.57	0.55	100.0
S10	CL	0.7	179	5.1	5.8	5.0	6.1	-0.04	0.21	0.09	0.26	0.25	100.1
S11	CL	0.8	91	2.3	4.4	2.3	3.2	-0.06	0.14	0.03	0.06	0.06	100.3

Module 1 carried ≈ 0 A in every window, so a plain series string would have delivered nothing to a 61 V load: $E_{ret} \approx E_{load}$.

Table 21: Detected shade / irradiance events in the 20 Oct solar windows (current-mismatch and string-power level shifts; irradiance was not logged).

Case	Event	Time	Signal	y_0	y_∞	Δ	Peak dev.	$t_{s,2\%}$ [s]
S01	mismatch_off	15:07:31	id2	-0.62	-0.20	0.42	–	0.66
S01	mismatch_off	15:07:31	vd2	-0.25	-0.11	0.14	–	2.93
S01	mismatch_off	15:07:31	Pstr_i	53.35	18.40	-34.95	–	4.89
S01	mismatch_off	15:07:31	IM3	1.18	0.41	-0.77	–	4.14
S01	irradiance_down	15:07:32	Pstr_i	51.23	18.39	-32.83	–	1.84
S01	irradiance_down	15:07:32	id2	-0.61	-0.21	0.39	–	0.00
S01	irradiance_down	15:07:32	vd2	-0.24	-0.11	0.13	–	1.93
S02	mismatch_on	15:09:24	id2	-0.57	-1.44	-0.87	–	4.56
S02	mismatch_on	15:09:24	vd2	0.00	0.00	0.00	0.06	0.00
S02	mismatch_on	15:09:24	Pstr_i	42.20	98.24	56.04	–	5.92
S02	mismatch_on	15:09:24	IM3	0.97	2.33	1.35	–	7.51
S02	irradiance_up	15:09:24	Pstr_i	42.20	98.24	56.04	–	5.92
S02	irradiance_up	15:09:24	id2	-0.57	-1.44	-0.87	–	4.56
S02	irradiance_up	15:09:24	vd2	0.00	0.00	0.00	0.06	0.00
S02	irradiance_down	15:10:29	Pstr_i	86.29	53.38	-32.91	–	0.00
S02	irradiance_down	15:10:29	id2	-1.20	-0.71	0.50	–	0.00
S02	irradiance_down	15:10:29	vd2	0.00	0.00	0.00	-0.05	0.00
S02	mismatch_off	15:10:31	id2	-1.04	-0.60	0.44	–	11.40
S02	mismatch_off	15:10:31	vd2	0.00	0.00	0.00	0.05	0.00
S02	mismatch_off	15:10:31	Pstr_i	75.64	47.10	-28.54	–	0.00
S02	mismatch_off	15:10:31	IM3	1.78	1.05	-0.73	–	0.00
S02	mismatch_on	15:10:40	id2	-0.55	-0.33	0.22	-1.02	7.31
S02	mismatch_on	15:10:40	vd2	0.00	0.00	0.00	0.05	0.00
S02	mismatch_on	15:10:40	Pstr_i	42.57	30.44	-12.13	–	11.70
S02	mismatch_on	15:10:40	IM3	0.98	0.67	-0.31	–	9.95
S02	mismatch_off	15:10:48	id2	-0.79	-0.38	0.42	–	0.00
S02	mismatch_off	15:10:48	vd2	0.00	0.00	0.00	0.05	0.00
S02	mismatch_off	15:10:48	Pstr_i	56.71	32.50	-24.21	–	2.69

Case	Event	Time	Signal	y_0	y_∞	Δ	Peak dev.	$t_{s,2\%}$ [s]
S02	mismatch_off	15:10:48	IM3	1.30	0.71	-0.59	–	1.93
S03	mismatch_on	15:48:53	id2	-0.10	-0.10	-0.00	-0.61	0.00
S03	mismatch_on	15:48:53	vd2	-0.07	-0.07	0.00	-0.26	18.03
S03	mismatch_on	15:48:53	Pstr_i	10.43	10.00	-0.43	0.62	0.00
S03	mismatch_on	15:48:53	IM3	0.22	0.22	-0.01	0.30	0.00
S03	mismatch_off	15:48:59	id2	-0.11	-0.12	-0.01	0.63	0.00
S03	mismatch_off	15:48:59	vd2	-0.07	-0.07	0.00	-0.26	0.75
S03	mismatch_off	15:48:59	Pstr_i	10.35	9.88	-0.47	0.55	0.00
S03	mismatch_off	15:48:59	IM3	0.22	0.22	-0.00	-0.32	0.00
S03	mismatch_off	15:49:18	id2	-0.11	-0.11	0.00	0.62	0.00
S03	mismatch_off	15:49:18	vd2	-0.07	-0.07	0.00	0.24	8.93
S03	mismatch_off	15:49:18	Pstr_i	9.90	9.60	-0.30	0.36	0.00
S03	mismatch_off	15:49:18	IM3	0.22	0.21	-0.01	-0.31	0.00
S05	mismatch_on	15:58:21	id2	-0.10	-0.06	0.04	0.71	19.36
S05	mismatch_on	15:58:21	vd2	0.00	-0.00	-0.00	-0.25	14.98
S05	mismatch_on	15:58:21	Pstr_i	10.45	10.86	0.42	-0.72	0.00
S05	mismatch_on	15:58:21	IM3	0.22	0.21	-0.01	-0.36	19.36
S08	mismatch_on	16:10:42	id2	-0.08	-0.07	0.01	-0.64	0.00
S08	mismatch_on	16:10:42	vd2	0.01	-0.01	-0.02	0.22	17.23
S08	mismatch_on	16:10:42	Pstr_i	11.12	10.73	-0.39	0.57	0.00
S08	mismatch_on	16:10:42	IM3	0.22	0.21	-0.01	0.32	0.00

K Synthesis: Inventory and Metrics

All tables are generated from the pipeline CSVs (`8_Results/analysis/out/tables`) by `tools/make_tables.py`; `make all` in `8_Results/analysis` rebuilds the CSVs from the exported test windows (the full set, including every event and every signal, is in `REPORT.md` and the CSVs). Only the tables the report cites are reproduced here.

K.1 Metric definitions

Every metric has one definition for every phase (`stepmetrics.py`): initial and final levels (medians), step size, 10–90 % rise time, peak value and time, overshoot and undershoot, 2 %, 5 % and absolute-band settling times, steady-state error, IAE, and a fitted τ or (ζ, ω_n). Noise is the median absolute deviation of the pre-event window; steady state is reported as mean, standard deviation, peak-to-peak, RMS and dominant frequency (Welch); and energy as E_{load}, $E_{shuffled}$ and $E_{returned} = \int (I_{str} - \max(\min_k I_k, 0))\, V_{str}\, \mathrm{d}t$.

K.2 Test-case inventory and data rules

Table 22: Test-case inventory: the raw-data slide windows re-exported from the pod's InfluxDB (RUN_OL / RUN_CL rows only).

Case	Slide	Setup	f_{sw} [kHz]	Loop	k	Window (AEST)	RUN [s]	Rate [Hz]	Note
B01	p48	BENCH	200	OL	–	14:32:00-14:44:56	706	40	duties 0.5; e-load CV set-point steps 125 <→ 60 V; VM1 from
B01s		BENCH	200	OL	–	15:09:00-15:16:00	412	40	not on a slide: the only open-loop duty steps (15:11:42 S2 +
B02	p42	BENCH	200	CL	0.1	15:31:34-15:34:25	161	11	before the ADC fix; PSU unstable
B03	p43	BENCH	200	CL	0.3	16:02:05-16:04:33	143	14	slide says kscale 20 %, telemetry says 0.3 with pset 0 (boot
B04	p44	BENCH	200	CL	0.3/0.4	16:07:00-16:07:49	47	14	kscale 0.4 applied at 16:07:50 → OC fault within 1 s
B05	p49	BENCH	300	OL	–	16:31:36-16:37:04	304	36	nan
B06	p45	BENCH	300	CL	0.3	17:17:00-17:25:49	512	44	PSU / e-load set-point steps 17:22:30-17:25:30 while closed
B07	p46	BENCH	300	CL	0.3/0.4/0.5/0.6	17:34:08-17:41:47	295	44	nan
B08	p50	BENCH	400	OL	–	18:02:03-18:04:38	152	44	PSU change at 18:03:40
B09	p47	BENCH	400	CL	0.3	18:05:11-18:13:40	392	44	nan
S01	p24	PV80	200	OL	–	15:04:35-15:07:47	187	44	peak sun

Case	Slide	Setup	f_{sw} [kHz]	Loop	k	Window (AEST)	RUN [s]	Rate [Hz]	Note
S02	p25	PV80	200	CL	0.1	15:08:40-15:12:44	237	44	peak sun; shade event about 15:09:20-15:10:40
S03	p26	PV80	200	OL	–	15:48:22-15:51:20	174	44	minimal sun
S04	p27	PV80	200	CL	0.1	15:52:03-15:56:31	265	44	minimal sun
S05	p37	PV80	200	CL	0.2	15:56:51-16:00:47	229	44	minimal sun
S06	p38	PV80	200	CL	0.3	16:01:07-16:04:19	189	44	minimal sun
S07	p39	PV80	200	CL	0.4	16:05:30-16:08:17	156	44	minimal sun
S08	p40	PV80	200	CL	0.5	16:09:42-16:12:16	150	44	minimal sun
S09		PV80	200	CL	0.6	16:13:15-16:18:00	278	44	not on a slide (closed_loop_timeslots.pdf)
S10		PV80	200	CL	0.7	16:19:05-16:22:12	178	44	not on a slide (closed_loop_timeslots.pdf)
S11		PV80	200	CL	0.8	16:23:05-16:24:45	90	44	not on a slide (closed_loop_timeslots.pdf)

B01s and S09–S11 are not on a slide. B03 ran at $k = 0.3$ (telemetry), not the 20 % written on the slide.

- Only RUN_OL / RUN_CL frames are analysed; frames repeating all ten control fields are dropped; leading/trailing frames where V_{M2} or V_{M3} left the run median by > 2 V (supply switched off before the FAULT) are dropped.
- I_{M1r} / I_{M2r} are recomputed from I_{LA}, I_{LB}, the duties and the e-load current by the firmware's KCL (listing 3): $I_1 = I_{str} + d_{A1} I_{LA1} - d_{B1} I_{LB1}$, $I_3 = I_{str} - (i_{sh2} - d_{A2} I_{LA2} + d_{B2} I_{LB2})$, $I_2 = I_3 + I_{M2r}$.
- V_{M1}: where the telemetry V_{M1} disagrees with the 5 Hz e-load by > 2 V (200 kHz open-loop runs) it is replaced by $V_{eload} - V_{M2} - V_{M3}$; $v_{d,2}$ is the primary hardware voltage signal.
- Hardware times shorter than one telemetry frame (23 ms at 44 Hz; 70–90 ms at 11–14 Hz) are bounds, printed "$\leq$".

K.3 Hardware closed-loop steady state

Table 23: Closed-loop steady state on hardware (settled RUN_CL segments ≥ 5 s): the references $v_d = 0$, $i_c = 0$ are held with no measurable offset; σ is sample-to-sample sensing noise at 44 Hz.

Case	f_{sw}	k	T [s]	$\bar{v}_{d2}$ [V]	$\sigma(v_{d2})$	pk-pk	$\bar{i}_{c2}$ [A]	$\sigma(i_{c2})$	$\bar{i}_{d1}$ [A]	$\bar{i}_{d2}$ [A]	$\bar{P}_{load}$ [W]
B02	200	0.1	153	0.005	0.040	0.21	-0.023	1.36	0.35	-0.10	44.5
B03	200	0.3	140	0.007	0.074	0.36	0.050	0.40	0.09	-0.05	40.6
B04	200	0.3	44	0.010	0.078	0.39	0.066	0.41	0.19	0.04	40.6
B06	300	0.3	357	0.001	0.016	0.08	-0.000	0.05	0.31	-0.26	35.6
B06	300	0.3	148	0.001	0.016	0.08	-0.000	0.05	0.70	-0.49	61.7
B07	300	0.4	132	-0.002	0.069	0.27	-0.001	0.05	0.41	-0.48	61.0
B07	300	0.5	33	-0.005	0.095	0.36	0.000	0.06	0.36	-0.07	45.5
B07	300	0.6	6	0.052	1.406	11.81	0.004	0.06	0.19	-0.21	40.6
B07	300	0.4	18	-0.002	0.044	0.19	0.001	0.05	0.27	-0.12	45.8
B07	300	0.3	91	-0.000	0.013	0.06	-0.001	0.05	0.25	-0.18	44.7
B09	400	0.3	96	0.001	0.015	0.08	0.057	0.09	0.21	-0.09	49.8
B09	400	0.3	37	0.001	0.017	0.08	0.054	0.08	0.36	-0.28	63.6
B09	400	0.3	193	0.001	0.018	0.09	0.060	0.08	0.34	-0.28	62.9
B09	400	0.3	55	-0.003	0.035	0.16	0.061	0.09	0.36	-0.27	63.7
S02	200	0.1	234	0.001	0.031	0.14	-0.032	0.23	-1.51	-0.58	45.5
S04	200	0.1	262	0.008	0.149	0.70	-0.029	0.26	-0.35	-0.06	8.2
S05	200	0.2	226	0.001	0.108	0.56	-0.033	0.27	-0.44	-0.08	10.9
S06	200	0.3	186	0.002	0.088	0.41	-0.039	0.27	-0.51	-0.11	13.4
S07	200	0.4	153	-0.004	0.092	0.43	-0.045	0.28	-0.52	-0.13	13.5
S08	200	0.5	147	-0.002	0.114	0.62	-0.030	0.29	-0.43	-0.10	10.6
S09	200	0.6	275	-0.000	0.172	0.80	-0.050	0.30	-0.32	-0.02	7.3
S10	200	0.7	175	-0.002	0.138	0.71	-0.046	0.31	-0.26	-0.00	5.1
S11	200	0.8	88	-0.000	0.099	0.59	-0.048	0.31	-0.19	0.02	2.3

Table 24: +1 V reference step on v_{d2} (S4), 200 kHz gain set: fitted time constant τ, 10–90 % rise t_r and 2 % settling t_s in ms, analytical model | QSPICE switching model with the firmware DLL | Simulink carpark twin. $\tau_{design} = 1/(k\,2\pi\,150\,\text{Hz})$.

Setup	k	τ_{design}	τ	t_r	t_s	τ	t_r	t_s	OS %	τ	t_r	t_s	OS %
BENCH	0.1	10.61	10.54	23.2	41.3	9.72	19.8	30.7	0.0	10.15	22.9	41.4	0.0
BENCH	0.2	5.31	5.20	11.5	20.5	4.62	10.4	18.4	0.0	4.77	11.0	19.8	0.0
BENCH	0.3	3.54	3.42	7.6	13.6	2.84	6.5	11.9	0.0	2.97	6.9	12.4	0.0
BENCH	0.5	2.12	1.99	4.5	8.0	1.47	3.5	6.2	0.0	1.51	3.7	6.8	0.0
BENCH	1.0	1.06	0.91	2.0	3.8	0.55	1.3	2.2	0.0	0.40	0.8	2.7	8.8
PV80	0.1	10.61	12.90	28.4	50.6	12.40	22.5	32.1	0.0	11.64	26.0	45.5	0.0
PV80	0.2	5.31	6.38	14.1	25.1	5.87	12.6	21.8	0.0	5.51	12.5	22.4	0.0
PV80	0.3	3.54	4.21	9.4	16.7	3.71	8.7	15.2	0.0	3.47	8.1	14.2	0.0
PV80	0.5	2.12	2.47	5.5	9.9	1.94	4.6	17.0	7.5	1.83	4.5	8.0	0.0
PV80	1.0	1.06	1.16	2.6	4.8	0.77	1.9	8.5	4.2	0.56	0.9	3.0	0.0

Hardware cannot resolve this transient (one telemetry frame = 23 ms); its steady-state error and noise are in Table 23.

Table 25: Circulating-current channel i_{c2} during the +1 V v_{d2} reference step (S4, 200 kHz): peak deviation and recovery, i.e. the decoupling check (the current loop should barely move).

Setup	k	peak an. [A]	t_s an. [ms]	peak QSP [A]	t_s QSP [ms]	SSE QSP [A]	peak Sim [A]	t_s Sim [ms]	SSE Sim [A]
BENCH	0.1	-0.004	0.0	-0.038	0.0	0.005	-0.007	0.0	0.006
BENCH	0.2	-0.004	0.0	-0.033	0.0	0.005	-0.019	0.0	0.007
BENCH	0.3	-0.005	0.0	-0.022	0.0	0.006	-0.029	0.0	0.006
BENCH	0.4	-0.005	0.0	–	–	–	–	–	–
BENCH	0.5	-0.005	0.0	-0.030	0.0	0.007	-0.026	0.0	0.010
BENCH	1.0	-0.008	0.0	-0.025	0.0	0.005	-0.037	0.0	0.016
PV80	0.1	-0.005	0.0	-0.051	0.0	0.064	0.017	0.0	-0.010
PV80	0.2	-0.005	0.0	-0.049	0.0	0.057	0.015	0.0	-0.010
PV80	0.3	-0.005	0.0	0.072	0.0	0.041	0.019	0.0	-0.010
PV80	0.4	-0.005	0.0	–	–	–	–	–	–
PV80	0.5	-0.005	0.0	-0.106	7.0	0.075	0.018	0.0	-0.007
PV80	1.0	-0.007	0.0	0.120	0.9	0.027	-0.018	0.0	-0.002

Table 26: Shade disturbance (S3: module 3 loses 1/3 of its current, PV80 plant): simulated recovery (ms) against the detected hardware shade events of 20 Oct.

k	Signal	τ an.	t_s an.	τ QSP	t_s QSP	OS QSP	t_s Sim	HW Δ	HW peak	HW cases
0.1	Pstr	–	–	0.00	6.3	6.2	0.1	21.95	–	S02 S02
0.2	Pstr	–	–	0.00	0.1	3.9	0.1	-0.01	-0.05	S03 S05
0.3	Pstr	–	–	0.00	3.3	7.4	0.1	–	–	nan
0.5	Pstr	–	–	0.00	1.6	4.1	0.1	-0.39	0.57	S08
1.0	Pstr	–	–	0.00	0.7	6.1	0.1	–	–	nan
0.1	id2	0.08	18.9	0.07	15.6	0.0	0.2	-0.33	-1.02	S02 S02
0.2	id2	0.08	9.3	0.05	0.3	0.0	0.2	0.02	0.05	S03 S05
0.3	id2	0.08	6.1	0.07	7.5	0.0	0.2	–	–	nan
0.4	id2	0.08	4.5	–	–	–	–	–	–	nan
0.5	id2	0.08	3.6	0.06	3.0	0.0	0.2	0.01	-0.64	S08
1.0	id2	0.08	1.6	0.06	0.9	0.0	0.2	–	–	nan
0.1	vd2	–	–	–	–	–	–	0.00	0.05	S02 S02
0.2	vd2	–	–	0.01	2.0	20.5	–	-0.00	-0.25	S03 S05
0.5	vd2	–	–	–	–	–	–	-0.02	0.22	S08

K.4 Cross-phase step metrics

L Thermal Data and Oscilloscope Captures

On the bench the hottest die rose from 25–28 °C at 200 kHz to 29–31 °C at 300 and 400 kHz (fig. 34), consistent with the dead-time commutation loss behind R_{sw}, which grows as f_{sw}^3, while the open-loop residual and the current-loop gain fall; 300 kHz balanced the two. The highest reading on either day was 39 °C, in full sun at 200 kHz, far below the 85 °C trip.

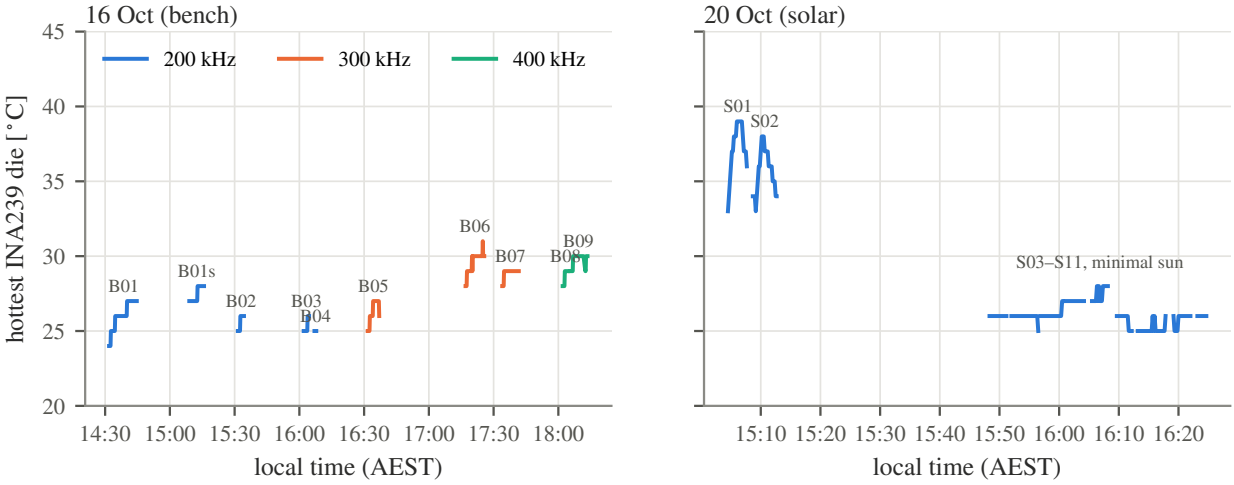


Figure 34: Hottest INA239 die temperature during every test window (10 s maxima of the telemetry), coloured by switching frequency. The over-temperature trip is at 85 °C (INA die) and 80 °C (heatsink, Pi-side E-stop).

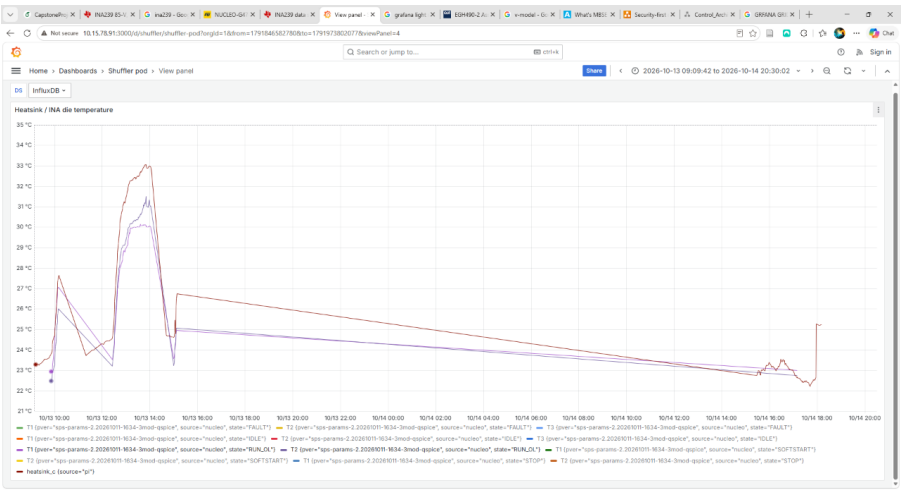


Figure 35: Grafana capture of the 100 kHz open-loop exploration on 13–14 Oct, which ran but behaved erratically.

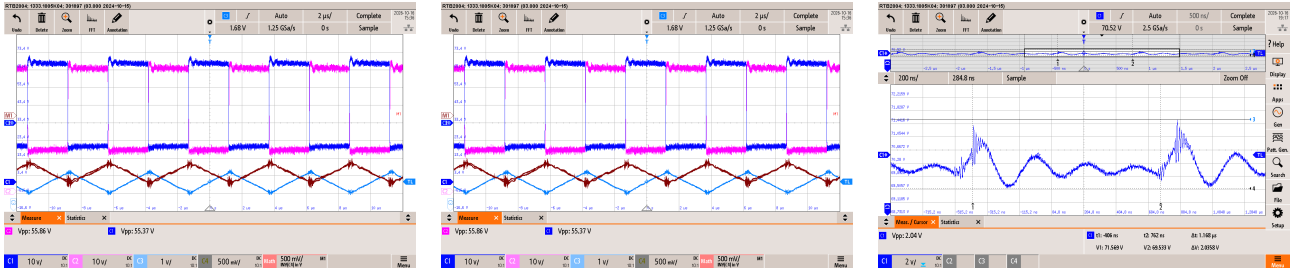
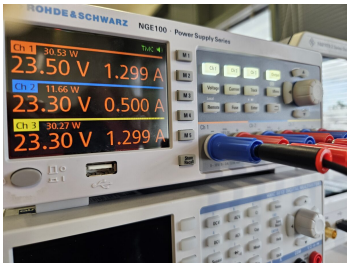
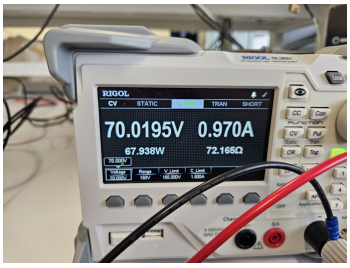


Figure 36: Oscilloscope captures of the switch-node waveforms at 200, 300 and 400 kHz (dead time and 180° interleave check).

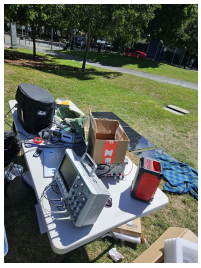
M The Journey



(a) NGE103 PSU: 23.30 V / 1.299 A, 23.30 V / 0.500 A, 23.30 V / 1.299 A.

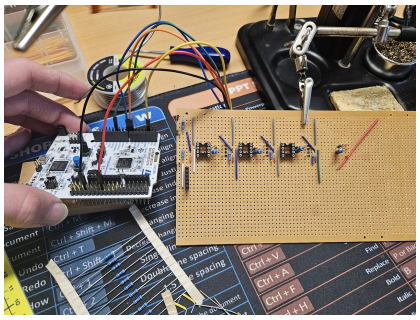


(b) DL3021 e-load in CV: 70.02 V, 0.970 A, 67.9 W.



(c) Carpark string on 20 Oct 2026.

Figure 37: Photographs of the two set-ups (more in appendix M).



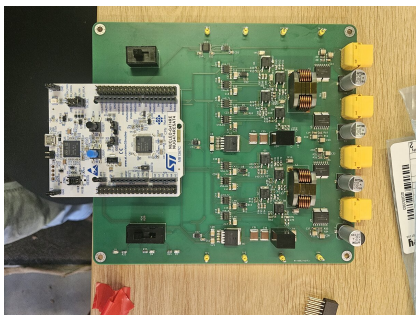
(a) Aug: first proto-board with the Nucleo.



(b) Sep: the Version 0 PCB arrives.



(c) Oct 4: reflow in the PCB oven.



(d) Oct 4: assembled board with the Nucleo.



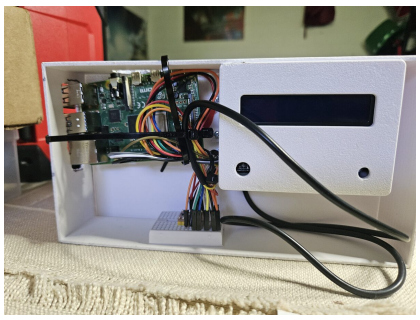
(e) Oct 4: the hardware teammate with the board.



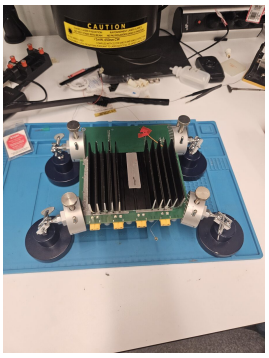
(f) Oct 6: heatsink.



(g) Oct 7: enclosure work.



(h) Oct 11: pod with the Pi, LCD and IR receiver.

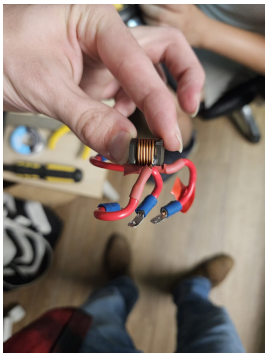


(i) Oct 14: heatsink mounted, thermal paste.

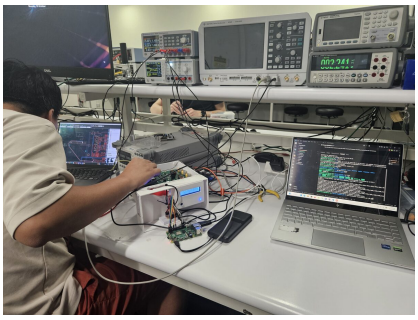
Figure 38: Build photographs (1 of 2).



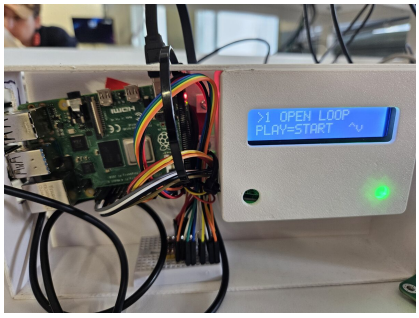
(a) Oct 14: U10, the top INA239 behind the reversed isolator.



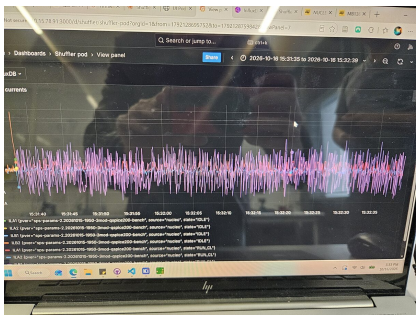
(b) Oct 14: ERUC23 coupled inductor rework.



(c) Oct 12: first bench runs.



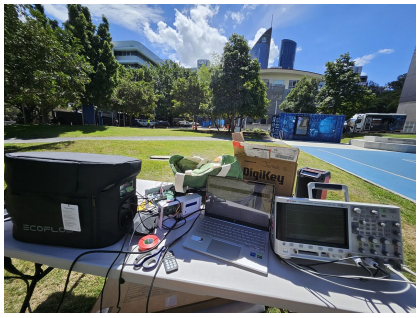
(d) Oct 16: pod LCD during the bench test.



(e) Oct 16: Grafana live during closed loop.



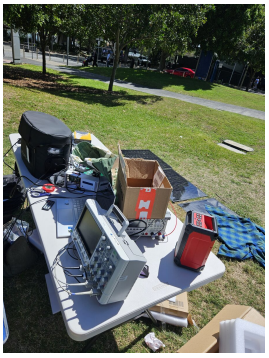
(f) Oct 14: late-night bring-up.



(g) Oct 20: carpark set-up.



(h) Oct 20: it works in the sun.



(i) Oct 20: the string.

Figure 39: Build and test photographs (2 of 2). Videos of the bench runs are in the project repository.